



KDT Cloud

쿠버네티스 심화 (Kubernetes Advanced)

2026.10

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



1. 쿠버네티스 아키텍처 심화
2. 워크로드 컨트롤러 심화
3. 스케줄링과 리소스 관리
4. 네트워킹 심화
5. 스토리지와 상태 관리
6. 설정과 Secret 관리
7. 보안 심화
8. 관측성: 로그 · 메트릭 · 추적
9. GitOps 와 배포 전략
10. 운영 · 트러블슈팅 · 비용

1. 쿠버네티스 아키텍처 심화

- 컨트롤 플레인 4개 구성요소는 각각 무엇을 책임지는가
- `kubectl apply` 한 줄이 파드가 되기까지의 전체 경로
- 선언형 API 와 조정 루프. 쿠버네티스가 동작하는 원리
- 데이터 플레인: kubelet · 컨테이너 런타임 · CRI/CNI/CSI

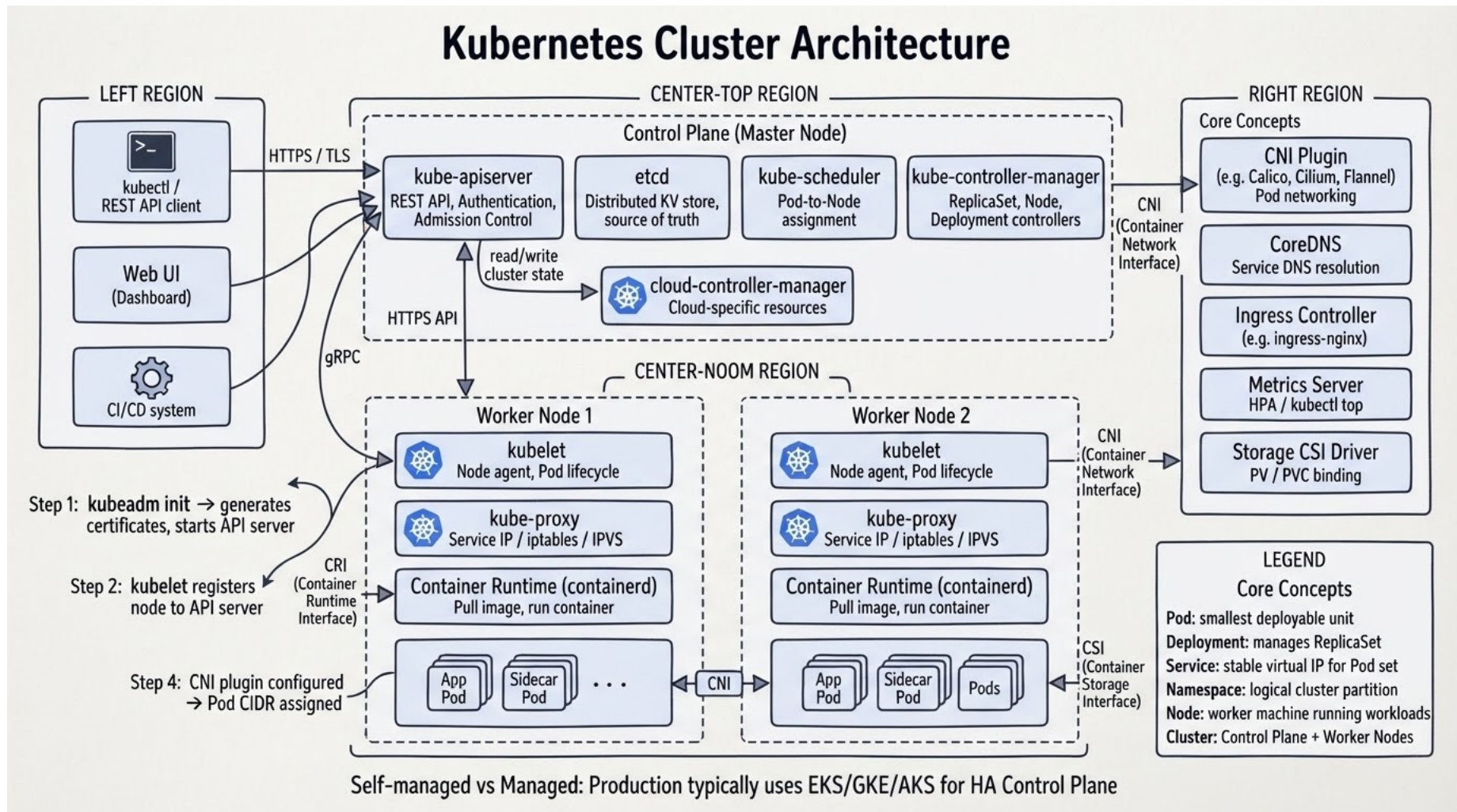
쿠버네티스가 동작하는 방식

원하는 상태를 적어 두면 시스템이 계속 그 상태로 맞춘다.

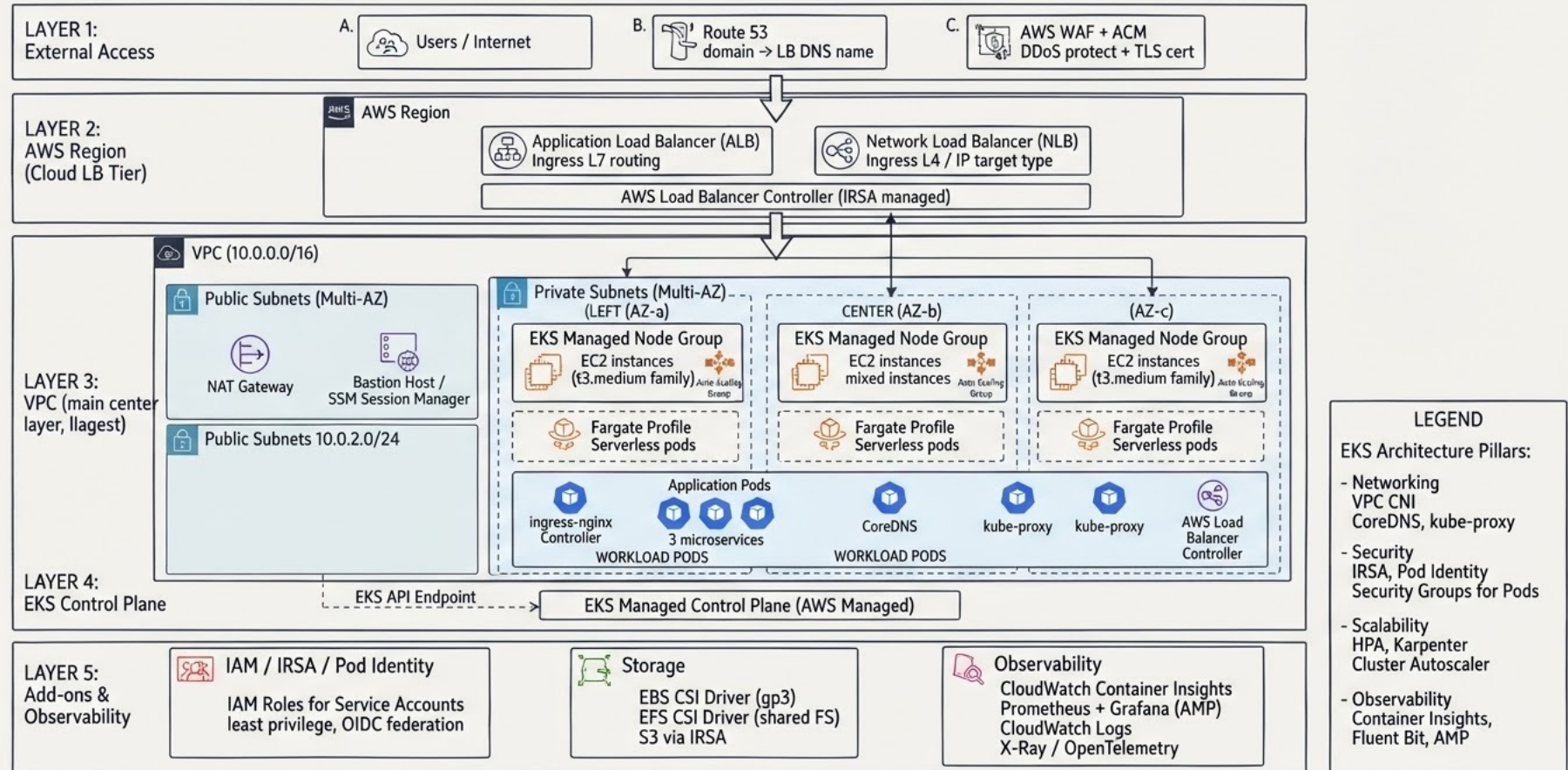
- 선언형이다. '만들어라' 가 아니라 '**3개인 상태여야 한다**'
- 자가치유(self-healing) · 확장 · 롤링 업데이트 · GitOps
- 위 넷이 전부 이 원리 하나에서 나온다



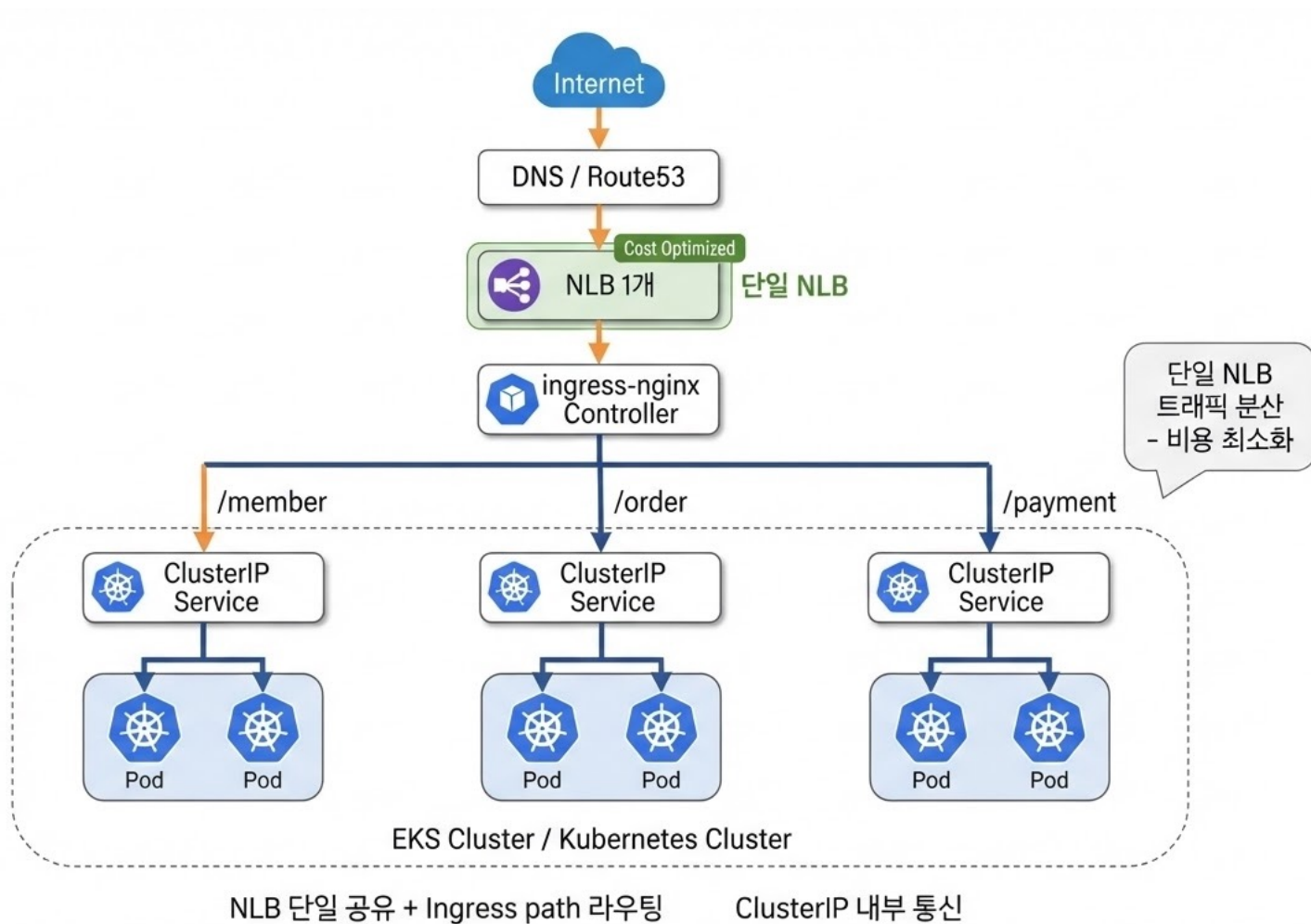
왼쪽 하나를 적으면 가운데 조정 루프가 오른쪽 4개를 알아서 만들어 낸다.



Amazon EKS - Reference Architecture



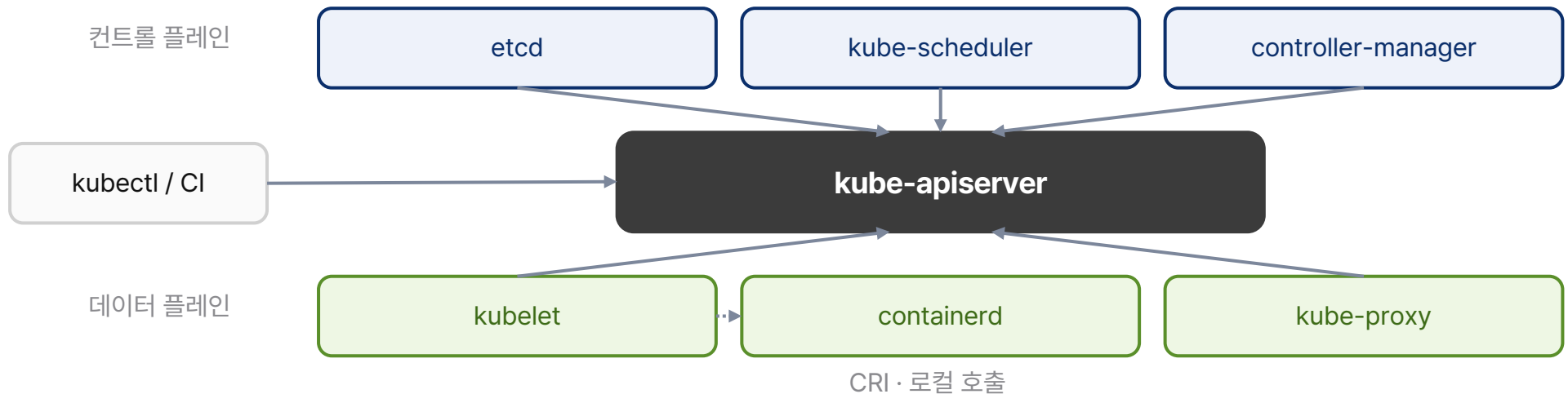
Recommended for production: Multi-AZ, Multi-Node Group, IRSA, GitOps (ArgoCD/Flux)



클러스터의 두 영역

클러스터는 결정하는 쪽(컨트롤 플레인) 과 실행하는 쪽(데이터 플레인) 으로 나뉜다.

- 모든 통신은 **API 서버를 거친다**. 구성요소끼리 직접 호출하지 않는다
- 이 구조 덕분에 인증·인가·감사 로그가 **한 곳에서 처리된다**



유일한 진입점 kube-apiserver

클러스터를 바꾸는 방법은 **API 서버에 REST 요청을 보내는 것** 하나뿐이다.

- `kubectl` 도 컨트롤러도 **같은 REST API** 를 부른다 (특권 경로 없음)
- **수평 확장 가능한 유일한 컨트롤 플레인 구성요소**이다 (상태를 etcd 에 두기 때문)

```
# kubectl 이 실제로 무엇을 보내는지 본다: -v=8 이면 HTTP 요청이 그대로 보인다
kubectl get pods -n $NS -v=8 2>&1 | grep -E 'GET|Response Status'
# GET https://XXXX.eks.amazonaws.com/api/v1/namespaces/shop/pods?limit=500
# Response Status: 200 OK
```

```
# API 를 직접 부를 수도 있다. kubectl 은 편의 도구일 뿐이다
kubectl get --raw /api/v1/namespaces/shop/pods | head -c 300
```

```
# 이 클러스터가 제공하는 모든 리소스 종류 - CRD 를 설치하면 여기 늘어난다
kubectl api-resources --namespaced=true -o wide | head -12
```

```
# 어떤 API 그룹/버전이 살아 있는가 (deprecated 확인의 출발점)
kubectl api-versions | grep -E 'apps|networking|batch'
```

API 요청의 수명주기

`kubectl apply` 한 줄은 저장되기 전에 네 단계의 검사를 통과한다.

- 고정된 순서: 인증 → 인가 → Admission → 스키마 검증 → etcd 저장
- 실패 지점마다 HTTP 상태 코드가 달라서 코드만 봐도 어디서 막혔는지 인지할 수 있다

단계	무엇을 보나	실패 시	대표 원인
① 인증(Authentication)	너는 누구인가	401	kubeconfig 만료 · IAM 매핑 누락
② 인가(Authorization)	그것을 할 권한이 있는가	403	RBAC Role/Binding 누락
③ Admission (Mutating)	값을 채워 넣을 것이 있나	없음	사이드카 주입 · 기본값 설정
④ Admission (Validating)	정책을 어기지 않았나	400/403	PSS 위반 · Kyverno 정책 거부
⑤ 스키마 검증	필드 이름·타입이 맞나	422	오타 · 잘못된 apiVersion
⑥ etcd 저장	충돌 없이 저장할 수 있나	409	resourceVersion 충돌(동시 수정)

[기초] 분산 합의가 필요한 이유

저장소를 여러 대로 확장시키면 무엇이 정확한 값 인지에 대한 문제가 새로 발생한다.

- 한 대만 두면 그 한 대가 멈출 때 전체가 멈춘다
- 여러 대로 늘리면 값이 갈린다
- 그래서 **누구를 믿을지**를 정하는 규칙이 필요하다

대수	과반수	견딜 수 있는 고장	사용하는 이유
1대	1	0대	실습용. 그 한 대가 곧 단일 장애점
2대	2	0대	의미가 없다. 한 대만 빠져도 과반수가 깨진다
3대	2	1대	운영 최소 구성
5대	3	2대	대규모. 대신 쓰기가 느려진다
4대	3	1대	3대와 같은 내구성에 비용만 늘어난다

etcd, 클러스터의 유일한 저장소

클러스터의 모든 상태가 **etcd** 한 곳에 있어서, etcd 를 복구하면 클러스터가 복구된다.

- 분산 합의(Raft) 기반 KV 저장소이다. 홀수 노드(3·5)로 구성해 **과반수**로 결정한다
- 쓰기가 느려서 **대용량·고빈도 쓰기를 넣으면 안 된다**

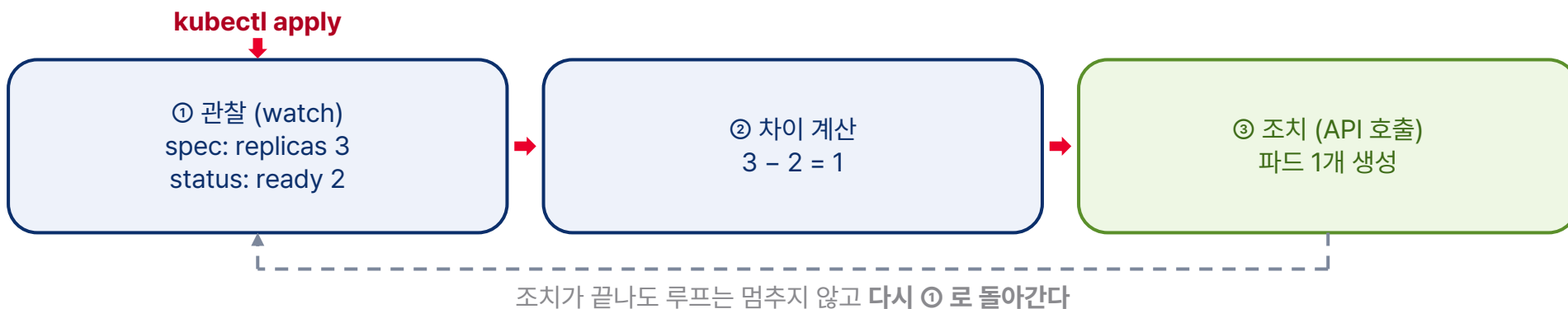
특성	값 / 규칙	실무에서 의미하는 것
객체 크기 한도	기본 1.5 MiB	큰 ConfigMap/Secret 을 넣으면 저장 자체가 실패한다
권장 DB 크기	2 GB (최대 8 GB)	이벤트·리소스가 쌓이면 성능이 급락한다
합의 노드 수	3 또는 5 (홀수)	3대 중 2대가 살아야 쓰기가 된다. 2대 구성은 무의미
백업 방식	스냅샷 (<code>etcdctl snapshot save</code>)	EKS 는 AWS 가 자동 수행. 우리가 할 일 없음
암호화	저장 시 암호화는 별도 설정	Secret 은 기본적으로 평문에 가깝게 저장된다
watch	변경 스트림을 구독	모든 컨트롤러가 이 위에서 동작한다

[주의] ConfigMap 에 대용량 파일(모델·덤프)을 넣는 시도는 **1.5 MiB 한도 때문에** 반드시 실패한다.

선언형 API 와 조정 루프

쿠버네티스는 명령을 실행하는 대신 원하는 상태를 적어 두면 계속 맞춰 준다.

- 명령형: '파드 3개 만들어' → 한 번 하고 끝
- 하나가 중단돼도 아무도 모른다
- 선언형: '파드가 3개여야 해' → 차이가 생길 때마다 무한히 맞춘다



파드를 지워도 다시 생기는 것은 이 고리 때문이다. 지워진 뒤 ②에서 차이가 잡히고, ③이 새로 만든다.

[정리] spec 은 사람이 적는 곳이고 status 는 컨트롤러가 적는 곳이라 손으로 수정해도 곧 덮어써진다.

컨트롤러 매니저의 구조

controller-manager 하나의 프로세스 안에서 수십 개의 컨트롤러 루프가 동시에 실행된다.

- 각 컨트롤러가 **한 종류의 리소스만** 책임진다 (단일 책임 원칙)
- 우리가 만들 오퍼레이터도 **정확히 같은 패턴**의 컨트롤러이다

컨트롤러	지켜보는 것	하는 일	중단되면
Deployment	Deployment	ReplicaSet 생성·교체	롤링 업데이트가 멈춘다
ReplicaSet	ReplicaSet	파드 개수 유지	중단된 파드가 다시 생성되지 않는다
Node	Node 하트비트	NotReady 판정·파드 축출	중단된 노드의 파드가 옮겨지지 않는다
Endpoint(Slice)	Service · Pod	서비스 뒤 파드 목록 갱신	트래픽이 중단된 파드로 간다
Job	Job	완료까지 파드 재시도	배치가 끝나도 정리 안 됨
PV / PVC 바인더	PVC	PVC ↔ PV 연결	볼륨은 있는데 바인딩되지 않는다
ServiceAccount	Namespace	<code>default</code> SA 자동 생성	새 네임스페이스에 파드가 생성되지 않는다

[참고] 노드 하트비트가 끊기면 기본 **40초**에 NotReady, **5분** 뒤에 파드 축출이 시작된다.

스케줄러의 두 단계

스케줄러는 노드를 거르고(Filter) 점수 매기는(Score) 두 단계로 가장 높은 곳을 고른다.

- 1단계 **Filter**: 못 놓는 노드를 전부 제외한다 (리소스·테인트·어피니티·볼륨 존)
- 2단계 **Score**: 남은 노드에 점수를 매긴다 (여유 자원·분산·이미지 캐시)

```
# Pending 파드의 진짜 이유는 이벤트에 있다 - 필터 탈락 사유가 노드 수와 함께 찍힌다
kubectl describe pod shop-api-7d9f-x2k4 -n $NS | tail -6
# Warning FailedScheduling 0/6 nodes are available:
#   2 Insufficient cpu,
#   3 node(s) had intolerated taint {workload: batch},
#   1 node(s) didn't match pod affinity rules.

# 위 문장 읽는 법: 6개 노드 전부 탈락. 원인이 세 종류로 나뉜다
#   - Insufficient cpu      → requests 를 낮추거나 노드를 늘린다
#   - intolerated taint     → toleration 을 붙인다
#   - didn't match affinity → 어피니티 조건을 완화한다

# 스케줄링이 끝난 뒤 어디로 갔는지 확인
kubectl get pod -n $NS -o wide --sort-by=.spec.nodeName
```

[체크] Pending 이면 `describe` 의 **FailedScheduling** 한 줄만 정확히 읽어도 원인이 나온다.

kubelet, 노드의 대리인

kubelet 은 API 서버가 시킨 일을 노드에서 실제로 하는 유일한 주체이다.

- `spec.nodeName` 이 자기 이름인 파드만 감시한다
- 컨테이너를 직접 만들지 않고 **CRI** 로 **containerd** 에 시킨다

kubelet 의 책임	구체적으로	실패했을 때 보이는 증상
파드 동기화	배정된 파드를 만들고 지운다	노드 NotReady → 파드가 안 뜨거나 안 지워짐
이미지 관리	레지스트리에서 pull, 오래된 것 정리	ImagePullBackOff · ErrImagePull
볼륨 마운트	CSI 에 attach/mount 요청	ContainerCreating 에서 멈춤
Probe 실행	liveness/readiness/startup 검사	CrashLoopBackOff · 트래픽 미유입
상태 보고	노드·파드 상태를 API 서버에 갱신	상태가 낡은 값으로 굳는다
리소스 관리	cgroup 설정, 축출(eviction) 판단	OOMKilled · Evicted
노드 자원 확보	디스크·메모리 압박 시 파드 축출	DiskPressure · MemoryPressure

[주의] kubelet 이 멈춰도 이미 실행 중인 컨테이너는 계속 실행되지만, 아무것도 갱신되지 않고 Probe 도 멈춘다.

컨테이너 런타임과 CRI

쿠버네티스는 **CRI** 규격을 만족하는 런타임이면 무엇이든 사용한다.

- 1.24 의 dockershim 제거 이후 EKS 표준은 **containerd**
- **이미지 형식은 그대로**라 Docker 로 빌드한 것도 그대로 실행된다

```
# 이 노드가 무슨 런타임을 쓰는가
kubectl get nodes -o wide | awk '{print $1, $NF}'
#   ip-10-0-1-23...   containerd://1.7.x

# 노드에 직접 붙어서 컨테이너를 본다 (docker 명령은 없다. crictl 을 쓴다)
#   $ sudo crictl ps                # docker ps   에 해당
#   $ sudo crictl images             # docker images
#   $ sudo crictl logs <container>  # docker logs

# 왜 알아야 하나 - 이미지가 노드 디스크를 채워 DiskPressure 가 날 때
#   $ sudo crictl rmi --prune        # 안 쓰는 이미지 정리
#   kubelet 도 자동 정리하지만 임계값(85%)에 도달해야 시작한다
```

[참고] 'Docker 지원 중단'은 런타임 이야기이다. Dockerfile 과 이미지는 계속 사용한다. 이 오해가 여전히 많다.

파드가 만들어지기까지

`kubectl apply` 부터 컨테이너가 Running 이 되기까지 **8개의** 홉을 지난다.

- 각 홉마다 **멈출 수 있는 지점**이 있고, 멈춘 위치가 파드 상태로 드러난다

#	무엇이	무엇을 한다	여기서 멈추면
1	kubectl	매니페스트를 REST 요청으로 변환	인증 오류(401)
2	API 서버	인가 · Admission · 검증	403 / 422
3	etcd	Deployment 저장	쓰기 충돌(409)
4	Deployment 컨트롤러	ReplicaSet 생성	ReplicaSet 이 안 생김
5	ReplicaSet 컨트롤러	파드 객체 생성 (nodeName 없음)	파드가 안 생김
6	스케줄러	노드 선택 → <code>spec.nodeName</code> 기록	Pending
7	kubelet	이미지 pull · 볼륨 · CNI · 컨테이너 시작	ContainerCreating
8	kubelet	Probe 통과 → Ready 보고	Running 이지만 0/1 Ready

[정리] 파드 상태가 곧 어느 홉에서 멈췄는지를 말해 준다 (Pending=6번, ContainerCreating=7번, 0/1 Ready=8번).

CRI · CNI · CSI 세 확장점

쿠버네티스는 컨테이너 실행 · 네트워크 · 스토리지를 규격만 정하고 위임한다.

- 세 인터페이스 덕분에 온프레임과 클라우드가 같은 매니페스트로 동작한다
- 바꿔 말하면, 이 세 곳은 환경마다 다르게 동작하는 지점이기도 하다

인터페이스	책임	EKS 기본 구현	여기서 나는 대표 장애
CRI (Container Runtime)	컨테이너 실행·이미지	containerd	ImagePullBackOff · 디스크 부족
CNI (Network)	파드 IP 할당·경로	Amazon VPC CNI	IP 고갈 → 파드 Pending
CSI (Storage)	볼륨 생성·연결·마운트	EBS CSI / EFS CSI	AZ 불일치로 attach 실패

[주의] 세 가지 모두 애드온으로 별도 설치·업그레이드되므로, 클러스터 버전만 올리고 방치하면 호환성 문제가 발생한다.

EKS 에서 달라지는 것

관리형 컨트롤 플레인 은 **책임의 경계**를 바꾼다. 무엇을 안 해도 되고 무엇이 여전히 우리 몫인지 가른다.

- AWS 가 하는 일: 컨트롤 플레인 **HA · 패치 · 백업 · 확장**
- 여전히 우리 일: **노드 · 애드온 · 워크로드 · 네트워크 설계 · 비용**

항목	self-managed (kubeadm)	EKS	우리가 할 일
API 서버 HA	직접 3대 구성 + LB	자동 (다중 AZ)	없음
etcd 백업	cron + etcdctl snapshot	자동	없음
컨트롤 플레인 업그레이드	노드마다 수동	콘솔/CLI 한 번	호환성 사전 점검
노드 업그레이드	직접	관리형 노드그룹이 롤링	PDB 설정
애드온(CNI/CSI/CoreDNS)	직접 설치·관리	EKS Add-on 으로 관리 가능	버전 정합성 유지
인증	인증서 / OIDC	IAM + aws-auth / Access Entry	IAM 매핑 관리
감사 로그	직접 수집	CloudWatch 로 옵트인	켜 두기 (기본 꺼짐)

[주의] EKS 감사 로그는 기본이 꺼짐이라, 사고가 난 뒤에 켜면 그 이전 기록은 영원히 없다.

아키텍처 관점 장애 진단 지도

증상을 구성요소로 번역하는 표. 사흘 내내 이 표로 돌아온다.

- 진단 순서: 파드 상태 → 이벤트 → 담당 구성요소 → 그 구성요소의 로그

증상	의심 구성요소	첫 명령	자주 나오는 원인
Pending 지속	스케줄러	<code>describe pod</code> → FailedScheduling	리소스 부족 · 테인트 · PVC AZ
ContainerCreating 지속	kubelet · CNI · CSI	<code>describe pod</code> 이벤트	IP 고갈 · 볼륨 attach 실패
ImagePullBackOff	kubelet · 레지스트리	<code>describe pod</code> 이벤트	태그 오타 · ECR 권한 없음
CrashLoopBackOff	애플리케이션 · Probe	<code>logs --previous</code>	설정 누락 · liveness 과민
Running 0/1 Ready	readinessProbe	<code>describe pod</code> Conditions	엔드포인트 경로·타임아웃
Terminating 지속	finalizer · 컨트롤러	<code>get -o yaml finalizers</code>	컨트롤러 부재 · 볼륨 detach
Evicted 다발	kubelet 축출	<code>describe node</code> Conditions	Disk/MemoryPressure
<code>kubectl</code> 느림/타임아웃	API 서버 · etcd	<code>get --raw /readyz?verbose</code>	대량 watch · etcd 지연

[정리] 이 표에 없는 증상이 나오면 8쪽 표로 돌아가 어느 홈인지부터 정한다.

[핵심 요약] 1장 아키텍처

증상을 구성요소로 번역하는 좌표계. 이 장은 그것 하나를 남긴다.

항목	한 줄 정리	실무 포인트
두 개의 평면	결정(컨트롤)과 실행(데이터)이 분리돼 있다	컨트롤 플레인 장애 = 변경 중단이지 서비스 중단이 아니다
허브-스포크	모든 통신이 API 서버를 거친다	감사·인증·인가를 한 곳에서 본다
요청 6단계	인증→인가→Admission→검증→저장	상태 코드로 실패 지점을 특정한다 (401/403/422/409)
조정 루프	spec 과 status 의 차이를 무한히 메운다	파드를 지워도 다시 생기는 이유. status 는 수정하지 않는다
스케줄러 2단계	Filter 로 거르고 Score 로 고른다	Pending 은 FailedScheduling 한 줄에 답이 있다
kubelet	노드에서 실제로 일하는 유일한 주체	kubelet 이 중단돼도 컨테이너는 계속 실행된다(=더 위험하다)
8홉 경로	apply 부터 Running 까지 8단계	파드 상태 = 멈춘 홉 번호
EKS 경계	컨트롤 플레인 = AWS, 나머지는 우리	감사 로그는 기본 꺼짐. 반드시 켜다

[체크] '지금 우리 클러스터에서 파드가 Pending 이면 나는 첫 명령을 무엇으로 시작하는가?' 답이 바로 나오면 통과.

2. 워크로드 컨트롤러 심화

- 파드 생명주기. 시작·종료·재시작이 실제로 어떻게 진행되는가
- Deployment 무중단 배포의 조건 네 가지
- StatefulSet · DaemonSet · Job · CronJob 을 언제 고르는가
- 워크로드 타입 선택 결정표와 진단 지도

[기초] 라벨과 셀렉터의 연결

쿠버네티스에는 리소스끼리 가리키는 **포인터가 없고**, 라벨이 그 자리를 대신한다.

- 리소스에 **key: value** 라벨을 붙이고 다른 리소스가 그 라벨로 대상을 고른다.

이름이 아니라 조건이므로 대상이 늘거나 줄어도 설정을 수정하지 않는다

```
# ① 파드에 라벨을 붙인다
metadata:
  labels:
    app: shop-api          # '나는 shop-api 이다'
    tier: backend

# ② Service 가 그 라벨로 대상을 고른다 - 이름을 적지 않는다
spec:
  selector:
    app: shop-api          # 이 라벨이 붙은 파드 '전부'

# 파드가 3개든 30개든 이 두 줄은 그대로이다.
# 새 파드가 같은 라벨을 달고 뜨면 자동으로 대상에 들어온다.

# ③ kubectl 도 같은 방식으로 고른다
kubectl get pod -l app=shop-api
```

[주의] 셀렉터와 라벨의 오타는 오류를 내지 않고 대상이 0개가 될 뿐이라 조용히 실패한다.

파드를 직접 만들지 않는 이유

`kind: Pod` 로 만든 파드에는 아무 보호 장치가 없다. 이것이 컨트롤러가 존재하는 이유이다.

- 직접 만든 파드는 중단되거나 노드가 사라지면 그대로 없어진다
- 컨트롤러가 붙어야 '몇 개를 유지하라' 가 살아 있다
- 예외: 일회성 디버깅 파드(`kubectl run --rm -it`)

상황	직접 만든 파드	Deployment 가 관리하는 파드
프로세스가 죽음	restartPolicy 로 재시작은 됨	재시작 + 반복되면 이벤트로 드러남
노드가 죽음	영원히 사라짐	다른 노드에 5분 뒤 재생성
<code>kubectl delete pod</code>	사라짐	즉시 새 파드 생성
이미지 교체	지우고 다시 만들어야 함	<code>set image</code> 한 줄로 롤링 교체
개수 조절	손으로 하나씩	<code>scale --replicas=N</code>
이전 버전 복귀	불가능	<code>rollout undo</code> 로 즉시

[주의] 운영 클러스터의 `kind: Pod` 매니페스트는 대개 실수이니 **의도인지 반드시 확인한다.**

파드 phase 다섯 가지

우리가 흔히 보는 상세 상태는 phase 가 아니라 컨테이너 상태이다.

- `kubectl get pods` 의 STATUS 열은 phase 가 아니다
- 더 자세한 합성값이다
- 그래서 CrashLoopBackOff 는 Running phase 안의 **컨테이너 상태**이다

컨테이너 상태(state). 이 세 가지가 STATUS 열에 섞여 나온다

phase	의미	여기서 오래 머물면
Pending	아직 노드에 배정되지 않았거나 이미지 준비 중	스케줄러 이벤트 또는 이미지 pull 확인
Running	노드에 배정되고 컨테이너가 최소 1개 살아 있음	정상. 단 READY 0/1 이면 Probe 확인
Succeeded	모든 컨테이너가 0 으로 정상 종료	Job 의 목표 상태
Failed	컨테이너가 0 이 아닌 코드로 종료, 재시작 안 함	<code>logs --previous</code> 로 종료 직전 확인
Unknown	노드와 통신 불가	노드 상태·kubelet 확인

[정리] `Waiting`(이유: `ImagePullBackOff` 등) · `Running` · `Terminated`(종료 코드) 세 가지가 컨테이너 상태이다.

파드의 종료 과정

종료는 즉시 중단시키는 것이 아니다. 유예 시간 동안 정리할 기회를 준다. 이 구간을 이해해야 무중단이 된다.

- 삭제 요청 → 동시에 ①엔드포인트 제거 ②preStop 실행 ③SIGTERM 전송
- 유예 시간은 기본 **30초**
- 지나면 정리할 기회 없이 SIGKILL

```
spec:
  terminationGracePeriodSeconds: 45  # 기본 30. 앱의 최대 요청 처리 시간 + 여유
  containers:
  - name: shop-api
    lifecycle:
      preStop:
        exec:
          # 왜 필요한가: 엔드포인트 제거가 kube-proxy 에 퍼지는 데 시간이 걸린다.
          # 그 사이에 들어온 요청을 받아 주려면 SIGTERM 을 늦춰야 한다.
          command: ["sh", "-c", "sleep 8"]
    # 앱은 SIGTERM 을 받으면: 새 요청 거부 → 진행 중 요청 완료 → 커넥션 정리 → exit 0
```

[주의] preStop 없이 배포하면 **엔드포인트 전파 지연** 때문에 롤링 업데이트 중 502 가 발생한다.

init 컨테이너와 사이드카

본 컨테이너가 시작되기 전에 반드시 끝나야 하는 일은 init 컨테이너로 분리한다.

- init 컨테이너는 **순서대로 하나씩** 실행된다
- 전부 성공해야 본 컨테이너가 뜬다
- 1.29+ 는 **restartPolicy: Always** 인 init 컨테이너가 네이티브 사이드카이다

```
spec:
  initContainers:
    - name: wait-for-db          # DB 가 뜰 때까지 대기 - 앱에 재시도를 안 넣어도 된다
      image: busybox:1.36
      command: ['sh', '-c', 'until nc -z shop-db 5432; do sleep 2; done']
    - name: db-migrate          # 순서 보장: wait-for-db 성공 후에 실행된다
      image: myrepo/shop-migrator:1.4
      args: ["migrate", "--url", "${DB_URL}"]
  containers:
    - name: shop-api
      image: myrepo/shop-api:1.4
```

[참고] 마이그레이션을 init 컨테이너에 두면 **replicas** 수만큼 **동시 실행**되므로, Lock 이 없으면 Job 으로 개별 분리한다.

Deployment 의 ReplicaSet 교체

Deployment 는 **ReplicaSet** 을 만들고 새 버전과 이전 버전의 replicas 를 조절할 뿐이다.

- 파드 템플릿이 바뀌면 새 ReplicaSet 이 만들어진다
- 이전 ReplicaSet 은 `replicas: 0` 으로 남는다
- 이것이 롤백이 즉시 되는 근거이다

```
kubectl get rs -n $NS -l app=shop-api
```

# NAME	DESIRED	CURRENT	READY	AGE	
# shop-api-6b4c9d8f7d	3	3	3	2m	← 새 버전 (1.4)
# shop-api-5f8b7c6a9e	0	0	0	1h	← 이전 버전 (1.3), 롤백용
# shop-api-4d7a6b5c8f	0	0	0	3h	← 그 이전 (1.2)

몇 개까지 남길지 - 기본 10. 너무 많으면 etcd 를 채운다

```
kubectl patch deploy shop-api -n $NS \
  -p '{"spec":{"revisionHistoryLimit":5}}'
```

리비전 목록과 변경 사유

```
kubectl rollout history deploy/shop-api -n $NS
```

[정리] `revisionHistoryLimit` 을 0 으로 두면 롤백이 불가능해지므로 **3~5** 가 실무 기본값이다.

롤링 업데이트 파라미터

`maxSurge` 와 `maxUnavailable` 두 숫자가 교체 속도와 안전 사이의 균형을 정한다.

- **maxSurge**: 목표 개수를 얼마나 초과해도 되는가 (속도)
- **maxUnavailable**: 동시에 몇 개까지 없어도 되는가 (안전)
- **maxUnavailable**: 0 이면 용량이 줄어들지 않는다

설정	replicas=4 일 때	특징	언제 쓰나
surge 25% / unavail 25% (기본값)	최대 5개, 최소 3개	적당히 빠르고 적당히 안전	대부분의 스테이트리스 서비스
surge 1 / unavail 0	최대 5개, 최소 4개	용량이 절대 줄지 않는다. 느림	피크 시간대 배포 · 여유 없는 서비스
surge 100% / unavail 0	최대 8개, 최소 4개	가장 빠르고 안전. 리소스 2배	노드 여유가 충분할 때
surge 0 / unavail 1	최대 4개, 최소 3개	리소스 추가 없음. 용량 감소	노드가 꽉 찼을 때
Recreate	0개까지 내려감	다운타임 발생	동시 실행이 불가능한 앱

배포 전략 비교

쿠버네티스가 기본 제공하는 것은 두 가지(Recreate, RollingUpdate)뿐이고, 나머지는 도구나 트래픽 분할로 만든다.

- 블루-그린과 카나리는 **Deployment** 기능이 아니다
- 블루-그린과 카나리는 Service 선택어나 Ingress 가중치로 구현한다

전략	방식	장점	대가
Recreate	전부 내리고 전부 올림	가장 단순. 버전 혼재 없음	다운타임
RollingUpdate	점진적으로 교체	무중단 가능. 기본값	두 버전이 잠시 공존
Blue-Green	새 세트 전체를 기동하고 한 번에 전환	즉시 롤백. 버전 혼재 없음	리소스 2배 · DB 스키마 호환 필요
Canary	일부 트래픽만 새 버전으로	실사용자로 검증. 영향 최소	트래픽 분할 장치 필요 · 관측 필수
A/B	특정 조건(헤더·사용자)만 새 버전	타겟 실험 가능	라우팅 규칙 복잡

블루그린의 셀렉터 전환

두 세트를 동시에 실행해 두고 **Service 셀렉터만 바꿔** 한순간에 전환한다.

- **Deployment 두 개와 Service 하나**면 되고 추가 도구가 필요 없다.
롤백은 셀렉터를 되돌리는 것이라 즉시 끝나지만, 배포 중 리소스가 2배이다

```
# 라벨에 version 을 하나 더 둔다. app 은 같고 version 만 다르다
Deployment shop-api-blue    labels {app: shop-api, version: blue}    replicas 4
Deployment shop-api-green   labels {app: shop-api, version: green}   replicas 4
    green 은 떠 있지만 아직 트래픽이 가지 않는다

# Service 가 version 까지 보고 고른다 - 이 한 줄이 스위치이다
kind: Service
spec:
  selector: {app: shop-api, version: blue}

# 전환 - 셀렉터만 바꾸면 엔드포인트가 통째로 교체된다
SEL='{ "spec": { "selector": { "app": "shop-api", "version": "green" } } }'
kubectl patch svc shop-api -n $NS -p "$SEL"

# 롤백 - version 을 blue 로 되돌린다. blue 는 아직 살아 있다
```

[주의] 전환 순간 기존 커넥션은 유지된다. 옛 파드를 바로 지우면 처리 중인 요청이 끊긴다 (커넥션).

카나리의 트래픽 분할

일부만 새 버전으로 보낸다. 나누는 방법이 두 가지이고 정밀도가 다르다.

- 레플리카 비율은 파드 개수로만 조절해 정밀도가 낮고, **Nginx 어노테이션 두 줄**이면 퍼센트를 직접 지정한다

```
# — 방법 A. 레플리카 비율 (추가 도구 없음) —————
kind: Service
spec:
  selector: {app: shop-api}          # version 없음 -> 두 세트가 함께 대상
  shop-api-stable replicas 9          # 약 90%
  shop-api-canary replicas 1          # 약 10%
  # 10% 를 만들려면 9:1. 1% 는 99:1 이라 현실적이지 않다

# — 방법 B. Ingress 가중치 (nginx) - 퍼센트를 직접 적는다 —————
nginx.ingress.kubernetes.io/canary: "true"
nginx.ingress.kubernetes.io/canary-weight: "10"

# — 방법 C. 헤더로 지정 (nginx) - 비율을 열기 전에 확인한다 —————
nginx.ingress.kubernetes.io/canary: "true"
nginx.ingress.kubernetes.io/canary-by-header: "X-Canary"
nginx.ingress.kubernetes.io/canary-by-header-value: "always"
# 헤더를 단 요청만 새 버전으로 간다. QA 확인용
```

[체크] 카나리는 관측이 없으면 의미가 없다. 10% 에 오류가 나는지 볼 수 없으면 천천히 배포하는 것일 뿐이다.

롤링 · 블루그린 · 카나리 비교

세가지 모두 무중단이지만 위험을 다루는 방식과 비용이 상이하다.

- 쿠버네티스가 기본으로 주는 것은 **롤링 하나뿐**이라 나머지는 만들어야 한다

	롤링 업데이트	블루그린	카나리
구현 수단	Deployment 기본	Deployment 2 + 셀렉터	Ingress 가중치 또는 레플리카 비
전환 방식	점진적으로 교체	0% → 100% 한순간	10% → 50% → 100%
롤백	옛 ReplicaSet 으로 (수십 초)	셀렉터 되돌림 (즉시)	가중치 되돌림 (빠름)
배포 중 리소스	maxSurge 만큼	2배	한 단계 분량
버전 공존	교체 구간 동안	전환 순간만	배포 기간 내내
실패 감지	배포 후에야	전환 후에야	소수 사용자로 먼저
필요한 것	Probe · preStop	여유 리소스	트래픽 분할 + 관측

[정리] 의심스러우면 롤링이다. 리소스 여유가 있고 즉시 롤백이 필요하다면 블루그린, 실사용 검증이 필요하다면 카나리.

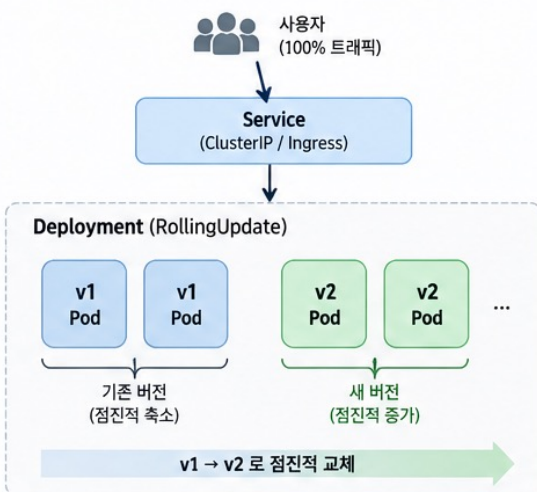
롤링 · 블루그린 · 카나리 비교

1 RollingUpdate (기본 전략)

- 기존 Pod를 점진적으로 교체하는 방식
- 별도 인프라 변경 없이 간단하게 적용 가능
- 업데이트 중에 일부 Pod가 계속 서비스하여 무중단 배포

배포 흐름

- 1 새 버전 Pod 생성 (Max Surge)
- 2 기존 버전 Pod 종료 (Max Unavailable)
- 3 반복하여 순차 교체

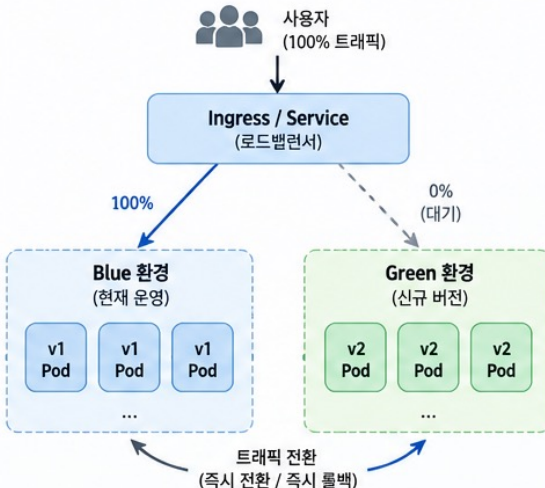


2 Blue / Green (이중 환경 전환)

- 동일한 환경을 2개(Blue, Green)로 유지하고, 전체 트래픽을 한 번에 전환
- 빠른 롤백이 가능 (트래픽만 되돌리면 됨)
- 배포 리스크가 낮지만, 두 배의 자원이 필요

배포 흐름

- 1 Green 환경에 새 버전 배포 (병렬 구성)
- 2 테스트 후 트래픽 전환 (Service/Ingress 변경)
- 3 필요 시 Blue로 즉시 롤백

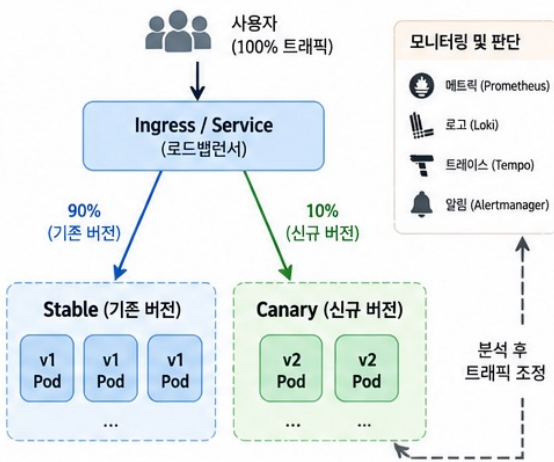


3 Canary (점진적 트래픽 분할)

- 신규 버전을 일부 사용자에게만 먼저 배포
- 메트릭을 모니터링하면서 점진적으로 트래픽을 확대
- 문제 발생 시 즉시 트래픽을 축소하거나 롤백 가능

배포 흐름

- 1 새 버전 Pod 배포 (소수 트래픽)
- 2 모니터링 후 단계적 증가 (예: 5% → 20% → 50% → 100%)
- 3 이상 시 트래픽 축소 또는 롤백



항목	RollingUpdate	Blue / Green	Canary
배포 방식	기존 Pod를 점진적으로 교체	두 개의 동일한 환경을 구성하고 전체 전환	일부 사용자에게 먼저 배포 후 점진적 확대
트래픽 흐름	새 버전 Pod로 순차 분산	모든 트래픽을 한 번에 전환	트래픽을 비율로 분할 (예: 5% → 20% → 100%)
장점	• 구현이 간단함 • 별도 인프라 없이 기본 제공 • 무중단 배포 가능	• 빠른 롤백 가능 • 배포 리스크가 낮음 • 운영 환경과 동일한 검증 가능	• 실제 사용자 기반 검증 가능 • 리스크 최소화 • 점진적 확대를 통해 안정성 확인
단점	• 배포 중 일부 버전이 혼재 • 문제 발생 시 롤백 시간이 소요될 수 있음	• 두 배의 자원 필요 • 환경 구성 및 운영 복잡도 증가	• 트래픽 분할 설정이 필요 (Ingress, Service Mesh 등) • 모니터링 및 분석 체계가 필수
적합한 경우	대부분의 일반적인 서비스	중요 서비스, 빠른 롤백이 필요한 경우	사용자 영향 최소화가 중요한 서비스 (금융, 대규모 트래픽 등)

롤백과 리비전

롤백은 옛 **ReplicaSet** 을 다시 키우는 동작이라 즉시 끝난다.

- `rollout undo` 는 되감기가 아니라 **새 리비전을 만드는 것**이다. 실패하면 멈출 뿐, 롤백은 사람이나 파이프라인의 몫이다

진행 상황 확인 - 배포 파이프라인은 이 명령의 종료 코드로 성패를 판단한다

```
kubectl rollout status deploy/shop-api -n $NS --timeout=300s
```

이력 확인. **CHANGE-CAUSE** 는 어노테이션에서 온다

```
kubectl rollout history deploy/shop-api -n $NS
```

```
kubectl rollout history deploy/shop-api -n $NS --revision=3
```

직전으로 / 특정 리비전으로

```
kubectl rollout undo deploy/shop-api -n $NS
```

```
kubectl rollout undo deploy/shop-api -n $NS --to-revision=3
```

진행 중인 배포를 멈추고(원인 조사) 다시 재개

```
kubectl rollout pause deploy/shop-api -n $NS
```

```
kubectl rollout resume deploy/shop-api -n $NS
```

[체크] `rollout status` 의 종료 코드를 CI 에서 확인하지 않으면 **실패한 배포가 성공으로 보고된다.**

무중단의 실제 조건 네 가지

하나라도 빠지면 무중단이 아니다. 배포 전 **체크리스트**로 사용한다.

- 서로 보완하지 않으므로 **전부 있어야** 성립한다

PodDisruptionBudget 은 자발적 중단(드레인·업그레이드)에만 적용된다

(관리자나 애플리케이션 소유자가 **의도적이고 계획적으로** 수행하는 작업으로 인해 발생하는 파드 중단)

조건	무엇을 하나	빠지면 생기는 일
① readinessProbe	준비된 파드만 엔드포인트에 넣는다	기동 중인 파드로 트래픽 → 5xx
② preStop + graceful	종료 전에 엔드포인트 전파를 기다린다	종료 중인 파드로 트래픽 → 502
③ PodDisruptionBudget	동시에 내려가는 파드 수를 제한	노드 드레인 시 전부 동시 종료
④ 앞뒤 호환	두 버전이 같은 DB·API 와 공존 가능	혼재 구간에서 데이터 오류
(+) topologySpread	여러 노드·AZ 에 분산	노드 하나 죽을 때 전부 죽음

[주의] PDB 는 노드 장애 같은 비자발적 중단은 막지 못한다. 드레인·업그레이드에만 효과가 있다.

StatefulSet 파드의 고유 이름

Deployment 의 파드는 교체 가능한 부품이고, StatefulSet 의 파드는 이름이 있는 개체이다.

- 비유: Deployment 는 콜센터 상담원(누가 받아도 됨), StatefulSet 은 지정 좌석(0번은 항상 0번)
- 고정되는 것 셋: **이름 · 순서 · 볼륨**
- 셋 중 하나도 필요 없으면 Deployment 이다

항목	Deployment	StatefulSet
파드 이름	shop-api-6b4c9d-x7k2 (랜덤)	shop-db-0, shop-db-1 (고정·순서)
생성 순서	동시에 전부	0 → 1 → 2 순차 (앞이 Ready 여야 다음)
삭제 순서	임의	역순 (2 → 1 → 0)
볼륨	모두 같은 PVC 공유 또는 없음	파드마다 전용 PVC (volumeClaimTemplates)
네트워크 ID	Service 를 통해서만	shop-db-0.shop-db.shop.svc 로 개별 지정 가능
재생성 시	새 이름·새 IP	같은 이름·같은 볼륨 재연결
스케일 다운	임의 파드 제거	높은 번호부터. PVC 는 남는다

[주의] StatefulSet 을 삭제해도 **PVC 는 그대로 유지된다**. 의도된 안전장치지만, 삭제하지 않으면 EBS 비용이 계속 발생된다.

StatefulSet 운영

StatefulSet 은 **headless Service** 와 짝으로 써야 개별 파드 주소가 만들어진다.

- **clusterIP: None** 이면 Service 가 로드밸런싱 대신 **파드별 DNS 레코드**를 만든다.

역순 롤링이고, **partition** 으로 일부만 먼저 올리는 카나리도 된다

```
kind: Service
metadata: {name: shop-db}
spec:
  clusterIP: None      # headless - shop-db-0.shop-db.shop.svc 가 생긴다
  selector: {app: shop-db}
---
kind: StatefulSet
metadata: {name: shop-db}
spec:
  serviceName: shop-db # 위 headless Service 이름과 반드시 일치
  replicas: 3
  updateStrategy:
    rollingUpdate: {partition: 2} # 인덱스 2 이상만 교체 → 카나리
  volumeClaimTemplates:          # 파드마다 전용 PVC 가 생긴다
  - metadata: {name: data}
    spec: {accessModes: [ReadWriteOnce], resources: {requests: {storage: 20Gi}}}
```

[체크] **serviceName** 이 실제 headless Service 와 다르면 **파드별 DNS** 가 만들어지지 않는다. 오타 실수가 자주 발생하는 경우이다.

DaemonSet 의 노드별 실행

로그 수집기 · 노드 모니터 · CNI 처럼 '모든 노드에 하나씩' 이 필요한 것들이 있다.

- 노드가 늘어나면 자동으로 따라 붙는다. `replicas` 필드가 없고 개수는 노드 수가 정한다

```
kind: DaemonSet
metadata: {name: fluent-bit, namespace: logging}
spec:
  template:
    spec:
      tolerations:           # 테인트 걸린 노드에도 들어가야 한다
      - operator: Exists     # 모든 테인트 감수 = 정말 모든 노드에
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:3.1
        resources:           # 노드 수만큼 곱해진다. 작게 잡는다
          requests: {cpu: 50m, memory: 100Mi}
        volumeMounts:
        - {name: varlog, mountPath: /var/log, readOnly: true}
      volumes:
      - {name: varlog, hostPath: {path: /var/log}}
```

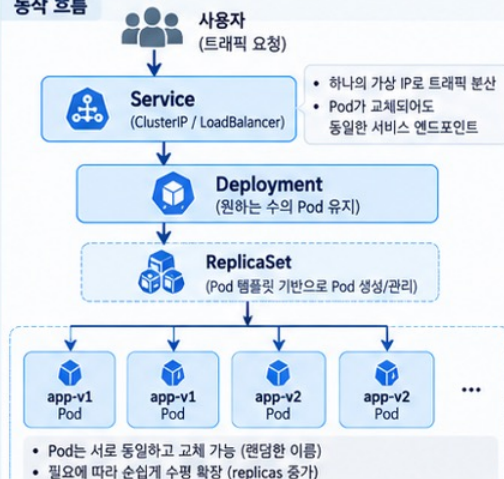
[주의] DaemonSet 의 `requests` 는 노드 수만큼 곱해진다. 200노드에 500m 으로 지정하면 100코어가 상시 예약된다.

Deployment vs. StatefulSet vs. DaemonSet 비교

1 Deployment (일반 stateless 앱)

- stateless 서비스에 적합 (상태를 Pod에 저장하지 않음)
- ReplicaSet 기반으로 원하는 수의 Pod를 유지
- Pod 이름/스토리지의 고정성이 없음 (교체 가능)
- 수평 확장이 용이 (replicas 수 증가/감소)
- RollingUpdate를 통한 무중단 배포가 기본 지원

동작 흐름



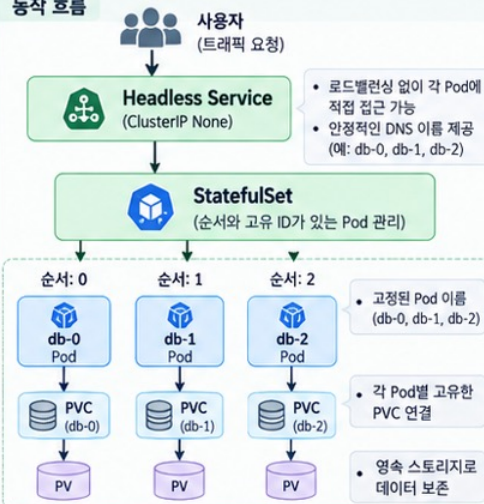
주요 사용 사례



2 StatefulSet (상태 저장 워크로드)

- Pod의 순서와 고유한 식별자(이름) 보장 (db-0, db-1, ...)
- 안정적인 네트워크 ID (고정된 DNS 이름)
- 각 Pod별로 고유한 PersistentVolume(PVC) 연결
- 순차적인 배포 및 삭제 (순서 보장)
- 데이터베이스와 같은 stateful 애플리케이션에 적합

동작 흐름



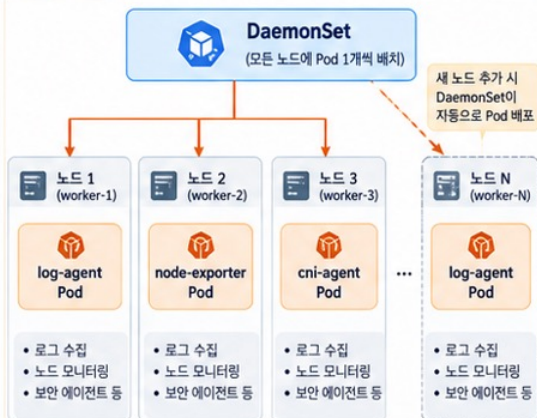
주요 사용 사례



3 DaemonSet (노드당 1개 배치)

- 클러스터의 모든 노드(또는 특정 노드)에 Pod 1개씩 배치
- 새 노드가 추가되면 자동으로 해당 노드에 Pod 배포
- 노드에서 실행되는 에이전트, 로그 수집기, 모니터링 도구 등에 사용
- 보통 Service를 통한 트래픽 분배가 필요하지 않음
- 모든 노드에서 동일한 기능의 Pod가 실행되는 것이 목적

동작 흐름



주요 사용 사례



항목	Deployment	StatefulSet	DemonSet
주요 대상	일반적인 stateless 애플리케이션	상태 저장이 필요한 애플리케이션	모든 노드에서 실행해야 하는 에이전트/시스템 앱
Pod 식별성	고정되지 않음 (랜덤한 이름, 교체 가능)	순서와 고유한 이름 보장 (예: db-0, db-1, ...)	각 노드당 1개, 노드와 1:1로 배치 (노드별로 하나)
스토리지	기본적으로 사용하지 않음 (필요 시 임시 볼륨)	각 Pod별 고유한 PersistentVolume 연결	일반적으로 사용하지 않음 (노드의 로컬 스토리지 활용 가능)
스케일 방식	replicas 수를 통한 수평 확장/축소	replicas 수를 통한 수평 확장/축소 (순서 보장)	노드 수에 따라 자동으로 Pod 수 조절 (수동 스케일 없음)
업데이트 특성	RollingUpdate 기본 지원 (무중단 배포)	순차적인 업데이트 (일반적으로 하나씩)	업데이트 시 각 노드의 Pod가 개별적으로 교체 (동시/순차 설정 가능)
대표 사례	웹 애플리케이션, API 서버, 프론트엔드	MySQL, PostgreSQL, Kafka, ZooKeeper	Fluent Bit, node-exporter, CNI plugin, 보안 에이전트

핵심 선택 기준

1 상태 없는 다중 복제 서비스 → Deployment
간단한 웹 서비스, API 서버 등

2 순서/고유성/영속 저장소가 필요한 워크로드 → StatefulSet
데이터베이스, 메시지 큐, 분산 시스템 등

3 모든 노드에 공통 에이전트 배치 → DaemonSet
로그 수집기, 모니터링, 네트워크/보안 에이전트 등

Job 의 성공 조건

Deployment 는 '계속 살아 있어야 성공' 이고 Job 은 '끝나야 성공' 이라, 성공의 정의가 상반된다.

- 동작을 정하는 세 숫자: `completions` · `parallelism` · `backoffLimit`
- 끝난 Job 은 자동으로 지워지지 않는다
- `ttlSecondsAfterFinished` 를 설정한다
(쿠버네티스(Kubernetes) **Job** 리소스에서 실행이 완료(Complete 또는 실패(Failed)된 잡과 관련 파드(Pod)를 **자동으로 삭제**하기 위해 지정하는 필드, 100 입력 시 완료 후 100초 뒤 삭제)

설정 조합	동작	사용하는 곳
completions 없음	1번 성공하면 끝	마이그레이션 · 일회성 작업
completions: 100 parallelism: 5	100번 완료까지, 동시 5개씩	대량 배치 (파일 100개 처리)
completions 없음 parallelism: 5	5개가 병렬로 돌아 하나 성공하면 끝	워커 풀 패턴
backoffLimit: 4	실패 시 4번까지 재시도(지수 백오프)	일시적 오류 대비. 0 이면 재시도 없음
activeDeadlineSeconds: 600	10분 넘으면 강제 종료	무한 루프 방지. 반드시 설정

[주의] `ttlSecondsAfterFinished` 가 없으면 완료된 Job 과 파드가 계속 쌓여 **etcd** 를 채운다. 86400(1일)이 무난하다.

CronJob 과 Job 생성

시각이 되면 **Job** 을 만들고, 파드 생성은 Job 에 맡긴다.

- 표준 cron 표현식을 사용하고 시간대는 기본 UTC 이다(`timeZone` 으로 변경). 겹침 처리(`concurrencyPolicy`)를 정하지 않으면 느린 배치가 겹쳐 쌓인다

```
kind: CronJob
metadata: {name: shop-settlement}
spec:
  schedule: "0 2 * * *"           # 매일 02:00
  timeZone: "Asia/Seoul"         # 없으면 UTC → 실제로는 한국 11:00 에 실행된다
  concurrencyPolicy: Forbid      # 이전 회차가 안 끝났으면 이번은 건너뛰다
  startingDeadlineSeconds: 300   # 5분 안에 못 시작하면 그 회차는 포기
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      activeDeadlineSeconds: 3600
      template:
        spec:
          restartPolicy: Never    # Never 또는 OnFailure 만 가능
```

[주의] `timeZone` 을 지정하지 않으면 **UTC** 기준이다. `0 2 * * *` 는 한국 시간 오전 11시에 실행된다.

1 CronJob (스케줄 기반 실행)

- 정해진 스케줄(cron 표현식)로 Job 생성
- 반복 실행 (주기적으로 Job 생성)
- 배치 작업, 정기 점검, 백업 등에 적합
- concurrencyPolicy, suspend, history 관리 가능

동작 흐름



주요 옵션 및 특징

- 성공/실패 이력 보관 (successfulJobsHistoryLimit, failedJobsHistoryLimit)
- startingDeadlineSeconds (지연 실행 허용 시간)
- concurrencyPolicy: Allow / Forbid / Replace
- suspend: 일시 중지 가능

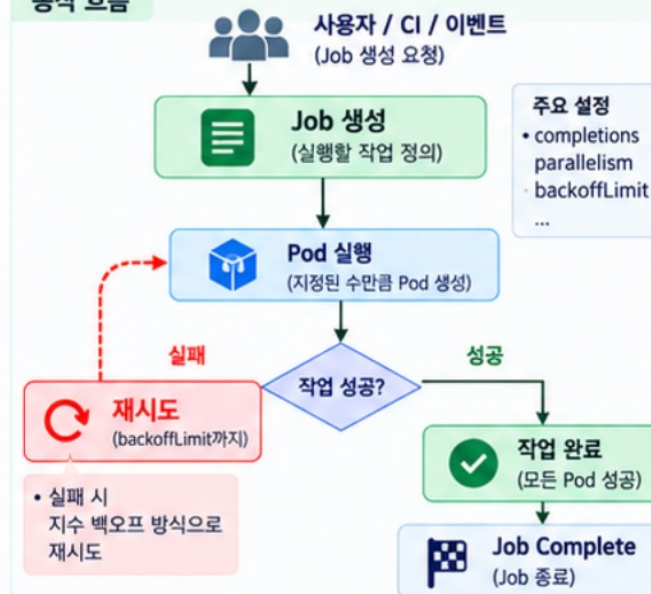
주요 사용 사례

- 백업 (데이터 백업)
- 배치 리포트 (정기 리포트)
- 정기 데이터 수집 (로그/지표 수집)
- 만료 데이터 정리 (일시 데이터 삭제)

2 Job (한 번 실행 후 완료)

- 완료를 목표로 하는 워크로드
- Pod가 성공적으로 끝날 때까지 재시도
- 병렬 처리 가능 (completions, parallelism)
- 완료 후 종료 (지속 실행하지 않음)

동작 흐름



주요 옵션 및 특징

- completions: 장상으로 완료해야 하는 Pod 수
- parallelism: 동시에 실행할 Pod 수
- backoffLimit: 실패 시 최대 재시도 횟수
- ttlSecondsAfterFinished: 완료 후 리소스 자동 삭제 시간

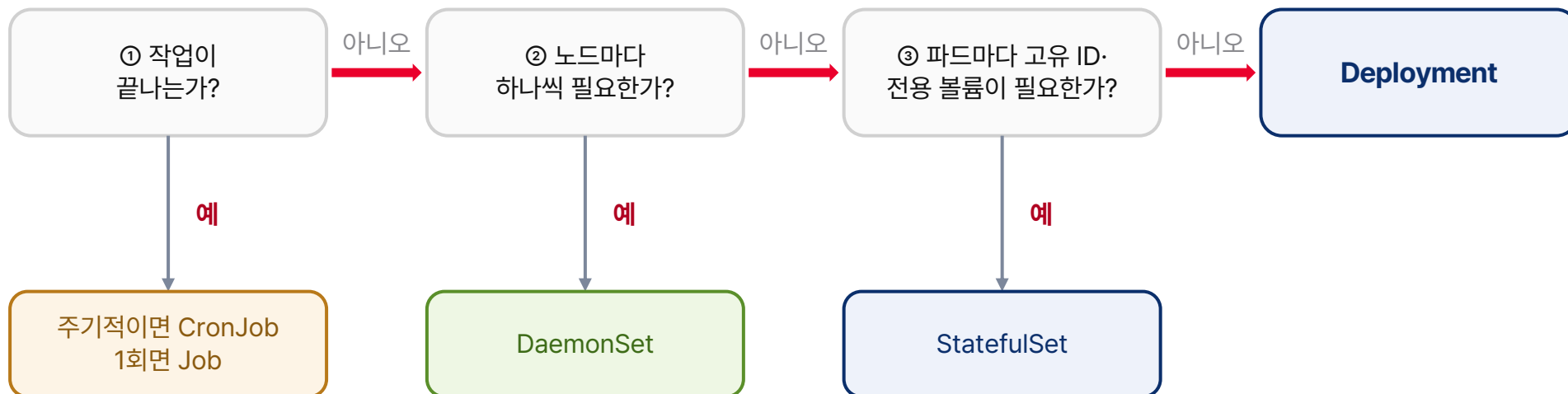
주요 사용 사례

- 데이터 마이그레이션
- 일회성 스크립트 실행
- 대량 처리 작업 (대규모 배치)
- 초기화 작업 (환경 설정)

워크로드 타입 선택 기준

다음 질문 3개로 결정될 수 있다. "끝나는가" · "노드마다인가" · "개체 식별"이 필요한가.

- 왼쪽부터 답해 내려오면서 "아니오" 이면 오른쪽, "예" 이면 아래로 선택된다.



3개 모두 '아니오'면 Deployment 이고, 대부분의 서비스가 Deployment 로 오게 된다.

[정리] 'DB 를 사용한다'가 StatefulSet 의 조건이 아니다. RDS 를 사용하는 API 서버는 Deployment 이다.

워크로드 진단 지도

증상	첫 확인	흔한 원인
배포가 진행 안 됨(멈춤)	<code>rollout status</code> · <code>describe deploy</code>	새 파드 Pending · Probe 실패 · <code>maxUnavailable 0</code> + 자원 부족
배포 중 5xx 발생	<code>describe pod</code> Probe 설정	<code>readiness</code> 없음 · <code>preStop</code> 없음
파드가 계속 재시작	<code>logs --previous</code>	<code>liveness</code> 과민 · 설정 누락 · OOM
Job 이 안 끝남	<code>describe job</code> · 파드 상태	사이드카가 안 죽음 · <code>deadline</code> 미설정
CronJob 이 안 뚫	<code>describe cronjob</code> LastSchedule	시간대(UTC) · <code>startingDeadline</code> 초과
StatefulSet 이 1번에서 멈춤	0번 파드 Ready 여부	0번이 Ready 가 아니면 1번은 만들어지지 않는다
스케일 다운했는데 PVC 남음	<code>get pvc</code>	정상 동작. 수동 정리 필요
DaemonSet 이 일부 노드에 없음	<code>describe ds</code> · 노드 테인트	<code>toleration</code> 누락 · <code>nodeSelector</code> 불일치

[핵심 요약] 2장 워크로드

항목	한 줄 정리	실무 포인트
파드 직접 생성 금지	컨트롤러가 만들어야 자가치유가 된다	매니페스트에서 <code>kind: Pod</code> 를 찾아 없앤다
종료 흐름	엔드포인트 제거·preStop·SIGTERM 이 동시에	preStop sleep 5~10초가 502 를 없앤다
Deployment 3계층	Deployment → ReplicaSet → Pod	옛 ReplicaSet 이 롤백의 근거이다. 지우지 않는다
surge / unavailable	속도와 용량 유지의 균형	<code>unavailable: 0</code> 은 노드 여유가 있어야 동작
무중단 4조건	readiness · preStop · PDB · 앞뒤 호환	하나라도 빠지면 무중단이 아니다
StatefulSet	이름·순서·볼륨이 고정된다	'DB 를 사용한다'가 아니라 '개체 식별이 필요하다'가 기준
Job / CronJob	끝나야 성공. 시각이 되면 Job 을 만든다	<code>timeZone·ttl·activeDeadline</code> 세 개는 필수
롤백	옛 ReplicaSet 을 다시 키운다	자동 롤백은 없다. CI 가 <code>rollout status</code> 를 봐야 한다

3. 스케줄링과 리소스 관리

- requests 와 limits, 두 숫자가 각각 무엇을 정하는가
- QoS 클래스와 축출 순서. 자원이 모자랄 때 누가 먼저 밀려나는가
- 어피니티 · 테인트 · 토폴로지 분산으로 배치를 의도대로
- HPA · VPA · Cluster Autoscaler 의 역할 분담

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[기초] 컨테이너의 자원 분배

- 리눅스는 **cgroup** 으로 프로세스가 사용할 자원의 상한을 강제한다
- CPU 는 시간을 쪼개 나눠 사용한다
- 메모리는 한 번 준 것을 회수할 수 없다

	CPU	메모리
나눠 사용할 수 있는가	있다. 시간을 잘게 쪼개 번갈아 사용한다	없다. 한 바이트는 한 주인에게만
상한에 닿으면	느려진다 (Throttling) (성능이나 속도를 강제로 낮추거나 제한하는 기능)	프로세스가 종료된다 (OOMKilled)
회수 방법	다음 주기에 덜 준다	회수할 수 없다. 죽이는 것뿐
단위	1 = 코어 1개, 500m = 0.5개	Mi = 1024 ² , M = 1000 ³
넉넉히 산정하면	노드가 놀아도 배치가 막힌다	노드가 놀아도 배치가 막힌다

[정리] CPU 는 압축할 수 있고 메모리는 압축할 수 없다. limits 를 다르게 다뤄야 하는 이유가 여기이다.

requests 와 limits

requests 는 스케줄러가 보는 숫자이고, **limits** 는 kubelet 이 강제하는 숫자이다. 쓰임이 다르다.

- **requests** = '이만큼은 확보해 줘' → 노드 선택과 예약에 쓰인다
- **limits** = '이 이상은 사용할 수 없다' → cgroup 으로 강제된다
- limits 는 실행 중에만 의미가 있다

	requests	limits
누가 보나	kube-scheduler	kubelet / 커널 cgroup
언제 쓰이나	배치할 노드를 고를 때 한 번	컨테이너가 실행되는 내내
CPU 초과 시	(해당 없음)	스로틀링. 느려지지만 중단되지 않는다
메모리 초과 시	(해당 없음)	OOMKilled. 즉시 강제 종료
비용에 미치는 영향	직접적. 노드 대수를 정한다	간접적
안 적으면	0 으로 간주 → 아무 데나 배치됨	무제한 → 노드를 다 먹을 수 있다

[정리] 노드 대수를 정하는 것은 **limits** 가 아니라 **requests** 이다. 비용 최적화는 여기서 시작한다.

CPU limits 의 함정

CPU limits 는 컨테이너를 종료시키지 않는다. 주기마다 강제로 지연이 발생할 수 있다.

- 리눅스 CFS 가 100ms 주기로 할당량을 주고, 소진하면 남은 시간 동안 정지한다.
`limits.cpu: 500m` 은 컨테이너 전체가 최대 0.5 CPU만 사용할 수 있다는 의미이다. CPU를 사용하는 스레드가 많으면 제한된 CPU quota를 빠르게 소진하여 CPU throttling이 발생할 수 있다.

```
# 스로틀링이 실제로 걸리는지 본다 - 컨테이너 cgroup 지표
kubectl exec -it shop-api-xxx -n $NS -- cat /sys/fs/cgroup/cpu.stat
#   nr_periods 12043      ← 지금까지 지나간 100ms 주기 수
#   nr_throttled 3891     ← 그중 할당량을 다 써서 멈춘 주기 수
#   throttled_usec 48210000 ← 멈춰 있던 총 시간

# 판단 기준: nr_throttled / nr_periods
#   < 1%      정상
#   1~5%      관찰. p99 지연이 튀기 시작한다
#   > 5%      limits 를 올리거나 제거한다. 지연 SLO 를 못 맞춘다

# Prometheus 라면
#   rate(container_cpu_cfs_throttled_periods_total[5m])
#     / rate(container_cpu_cfs_periods_total[5m])
```

[주의] CPU 사용률이 limits 의 절반인데도 스로틀링이 걸릴 수 있다. **평균이 아니라 100ms 구간으로** 판정하기 때문이다.

메모리 limits 와 OOMKilled

메모리는 압축할 수 없다. 초과하면 즉시 종료된다.

- 종료 코드 **137** = 128 + 9(SIGKILL) 이고 `describe pod` 의 Last State 에 남는다. JVM·Node.js 는 컨테이너 limits 를 자동 인식하지 못해 명시적으로 알려 줘야 한다

```
resources:
  requests: {memory: 512Mi, cpu: 200m}
  limits:   {memory: 1Gi}           # CPU limits 는 의도적으로 생략 (앞 슬라이드 참고)
env:
# JVM: 컨테이너 limits 의 75% 를 힙으로. 안 주면 호스트 전체 메모리 기준으로 잡는다
- {name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0"}
# Node.js: --max-old-space-size 를 limits 보다 작게 (MB 단위)
- {name: NODE_OPTIONS, value: "--max-old-space-size=768"}
---
# 죽은 원인 확인
# kubectl describe pod shop-api-xxx -n $NS
#   Last State:  Terminated
#       Reason:   OOMKilled
#       Exit Code: 137
```

[주의] 메모리 limits 는 **requests** 와 같거나 약간 크게. 차이가 크면 노드 전체가 메모리 압박에 빠진다.

requests 를 정하는 법

임의로 추측하지 않는다. 재고, 여유를 더하고, 다시 잰다.

- 출발점은 **CPU = p95 × 1.2, 메모리 = 최대 × 1.3** 이다. 재조정은 분기 1회면 충분하고, 트래픽 패턴이 바뀌면 그때 바로 한다
(p95는 측정 구간 중 95%의 시간 동안 이 값 이하의 CPU를 사용했다는 의미)

① 지금 얼마나 쓰는가 (metrics-server 필요)

```
kubectl top pod -n $NS --containers
```

```
# NAME CPU(cores) MEMORY(bytes)
```

```
# shop-api-xxx 147m 612Mi
```

② 요청한 양 대비 얼마나 쓰는가 - 과다 예약을 찾는다

```
kubectl get pods -n $NS -o custom-columns=\
```

```
'NAME:.metadata.name,CPU_REQ:.spec.containers[0].resources.requests.cpu'
```

③ 노드가 얼마나 예약됐는가 - 실사용이 아니라 '예약'이 기준이다

```
kubectl describe node ip-10-0-1-23 | grep -A6 'Allocated resources'
```

```
# cpu 3200m (81%) ← 81% 예약. 실사용은 30% 일 수도 있다
```

```
# memory 11Gi (73%)
```

QoS 클래스와 축출 순서

노드 메모리가 부족하면 kubelet 이 파드를 골라서 축출한다. 그 순서를 정하는 것이 QoS 이다.

- QoS 는 `requests` 와 `limits` 조합으로 자동 결정된다
- 직접 적는 필드가 없다
- 축출 순서: BestEffort → Burstable(초과분 큰 순) → Guaranteed

QoS 클래스	조건	축출 순서	사용하는 곳
Guaranteed	모든 컨테이너가 <code>requests == limits</code> (CPU·메모리 둘 다)	마지막	결제 · 인증 등 절대 중단되면 안 되는 것
Burstable	<code>requests</code> 는 있고 <code>limits</code> 와 다름	중간 (<code>requests</code> 초과분이 큰 것 부터)	일반 웹/API 서버 대부분
BestEffort	<code>requests</code> · <code>limits</code> 둘 다 없음	제일 먼저	사용하지 않는다. 실수로 만들어지는 경우가 대부분

[주의] `requests` 를 안 적으면 자동으로 BestEffort 가 된다. 노드가 압박받는 순간 가장 먼저 축출된다.

노드가 실제로 주는 자원

`m5.large` 는 2코어 8GB 지만, 파드가 사용할 수 있는 것은 그보다 훨씬 적을 수 있다.

- 노드 자원 = 시스템 예약 + kube 예약 + 축출 임계값 + **Allocatable**
- 파드가 쓸 수 있는 것은 Allocatable 뿐이다
- EKS 는 여기에 VPC CNI 의 파드 수 상한이 더 걸리게 된다.

항목	무엇	m5.large(2vCPU·8GiB) 예시
Capacity	인스턴스의 총 자원	2000m / 8 GiB
system-reserved	OS-sshd 등	약 100m / 0.2 GiB
kube-reserved	kubelet-containerd	약 70m / 1.0 GiB
eviction-threshold	축출 시작 여유분	약 0.1 GiB
Allocatable	파드가 실제로 사용할 수 있는 몫	약 1930m / 6.8 GiB
최대 파드 수 (VPC CNI)	ENI × (IP-1) 로 계산	29개 (CPU 가 남아도 더 뜨지 못한다)

[체크] `kubectl describe node` 의 **Allocatable** 을 보고 용량을 계산한다. Capacity 로 계산하면 항상 모자란다.

LimitRange 와 ResourceQuota

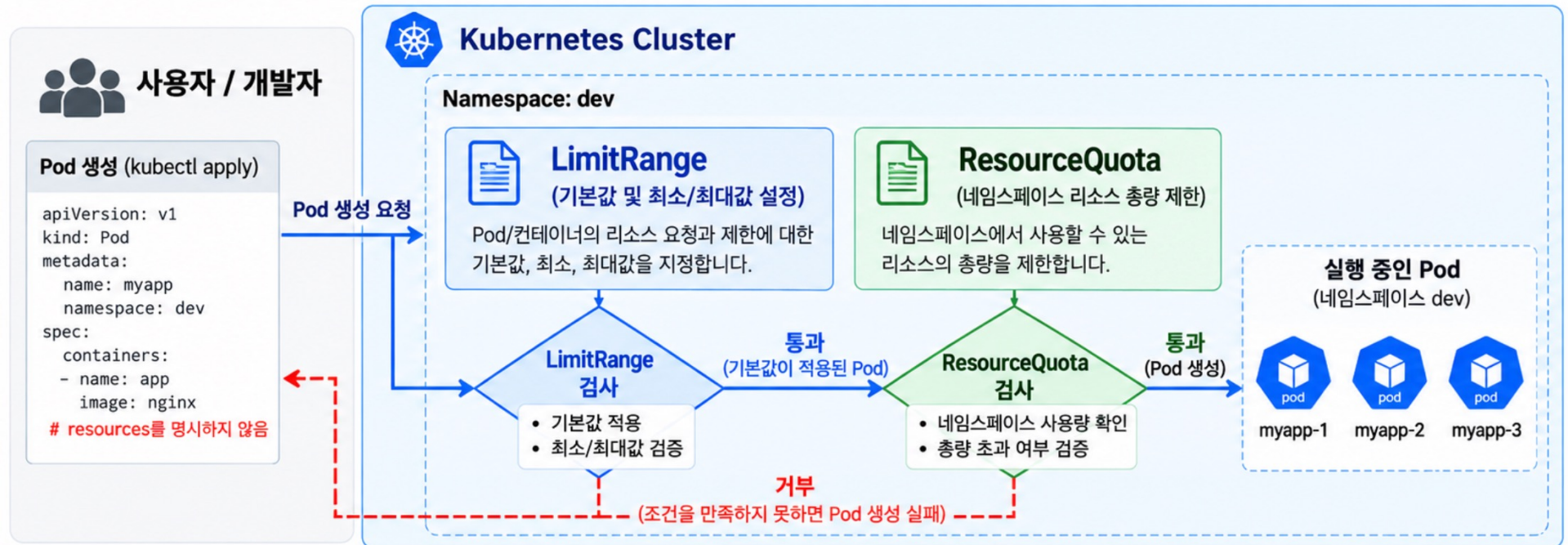
>> LimitRange는 개별 리소스의 크기를 관리하고, ResourceQuota는 Namespace 전체 리소스의 총합을 관리한다.

- **LimitRange**: 파드 하나의 기본값 · 최소값 · 최대값. BestEffort 를 원천 차단한다
- **ResourceQuota**: 네임스페이스 전체 총량. 한 팀의 클러스터 독점을 막는다

```
kind: LimitRange
metadata: {name: shop-defaults, namespace: <내 NS>} # 파드 하나에 적용
spec:
  limits:
    - type: Container
      default: {cpu: 500m, memory: 512Mi} # limits 미지정 시 주입
      defaultRequest: {cpu: 100m, memory: 128Mi} # requests 미지정 시 주입
      max: {cpu: "4", memory: 8Gi} # 넘으면 배포 거부
---
kind: ResourceQuota
metadata: {name: shop-quota, namespace: <내 NS>}
spec:
  hard: # 네임스페이스 전체 총량
    requests.cpu: "20"
    requests.memory: 40Gi
```

[주의] ResourceQuota 가 있는 네임스페이스에서는 **requests** 를 안 적은 파드가 아예 거부된다. LimitRange 와 함께 써야 한다.

LimitRange 와 ResourceQuota



nodeSelector 와 nodeAffinity

'이 파드는 이런 노드에 놓아라'를 표현하는 두 가지 방법. **엄격함의 정도가 다르다.**

- >> nodeSelector는 특정 label의 Node에 Pod를 고정 배치하는 단순 방식이고, nodeAffinity는 필수·선호 조건과 다양한 label 연산자를 이용해 보다 유연하게 Node를 선택하는 방식이다.
- nodeSelector는 단순 일치라 조건에 맞는 노드가 없으면 **영원히 Pending**
- nodeAffinity는 **선호(preferred)**도 표현하고, 없으면 아무 데나 놓는다

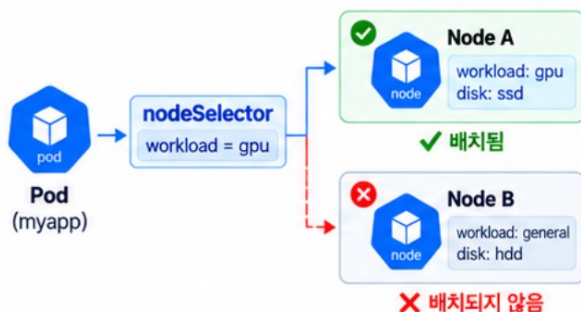
```
spec:
  nodeSelector:                # ① 가장 단순. AND 조건만 가능
    node.kubernetes.io/instance-type: m5.large
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution: # ② 필수 - 없으면 Pending
        nodeSelectorTerms:
          - matchExpressions:
              - {key: topology.kubernetes.io/zone, operator: In,
                values: [ap-northeast-2a, ap-northeast-2c]}
      preferredDuringSchedulingIgnoredDuringExecution: # ③ 선호 - 없어도 배치됨
        - weight: 80
          preference:
            matchExpressions:
              - {key: eks.amazonaws.com/capacityType, operator: In, values: [SPOT]}
```

[참고] IgnoredDuringExecution = 배치할 때만 본다. 이미 뜬 파드는 노드 라벨이 바뀌어도 옮겨지지 않는다.

nodeSelector 와 nodeAffinity

1. nodeSelector (단순 고정 배치)

노드의 label과 정확히 일치하는 노드에만 Pod를 배치하는 가장 단순한 방법입니다.



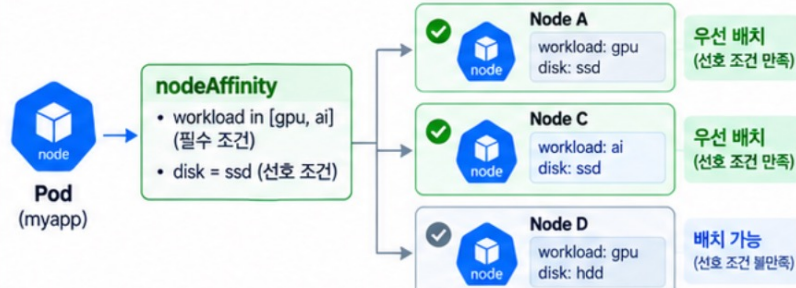
- ✓ 정확한 key-value label 일치가 필요합니다.
- ✓ 설정이 간단하고 이해하기 쉽습니다.
- ✓ 특정 노드에 고정 배치하는 단순한 경우에 적합합니다.
- ✗ 선호 조건이나 복잡한 조건을 표현할 수 없습니다.

YAML 예시

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp
spec:
  nodeSelector:
    workload: gpu
```

2. nodeAffinity (유연한 고급 배치)

label 표현식을 사용한 고급 노드 선택 방식으로, 필수 조건(required)과 선호 조건(preferred)을 모두 지원합니다.



필수 조건 (requiredDuringSchedulingIgnoredDuringExecution)

Pod가 배치되기 위해 반드시 만족해야 하는 조건입니다.

- ✓ In / NotIn / Exists / Gt / Lt 등 다양한 연산자를 사용할 수 있습니다.
- ✓ 필수 조건과 선호 조건을 구분하여 설정할 수 있습니다.
- ✓ 복잡한 스케줄링 규칙(여러 label, 우선순위 등)을 표현할 수 있습니다.

선호 조건 (preferredDuringSchedulingIgnoredDuringExecution)

조건을 만족하는 노드를 우선적으로 선택하지만, 만족하지 않아도 배치 가능합니다.

YAML 예시 (필수 + 선호 조건)

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp
spec:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution: # 필수 조건
      nodeSelectorTerms:
        - matchExpressions:
            - key: workload
              operator: In
              values: ["gpu", "ai"]
      preference: # 선호 조건
        - weight: 100
          preference:
            matchExpressions:
              - key: disk
                operator: In
                values: ["ssd"]
```

3. 핵심 차이 / 선택 기준

구분	nodeSelector	nodeAffinity
설정 난이도	단순	유연
표현력	낮음	높음
필수 조건	정확한 label 일치 (key-value)	다양한 matchExpressions (In/NotIn/Exists/Gt/Lt)
선호 조건	불가	가능 (preferred 방식)
대표 사용 사례	간단한 전용 노드 배치 (GPU 노드 등)	GPU, SSD, AZ, 환경별 세밀한 배치



동시에 사용 가능

nodeSelector와 nodeAffinity를 함께 지정하면 두 조건을 모두 만족해야 합니다.



taint/toleration과의 차이

nodeSelector/nodeAffinity는 '어디에 배치할지' (노드 선택 조건)이고, taint/toleration은 '어떤 Pod를 받을지' (노드의 수용 조건)입니다.



핵심 정리

1

단순한 label 고정 배치라면 **nodeSelector**를 사용합니다.

2

필수/선호 조건이 함께 필요하면 **nodeAffinity**를 사용합니다.

3

실무에서는 **label + nodeAffinity + taint/toleration** 조합을 자주 사용합니다.

파드 간 Affinity(친화도)

노드가 아니라 다른 파드를 기준으로 배치한다. 붙이거나(affinity) 떼거나(anti-affinity).

- '같다'의 기준은 `topologyKey`. 같은 노드인가, 같은 존인가
- 복제본을 분산시켜 단일 장애지점을 없애는 **안티어피니티가 훨씬 자주 쓰인다**
(같은 애플리케이션의 복제 Pod를 한 Node에 몰아넣지 않고 여러 Node에 분산해서, 특정 Node 장애가 전체 서비스 장애로 이어지는 것을 막는 방식이다)

```
spec:
  affinity:
    podAntiAffinity:
      # 같은 app 라벨 파드가 이미 있는 노드는 피한다 (선호)
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels: {app: shop-api}
            topologyKey: kubernetes.io/hostname      # '같은 노드'가 기준
    podAffinity:
      # 캐시를 API 와 같은 존에 → 지연.전송료가 준다
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 50
          podAffinityTerm:
            labelSelector: {matchLabels: {app: shop-api}}
            topologyKey: topology.kubernetes.io/zone
```

[주의] 안티어피니티를 `required` 로 적용하면 노드 수보다 `replicas` 가 많을 때 나머지가 **Pending** 이 된다.

Taint 와 Toleration

어피니티가 파드가 노드를 고르는 것이라면, 테인트는 노드가 파드를 거부하는 것이다. 방향이 반대이다.

- 테인트는 노드에 붙이는 '출입 금지' 표식이고, 톨러레이션이 있는 파드만 들어간다
- 톨러레이션은 허가일 뿐 보증이 아니라 다른 노드에 갈 수도 있다

effect	의미	사용하는 곳
NoSchedule	새 파드를 받지 않는다. 기존 파드는 유지	GPU 노드 · 배치 전용 노드 격리
PreferNoSchedule	가급적 받지 않는다 (soft)	특수 목적 노드지만 급하면 허용
NoExecute	새 파드 거부 + 기존 파드도 쫓아낸다	노드 격리·수리 · 자동 장애 대응

```
kubectl taint nodes ip-10-0-2-11 workload=batch:NoSchedule    # 붙이기
kubectl taint nodes ip-10-0-2-11 workload=batch:NoSchedule-  # 떼기 (끝에 -)
# 파드 쪽 톨러레이션
#   tolerations: [{key: workload, operator: Equal, value: batch, effect: NoSchedule}]
```

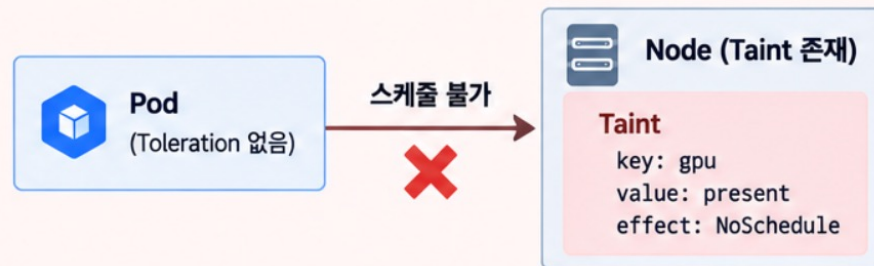
[정리] 격리 = 테인트(노드) + 어피니티(파드) 두 개를 함께 사용한다. 테인트만으로는 다른 파드가 안 올 뿐이다.

Taint 와 Toleration

1 Taint 가 없는 일반 노드 + Toleration 이 없는 Pod (스케줄 가능)



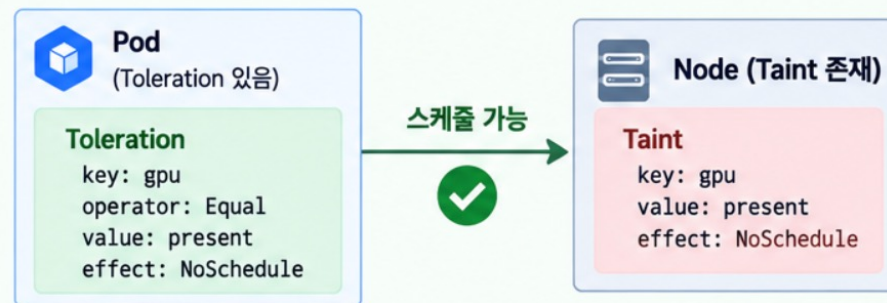
2 Taint 가 있는 노드 + Toleration 이 없는 Pod (스케줄 불가)



3 Taint 가 없는 일반 노드 + Toleration 이 있는 Pod (스케줄 가능)



4 Taint 가 있는 노드 + Toleration 이 있는 Pod (스케줄 가능)



topologySpreadConstraints

'골고루 분산시켜라'를 숫자로 표현하는 방식이라 안티어피니티보다 정밀하다.

- `maxSkew` = 가장 많은 그룹과 가장 적은 그룹의 **허용 차이**
- 존 단위와 노드 단위를 **동시에 적용**하는 것이 실무 기본 구성이다

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1                                # 존 간 차이는 1개까지
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule          # 못 지키면 Pending (엄격)
      labelSelector: {matchLabels: {app: shop-api}}
    - maxSkew: 1                                # 노드 간 차이도 1개까지
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway         # 못 지키면 그냥 배치 (완화)
      labelSelector: {matchLabels: {app: shop-api}}
# replicas: 6, 존 3개 → 각 존에 2개씩. 한 존이 통째로 죽어도 4개가 살아남는다
```

[체크] 존 단위는 `DoNotSchedule`, 노드 단위는 `ScheduleAnyway` 가 무난하다. 둘 다 엄격하면 Pending 이 잦다.

PriorityClass 와 선점

자원이 없으면 스케줄러는 낮은 우선순위 파드를 밀어내고 자리를 만든다. 이것이 선점이다.

- 우선순위는 숫자이고 큰 값이 먼저. 시스템 기본값 20억대는 넘볼 수 없다
- `preemptionPolicy: Never` 로 '우선순위는 높지만 밀어내지는 않음' 을 만들 수 있다

```
kind: PriorityClass
metadata: {name: shop-critical}
value: 10000000          # 결제·인증 - 자리가 없으면 다른 파드를 밀어낸다
globalDefault: false
description: "결제 등 중단 불가 워크로드"
---
kind: PriorityClass
metadata: {name: shop-batch}
value: 100               # 정산 배치 - 급할 때 가장 먼저 밀려난다
preemptionPolicy: Never  # 자기는 남을 밀어내지 않는다
---
# 파드에서 사용
# spec:
#   priorityClassName: shop-critical
```

[주의] 선점당한 파드는 **graceful 종료 시간이 짧다**(기본 0). 데이터를 기록하는 워크로드에는 낮은 우선순위를 주지 않는다.

HPA의 계산식

HPA의 공식은 하나이다. 원하는 개수 = 올림(현재 개수 × 현재 지표 / 목표 지표)

- CPU 기준은 **limits**가 아니라 **requests** 대비 비율이다. 이것이 잦은 오해이다.
- 지표는 metrics-server에서 오고, **15초 간격 갱신에 임의의 시간 지연**이 있을 수 있다

예시 replicas=4, requests.cpu=200m, target=70%
현재 평균 사용량 = 168m → 사용률 168/200 = 84%

$\text{desired} = \text{ceil}(4 \times 84 / 70) = \text{ceil}(4.8) = 5$

주의 ① requests를 낮추면 같은 사용량에서 '비율'이 커진다
→ requests 100m 이면 168%. $\text{desired} = \text{ceil}(4 \times 168/70) = 10$
→ requests 값이 HPA의 민감도를 바꾼다

주의 ② 10% 이내 변화는 무시한다 (tolerance)
→ 진동(flapping)을 막기 위한 기본 동작

[체크] HPA가 예상보다 과하게 늘어난다면 **requests가 실사용보다 작게 설정돼 있는지** 먼저 확인한다.

HPA 실전 설정

기본값은 내려갈 때 매우 보수적(5분 안정화)이다. 올라갈 때와 내려갈 때를 따로 지정한다.

- `behavior` 로 확대·축소 속도를 따로 제어한다 (빠르게 올리고 천천히 내린다)
- 여러 지표를 함께 사용하면 가장 큰 값을 택한다 (OR 조건)

```
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: {kind: Deployment, name: shop-api}
  minReplicas: 3      # 존이 3개면 최소 3 - 존당 1개는 유지
  maxReplicas: 30     # 노드 여유·DB 커넥션 수를 고려한 상한
  metrics:            # 여러 지표를 쓰면 계산 결과 중 가장 큰 값을 택한다
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 70}}
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0      # 즉시 확대
      policies: [{type: Percent, value: 100, periodSeconds: 30}] # 30초에 2배까지
    scaleDown:
      stabilizationWindowSeconds: 300    # 5분간 낮은 값이 유지돼야 축소
      policies: [{type: Percent, value: 20, periodSeconds: 60}] # 1분에 20%씩만
```

[주의] `maxReplicas` 는 **DB 커넥션 풀**과 함께 계산한다. 파드 30개 × 풀 20 = 600 커넥션을 DB 가 감당하는가?

VPA 와 Cluster Autoscaler

확장은 두 계층에서 일어난다. 파드를 늘리는 것(HPA)과 노드를 늘리는 것(Cluster Autoscaler). 둘 다 있어야 실제로 늘어난다.

- HPA 만 켜면 스케줄링될 노드가 없어 파드가 **Pending** 에 쌓인다
- VPA 와 HPA 를 같은 지표(CPU)로 동시에 사용하면 충돌한다

도구	무엇을 바꾸나	특징	주의
HPA	파드 개수	가장 기본. 15초 주기	requests 기준으로 계산
VPA	파드의 requests 값	실사용에 맞춰 자동 조정	적용 시 파드 재시작. HPA 와 CPU 중복 금지
Cluster Autoscaler	노드그룹의 노드 수	ASG(Auto Scaling Group) 단위. 미리 정의한 타입만	노드그룹 설계가 필요

[정리] 실무 조합은 HPA + Cluster Autoscaler. VPA 는 updateMode: Off 로 권고값만 받아 보는 용도가 안전하다.

스케줄링 진단 지도

증상 / 메시지	원인	대응
<code>Insufficient cpu/memory</code>	요청량 합이 Allocatable 초과	requests 하향 · 노드 추가
<code>untolerated taint</code>	노드에 테인트, 파드에 톨러레이션 없음	toleration 추가 또는 다른 노드로
<code>didn't match Pod anti-affinity</code>	안티어피니티 required 로 자리 없음	preferred 로 완화 · 노드 추가
<code>didn't match topology spread</code>	maxSkew 위반	<code>ScheduleAnyway</code> 로 완화
<code>node(s) had volume node affinity conflict</code>	PVC 가 다른 AZ 에 있음	StorageClass 를 WaitForFirstConsumer 로
<code>too many pods</code>	노드 파드 수 상한(VPC CNI)	프리픽스 위임 · 큰 인스턴스
<code>Evicted</code> 다발	노드 Disk/MemoryPressure	limits 재조정 · 노드 디스크 확대
<code>HPA unknown</code>	metrics-server 미설치 또는 requests 없음	metrics-server 확인 · requests 설정

[체크] Pending 원인은 한 번에 여러 개일 수 있다. `0/6 nodes` 뒤에 나오는 사유를 전부 읽는다.

[핵심 요약] 3장 스케줄링

항목	한 줄 정리	실무 포인트
requests vs limits	예약(스케줄러)과 강제(kubelet)	노드 대수와 비용을 정하는 건 requests
CPU limits	종료시키지 않고 100ms 주기로 재운다	<code>nr_throttled/nr_periods > 5%</code> 면 조정한다
메모리 limits	초과하면 즉시 SIGKILL (137)	JVM 은 <code>MaxRAMPercentage</code> 를 꼭 준다
QoS	requests/limits 조합으로 자동 결정	requests 누락 = BestEffort = 먼저 축출된다
Allocatable	Capacity 에서 예약분을 뺀 실제 몫	용량 계산은 Allocatable 로 한다
스케줄링 제어	어피니티(파드가 고름) vs 테인트(노드가 거부)	격리는 테인트 + 어피니티를 함께
topologySpread	maxSkew 로 분산을 숫자로 표현	존은 DoNotSchedule, 노드는 ScheduleAnyway
HPA + 노드 확장	동작하는 계층이 다르다. 둘 다 필요	maxReplicas 는 DB 커넥션과 함께 계산

[체크] '우리 클러스터의 예약률과 실사용률 차이는 몇 %인가?' 답을 모르면 비용이 새고 있을 가능성이 높다.

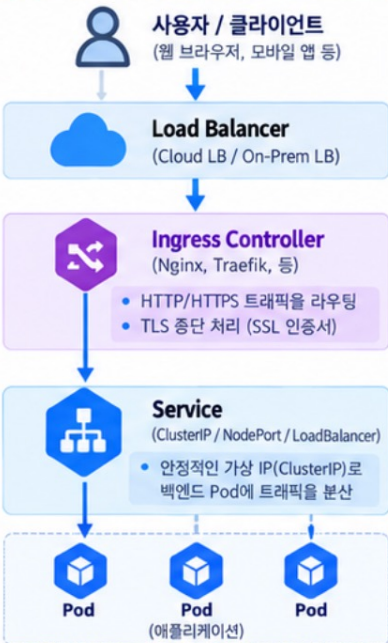
4. 네트워킹 심화

- 쿠버네티스 네트워크 모델의 네 가지 전제
- Service · EndpointSlice · kube-proxy. 트래픽이 실제로 가는 경로
- Amazon VPC CNI 와 IP 고갈. EKS 에서만 겪는 문제
- Ingress(**Nginx**) · Gateway API · CoreDNS · NetworkPolicy

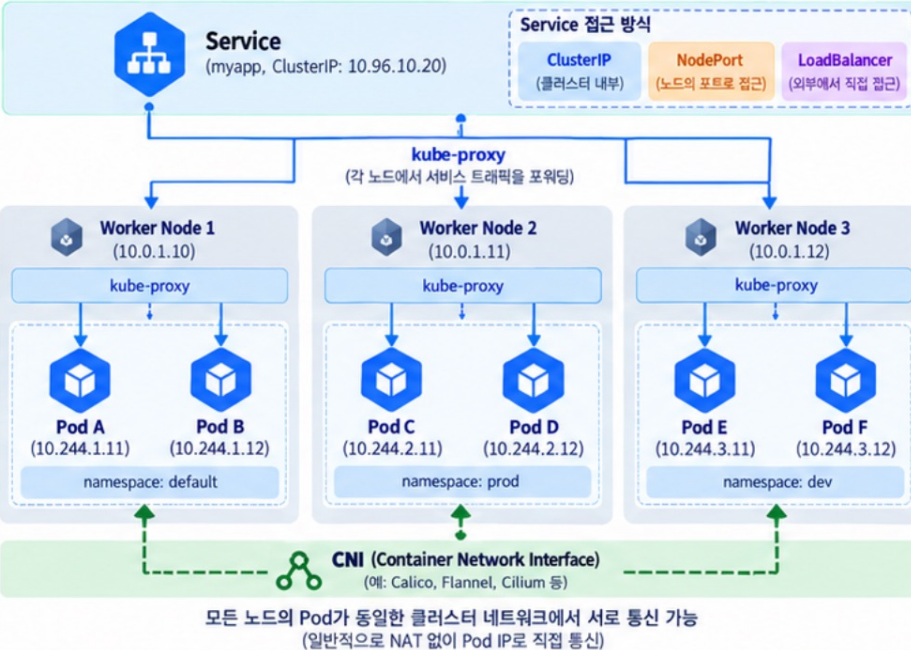
본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

쿠버네티스 네트워크 아키텍처

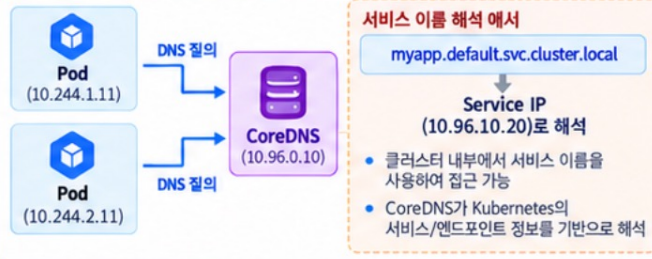
1 외부 트래픽 진입



2 Kubernetes Cluster 내부 네트워크



3 DNS (서비스 이름 해석)



4 NetworkPolicy (통신 범위 제어)



5 주요 트래픽 패턴 | 쿠버네티스에서 발생하는 대표적인 4가지 트래픽 흐름입니다.



6 핵심 포인트



[기초] IP · 포트 · NAT

앞으로 설명 없이 쓸 세 단어를 여기서 한 번 정리한다.

- IP 는 어느 기계인가, 포트는 그 기계의 어느 프로그램인가를 가리킨다
- NAT 는 지나가면서 주소를 바꿔치기하므로, 바뀌고 나면 원래 보낸 쪽을 알 수 없다

용어	무엇인가	쿠버네티스에서 만나는 곳
IP 주소	네트워크에서 기계 하나를 가리키는 번호	파드마다 하나씩 받는다
포트	한 기계 안에서 프로그램을 구분하는 번호	<code>containerPort</code> · <code>targetPort</code>
NAT	지나가면서 출발지나 목적지 주소를 바꾸는 것	kube-proxy 가 ClusterIP 를 파드 IP 로
SNAT	출발지 주소를 바꾸는 것	출발지 IP 가 사라지는 원인 (<code>externalTrafficPolicy</code>)
DNAT	목적지 주소를 바꾸는 것	Service 가 동작하는 방식 그 자체

[정리] 쿠버네티스 네트워킹의 전제는 파드끼리는 NAT 없이 직접 통신한다는 것이다. 다음 장에서 그 뜻을 본다.

네트워크 모델의 전제

쿠버네티스는 네트워크를 직접 구현하는 대신 지켜야 할 규칙 네 가지만 정했다.

- 이 규칙을 만족시키는 것이 CNI 플러그인의 일이고, 구현은 환경마다 다르다
- **NAT 가 없다**는 전제 하나로 도커의 포트 매핑 사고방식을 버려야 한다

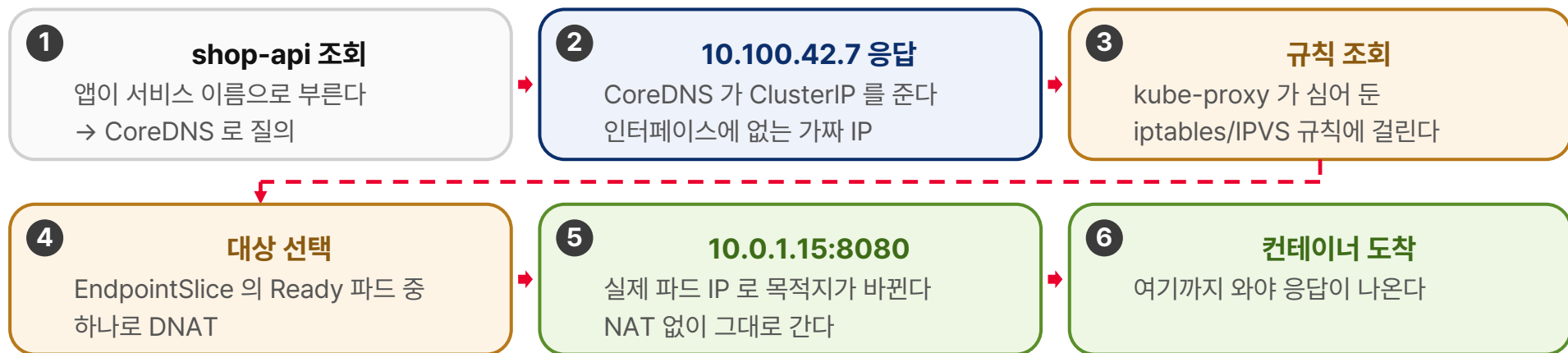
규칙	의미	실무에서의 결과
① 모든 파드는 고유 IP 를 갖는다	포트 충돌을 신경 쓸 필요가 없다	같은 노드에 nginx 파드 10개를 80 포트로 기동할 수 있다
② 파드끼리 NAT 없이 통신한다	A 가 보는 B 의 IP = B 가 아는 자기 IP	출발지 IP 로 호출자를 식별할 수 있다
③ 노드는 모든 파드와 통신한다	kubelet 이 Probe 를 보낼 수 있다	hostNetwork 없이도 헬스체크가 된다
④ 파드가 보는 자기 IP = 남이 보는 IP	IP 를 등록·광고할 수 있다	서비스 디스커버리·클러스터 구성이 가능

[정리] 파드 IP 는 언제든지 바뀐다. 규칙은 '고유하다'이지 '고정된다'가 아니다. 그래서 Service 가 필요하다.

Service 의 고정 주소

Service 는 가상 IP 하나를 주고, 그 뒤에 붙는 파드 목록을 계속 갱신한다.

- ClusterIP 는 어떤 인터페이스에도 없는 가짜 IP 이다
- iptables 규칙만 존재한다
- 그래서 `ping <clusterIP>` 는 실패한다 (정상 동작)



④ 에서 고를 대상이 없으면 그 앞은 전부 정상인데도 트래픽이 멈춘다. EndpointSlice 를 먼저 보는 이유이다.

[체크] 트래픽이 안 갈 때 첫 명령은 `kubectl get endpointslice`. 비어 있으면 셀렉터나 Probe 문제이다.

Service 타입

기본적으로 Headless 제외한 4가지 타입은 **계단식**이다.

NodePort 는 ClusterIP 를 포함하고, LoadBalancer 는 NodePort 를 포함한다.

- EKS 기준 비용: LoadBalancer 는 서비스마다 NLB 1개
- 매달 요금이 따로 붙는다
- 그래서 외부 노출은 대개 Ingress 하나에 모은다 (뒤에서 다룸)

타입	무엇을 만드나	접근 방법	언제 쓰나
ClusterIP (기본값)	가상 IP	클러스터 내부에서만	대부분의 서비스. 내부 통신
NodePort	+ 모든 노드의 30000~32767 포트	<노드IP>:<노드포트>	직접 사용할 일은 드뭄. LB 의 내부 구현
LoadBalancer	+ 클라우드 LB (EKS→NLB)	LB 의 DNS 이름	L4 노출 · gRPC · TCP 서비스
ExternalName	DNS CNAME 만	이름 그대로	외부 RDS·SaaS 를 내부 이름으로 감싸기
Headless (clusterIP: None)	파드별 DNS A 레코드	파드.서비스.NS.svc	StatefulSet · 클라이언트 측 LB

[주의] LoadBalancer 서비스를 10개 만들면 **NLB 10개 값을 낸다**. Ingress 로 묶으면 NLB 하나로 끝난다.

kube-proxy: iptables vs IPVS

kube-proxy 는 트래픽을 직접 나르지 않는다. 커널에 규칙을 심어 두고 빠진다.

- iptables 모드: 규칙이 **선형 체인**이라 서비스가 많아지면 규칙 갱신이 느려진다
- IPVS 모드: 해시 테이블 기반이라 **대규모에서 유리**하고 알고리즘 선택도 가능

	iptables (기본)	IPVS
규칙 구조	선형 체인 (순차 매칭)	해시 테이블 (O(1))
서비스 1000개 시	규칙 동기화가 수 초~수십 초	거의 일정
부하 분산	무작위(random)만	rr · lc · sh 등 선택 가능
세션 고정	clientIP 기반만	동일 + sh 알고리즘
문제 진단	<code>iptables -t nat -L</code> (길다)	<code>ipvsadm -Ln</code> (읽기 쉽다)
EKS 기본	iptables	설정으로 전환 가능

[체크] 서비스 수가 **1000개 미만**이면 iptables 로 충분하다. 바꾸는 것 자체가 리스크이므로 근거가 있을 때만.

Amazon VPC CNI 와 IP 고갈

EKS 의 파드는 **VPC 사설 IP** 를 직접 받는다. 장점이 크지만 IP 고갈이라는 대가가 따른다.

- 파드가 VPC IP 를 그대로 사용한다
- 보안그룹 · VPC 플로우 로그가 그대로 적용된다
- 노드당 파드 수 = ENI 수 × (ENI당 IP - 1) + 2. 인스턴스 타입이 상한을 정한다

프리픽스 위임: aws-node DaemonSet 에 ENABLE_PREFIX_DELEGATION=true 를 준다

인스턴스 타입	ENI 수	ENI당 IP	최대 파드 수	프리픽스 위임 시
t3.medium	3	6	17	110
m5.large	3	10	29	110
m5.xlarge	4	15	58	234
m5.4xlarge	8	30	234	737
계산식	ENI 수 × (ENI당 IP - 1) + 2		3 × 9 + 2 = 29	/28 블록(16개) 단위

[주의] 서브넷도 함께 본다. /24 서브넷은 IP 251개뿐이다. 파드용 서브넷은 /19 이상으로 정한다.

Ingress 의 L7 경로 분기

Service 가 L4 라면 Ingress 는 **L7** 이다. 호스트와 경로로 나뉘서 LB 하나를 공유한다.

- Ingress 는 규칙일 뿐이고 실제로 일하는 것은 **Ingress Controller** 이다
- 이 과정의 표준은 **Nginx Ingress Controller** 이고, 앞단 NLB 는 하나이다

```
kind: Ingress
metadata:
  name: shop
  annotations:
    # Nginx 설정은 어노테이션으로 넣는다
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/proxy-body-size: 10m    # 기본 1m. 넘으면 413
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60" # 응답 대기. 초
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  rules:
  - host: shop.example.com
    http:
      paths:
      - {path: /api, pathType: Prefix, backend: {service: {name: shop-api,
                                                                    port: {number: 80}}}}
```

[정리] Ingress 가 몇 개든 NLB 는 하나이다. 전부 같은 Nginx 컨트롤러를 거치므로 `group.name` 이 필요 없다.

[그림] 외부 요청이 파드에 닿기까지

외부 요청은 **NLB** 에서 시작해 Ingress Controller 와 Service 를 거쳐 파드에 닿는다.



[주의] **NLB** 는 경로를 보지 않는다. 호스트·경로 판단은 전부 Nginx 컨트롤러 파드가 한다. 이 파드가 멈추면 외부 유입이 한꺼번에 끊긴다.

Ingress vs Gateway API

Ingress의 한계를 넘으려고 만든 후속 표준이 **Gateway API**이다. 아직 전환 중이다.

- Ingress의 문제: 어노테이션 난립, 역할 분리 불가, HTTP 외 프로토콜 표현 불가
- ingress-nginx는 2026-03로 유지보수가 끝났다
- 새로 시작한다면 Gateway API를 먼저 검토한다

항목	Ingress	Gateway API
표준화 수준	핵심 기능만. 나머지는 어노테이션	트래픽 분할·헤더 조작까지 스펙에 포함
역할 분리	하나의 리소스에 전부	GatewayClass(플랫폼) / Gateway(운영) / Route(개발)
프로토콜	HTTP(S) 중심	HTTPRoute · GRPCRoute · TCPRoute · TLSRoute
카나리 배포	컨트롤러별 어노테이션	<code>backendRefs</code> 의 <code>weight</code> 로 표준 지원
성숙도	GA . 어디서나 동작	v1 GA(2023). 컨트롤러 지원이 빠르게 붙는 중
EKS 지원	AWS LB Controller	AWS Gateway API Controller (VPC Lattice)

[체크] 이미 Ingress로 돌고 있으면 당장 급하지는 않다. 다만 **보안 패치가 더 나오지 않으므로** 전환 계획은 세워 둔다.

[기초] DNS 의 이름 해석

파드는 IP 를 외우지 않는다. 이름을 물어보고 그때그때 주소를 받는다.

- `shop-db` 라고 부르면 **CoreDNS** 가 그 순간의 **ClusterIP** 를 알려 준다
- 이름은 계층으로 되어 있다. **서비스.네임스페이스.svc.cluster.local**

```
# 클러스터 안에서 이름이 풀리는 모양
#   shop-db                                같은 네임스페이스면 이것으로 충분하다
#   shop-db.shop                          다른 네임스페이스의 서비스를 부를 때
#   shop-db.shop.svc.cluster.local        완전한 이름 (FQDN)

# 실제로 확인해 본다
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- \
  nslookup shop-db.shop.svc.cluster.local
#   Name:      shop-db.shop.svc.cluster.local
#   Address 1: 10.100.42.7                  ← Service 의 ClusterIP

# 파드 안에 이 설정이 들어 있다 - kubelet 이 넣어 준다
kubectl exec deploy/shop-api -n $NS -- cat /etc/resolv.conf
#   nameserver 10.100.0.10                  ← CoreDNS 의 주소
#   search shop.svc.cluster.local svc.cluster.local cluster.local
```

[정리] 이름을 사용하면 파드가 바뀌어도 코드를 수정하지 않는다. IP 를 설정에 적는 순간 그 이점이 사라진다.

CoreDNS 와 이름 해석

클러스터 내부 DNS 는 CoreDNS 가 담당한다. **ndots:5** 때문에 조회가 여러 번 나간다.

- 서비스 이름 형식: `<svc>.<namespace>.svc.cluster.local`
- 외부 도메인 조회가 느리면 **ndots** 설정을 의심한다 (자주 나오는 성능 문제)

파드의 `/etc/resolv.conf`

```
search shop.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

`ndots:5` 의 뜻 - 점이 5개 미만인 이름은 '짧은 이름'으로 보고 `search` 를 붙여 시도

`api.github.com` (점 2개 < 5) → 다음 순서로 조회한다

<code>api.github.com.shop.svc.cluster.local</code>	NXDOMAIN
<code>api.github.com.svc.cluster.local</code>	NXDOMAIN
<code>api.github.com.cluster.local</code>	NXDOMAIN
<code>api.github.com</code>	OK ← 4번째에야 성공

해결: 끝에 점을 찍거나(`api.github.com.`) 파드에 `dnsConfig` 로 `ndots:2` 지정

[주의] 외부 API 호출이 많은 서비스는 이 때문에 **DNS 질의가 4배**가 된다. CoreDNS 부하와 지연의 흔한 원인이다.

NetworkPolicy 의 기본 허용

쿠버네티스의 기본값은 모든 파드가 모든 파드와 통신 가능하다. 막으려면 정책을 만들어야 한다.

- 정책이 하나라도 붙으면 그 파드는 **화이트리스트 모드**로 바뀐다
- **EKS 기본 VPC CNI 는 1.14+ 부터 NetworkPolicy 를 지원한다** (별도 활성화 필요)

```
kind: NetworkPolicy
metadata: {name: db-allow-api-only, namespace: <내 NS>}
spec:
  podSelector: {matchLabels: {app: shop-db}}    # 이 정책이 보호하는 대상
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector: {matchLabels: {app: shop-api}}    # shop-api 파드만
        ports: [{protocol: TCP, port: 5432}]
    ---
# 네임스페이스 전체를 기본 차단하는 관용구 (먼저 이걸 깔고 하나씩 연다)
kind: NetworkPolicy
metadata: {name: default-deny-ingress, namespace: <내 NS>}
spec:
  podSelector: {}    # 모든 파드
  policyTypes: [Ingress]    # ingress 규칙이 하나도 없다 = 전부 차단
```

[주의] 정책은 **합집합**이다. 두 정책이 있으면 둘 중 하나라도 허용하면 통과한다. 거부 규칙은 없다.

NetworkPolicy 실전 패턴

정책을 잘못 작성하면 **DNS** 가 막혀서 전부 중단된다. 가장 흔한 사고이다.

- egress 를 막을 때는 **kube-system** 의 **CoreDNS** 로 가는 **53** 포트를 반드시 허용한다
- `namespaceSelector` 는 이름이 아니라 네임스페이스 **라벨**로 고른다

```
# egress 를 제한하되 DNS 는 살려 둔다 - 이 블록이 없으면 이름 해석이 전부 실패한다
spec:
  podSelector: {matchLabels: {app: shop-api}}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: kube-system}
            podSelector: {matchLabels: {k8s-app: kube-dns}}
        ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
    - to:
        # DB 로 나가는 것
        - podSelector: {matchLabels: {app: shop-db}}
        ports: [{protocol: TCP, port: 5432}]
```

[체크] 정책 적용 후 첫 확인은 **DNS**. `nslookup shop-db` 가 안 되면 위 블록이 빠진 것이다.

externalTrafficPolicy

외부에서 온 트래픽의 **출발지 IP** 를 보존할 것인가, **균등 분산을 택할** 것인가. 둘 다는 안 된다.

- **Cluster**(기본): 균등 분산되지만 **SNAT 때문에 출발지 IP 가 노드 IP 로 바뀐다**
- **Local**: 출발지 IP 보존. 대신 **파드가 없는 노드로 온 트래픽은 버려진다**

	Cluster (기본)	Local
출발지 IP	노드 IP 로 치환됨(SNAT)	원래 클라이언트 IP 유지
홉 수	2홉 가능(다른 노드로 전달)	1홉 (자기 노드 파드만)
분산	전체 파드에 균등	노드별 파드 수에 비례해 불균등
파드 없는 노드	다른 노드로 전달	패킷 폐기 → LB 헬스체크 실패로 제외됨
필요 조건	없음	topologySpread 로 모든 노드에 고르게 배치

[정리] IP 타깃 NLB 에서는 이 설정이 출발지 IP 를 살리지 못한다. 노드를 거치지 않아 SNAT 자체가 없다.
`preserve_client_ip.enabled=true` 또는 PROXY protocol 로 해결한다.

서비스 간 통신과 커넥션

HTTP Keep-Alive 와 gRPC 는 **연결을 재사용**한다. 그래서 파드를 늘려도 부하가 나뉘지 않는다.

- Service 의 부하 분산 단위는 요청이 아니라 **연결**이다
- gRPC 는 하나의 연결에 여러 요청을 다중화하므로 **한 파드에 몰린다**

증상	원인	대응
파드를 늘려도 특정 파드만 CPU 가 높다	커넥션 재사용	클라이언트 커넥션 TTL 설정 · 주기적 재연결
gRPC 부하가 나뉘지 않는다	HTTP/2 다중화	클라이언트 측 LB(headless + DNS) 또는 서비스 메시
배포 후에도 옛 파드로 트래픽	기존 연결이 유지됨	preStop + graceful shutdown
504 가 간헐적	idle timeout 불일치	NLB idle(350s) < 앱 keep-alive 로 맞추다
커넥션 수가 계속 증가	풀 크기 × 파드 수	HPA maxReplicas 와 함께 계산

[주의] NLB 의 idle timeout 은 기본 350초이다. 앱의 keep-alive 가 그보다 짧으면 이미 끊긴 연결로 요청이 들어가 간헐적 502 가 발생한다. 앱 쪽을 더 길게 설정한다.

EndpointSlice 와 서비스 디스커버리

`Endpoints` 가 보통 사용되어 왔으나, `EndpointSlice` 가 대규모에서 쓰이는 실제 구조이다.

- 파드 1000개면 `Endpoints` 객체 하나가 거대해져 **하나만 바뀌어도 전체가 전송된다**
- `EndpointSlice` 는 **100개씩 쪼개** 변경 전파량을 줄인다

```
kubectl get endpointslice -n $NS -l kubernetes.io/service-name=shop-api
# NAME                ADDRESSTYPE  PORTS    ENDPOINTS                AGE
# shop-api-h7k2p      IPv4         8080     10.0.1.15,10.0.2.31,...  12m

# 각 엔드포인트의 준비 상태까지 보인다 - readinessProbe 결과가 여기 반영된다
kubectl get endpointslice shop-api-h7k2p -n $NS -o yaml | grep -A4 conditions
#   conditions:
#     ready: true           ← false 면 트래픽이 안 간다
#     serving: true
#     terminating: false   ← 종료 중인 파드 (preStop 구간)
```

[체크] `terminating: true` 인 엔드포인트가 오래 남아 있으면 `terminationGracePeriodSeconds` 가 과하게 길다는 신호이다.

Service 없이 직접 찾기

DNS 와 환경변수 두 방법이 있다. 환경변수는 순서 의존성 때문에 사용하지 않는다.

- 파드 시작 시점의 Service 만 환경변수로 주입되고 나중에 만든 것은 빠진다
- 환경변수 방식은 레거시 호환용이므로 항상 DNS 를 사용한다

방법	형식	문제	사용하는가
DNS (권장)	<code>shop-db.shop.svc.cluster.local</code>	없음	항상
DNS 짧은 이름	<code>shop-db</code> (같은 네임스페이스)	<code>ndots</code> 로 조회 증가	내부 통신에 편함
환경변수	<code>SHOP_DB_SERVICE_HOST</code>	파드보다 나중에 만든 Service 는 없다	아니오
파드 IP 직접	<code>10.0.1.15</code>	파드가 사라지면 끝	아니오
headless + DNS	<code>shop-db-0.shop-db</code>	개별 파드 지정	StatefulSet
ExternalName	외부 주소를 내부 이름으로	CNAME 만. 포트 변환 없음	외부 RDS 감싸기

[주의] 환경변수 주입은 Service 가 많으면 파드 환경변수가 수백 개가 된다. `enableServiceLinks: false` 로 끌 수 있다.

멀티 클러스터·하이브리드 연결

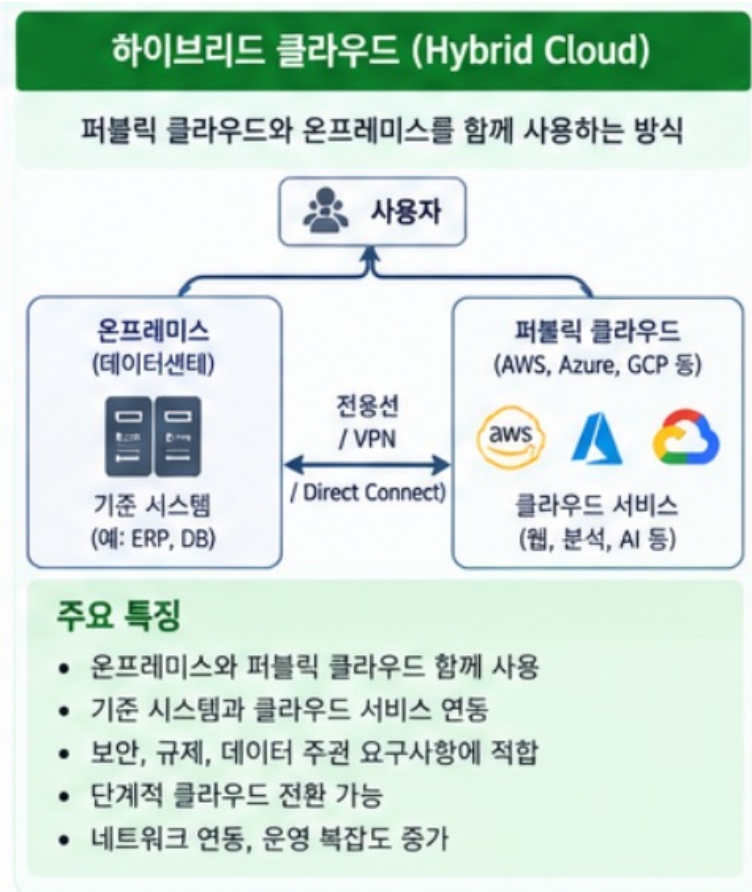
클러스터를 넘는 통신은 쿠버네티스가 해결해 주지 않는다. 네트워크 설계의 영역이다.

- VPC CNI 덕분에 EKS 파드는 **VPC 안의 다른 자원과 직접 통신**한다
- 온프레미 연동은 **IP 대역 충돌**부터 확인한다 (겹치면 되돌리기 어렵다)

연결 대상	방법	주의
같은 VPC 의 RDS·ElastiCache	보안그룹만 열면 끝	파드 IP 가 VPC IP 라 그대로 된다
다른 VPC	VPC Peering · Transit Gateway	CIDR 이 겹치면 불가
온프레미	Direct Connect · Site-to-Site VPN	파드 CIDR 을 온프레미에 광고 해야 한다
다른 EKS 클러스터	VPC 연결 + Service 노출	클러스터 간 DNS 는 자동이 아니다
외부 SaaS	NAT Gateway 경유	고정 출구 IP 가 필요하면 NAT EIP
클러스터 → 인터넷	NAT GW (프라이빗 서브넷)	NAT 데이터 처리 요금 이 붙는다

[체크] 파드가 외부로 나갈 때의 출발지 IP 는 **NAT Gateway 의 EIP** 다. 상대방 방화벽 허용 목록에 이 IP 를 넣는다.

멀티 클라우드 vs. 하이브리드 클라우드



서비스 메시 이해

1. 개념

서비스 메시(Service Mesh)는 Kubernetes 환경에서 서비스 간 통신을 제어하고 관리하기 위한 인프라 계층으로, 일반적으로 각 Pod에 주입되는 Sidecar Proxy를 사용하여 구현됩니다.

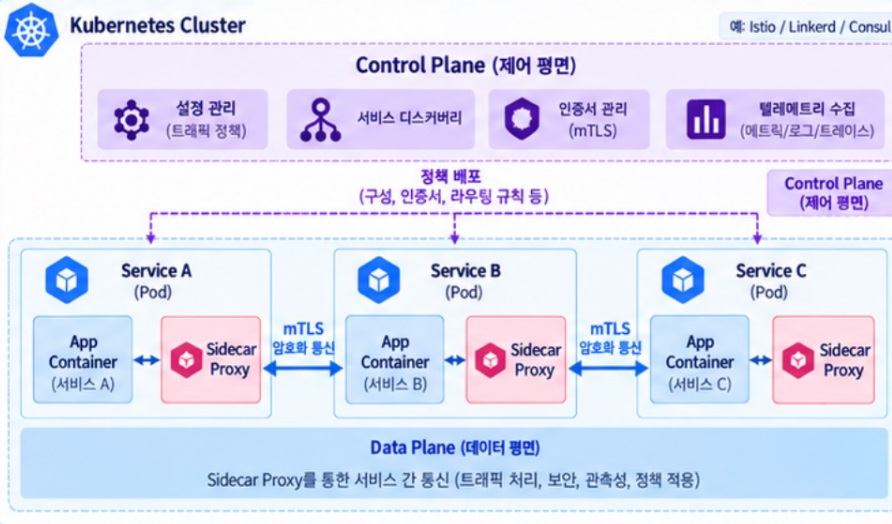
서비스 간 통신 제어
서비스 간 트래픽 흐름을 중앙에서 제어하고 관리

보안 (mTLS)
서비스 간 암호화 통신과 인증/인가 제공

관측성
통신에 대한 메트릭, 로그, 트레이스 수집 및 분석

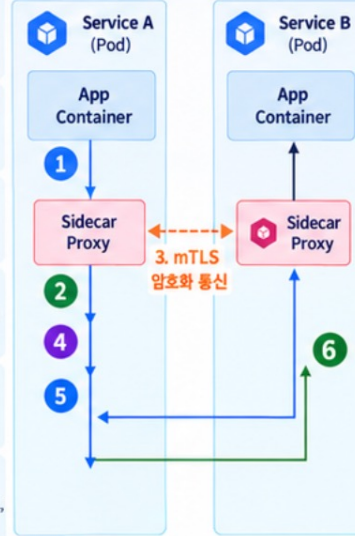
트래픽 정책
라우팅, 카나리 배포, 재시도, 타임아웃 등 다양한 트래픽 정책 적용

2. 기본 구조



3. 동작 흐름 (Service A → Service B 호출 예시)

- Service A 호출**
서비스 A가 서비스 B를 호출 (애플리케이션은 로컬로 요청)
- 정책 확인**
Sidecar A가 Control Plane에서 수신한 정책을 기반으로 트래픽 처리
- mTLS 통신**
Sidecar A가 mTLS로 암호화하여 Sidecar B와 통신
- 트래픽 정책 적용**
라우팅, 재시도, 타임아웃, 서킷 브레이커 등의 정책을 적용
- 관측성 데이터 수집**
메트릭, 로그, 트레이스를 수집하여 Control Plane으로 전송
- Service B 응답 반환**
Sidecar B가 요청을 서비스 B로 전달하고, 응답을 거쳐 다시 서비스 A로 반환



4. 주요 기능

- 트래픽 관리**
 - 라우팅 (A/B, 카나리)
 - 서킷 브레이커
 - 재시도 / 타임아웃
 - 트래픽 분할
- 보안**
 - mTLS 암호화
 - 서비스 인증/인가
 - 서비스 ID (SPIFFE 등)
 - 안전한 서비스 간 통신
- 관측성**
 - 메트릭 수집
 - 로그 수집
 - 분산 트레이싱
 - 서비스 맵 / 대시보드
- 정책 제어**
 - 접근 정책 (인증/인가)
 - 속도 제한 (Rate Limit)
 - 장애 격리 (Fault Isolation)
 - 트래픽 필터링
- 게이트웨이**
 - Ingress Gateway (외부 진입점)
 - Egress Gateway (외부 서비스 접근 제어)

7. 트래픽 흐름 비교



5. 구성 요소

- Control Plane (제어 평면)**
트래픽 정책 관리, 서비스 디스커버리, 인증서 관리, 텔레메트리 수집
예: Istio / Linkerd / Consul
- Sidecar Proxy (데이터 평면)**
각 Pod에 주입되어 트래픽을 가로채고 정책을 적용 (Envoy 등)
- Ingress Gateway**
외부에서 클러스터 내부 서비스로의 진입점 관리
- Egress Gateway**
클러스터 내부에서 외부 서비스로의 접근 제어
- 인증서 / 서비스 아이덴티티**
mTLS를 위한 인증서 발급 및 서비스 신원 관리 (예: SPIFFE, Kubernetes CSR 등)
- 관측성 스택**
메트릭, 로그, 트레이스 수집 및 시각화 (예: Prometheus, Grafana, Jaeger 등)

6. 언제 사용하는가 / 핵심 포인트

- 마이크로서비스 환경에 적합**
다수의 서비스가 상호 통신하는 환경에서 효과적
- 서비스 간 통신이 많을수록 효과적**
동서 트래픽(East-West)이 많은 경우 특히 유용
- 애플리케이션 코드 변경 없이 네트워크 기능 부여**
Sidecar를 통해 네트워크 기능을 인프라 레벨에서 제공
- 운영 복잡도와 리소스 오버헤드 고려 필요**
Sidecar 추가로 인한 리소스 사용량과 운영 복잡도 증가 가능
- 동서 트래픽(East-West) 제어에 강점**
클러스터 내부 서비스 간 통신 제어, 보안, 관측성에 최적화

서비스 메시의 도입 기준

mTLS · 세밀한 트래픽 제어 · 자동 관측이 필요하면 검토한다.

- 사이드카 하나당 CPU · 메모리와 지연이 추가된다
- 파드 수만큼 곱해진다
- 대부분의 요구는 Ingress + NetworkPolicy + OpenTelemetry 로 충족된다

원하는 것	메시 없이	메시가 나온 경우
서비스 간 암호화	앱에서 TLS · VPC 내부 신뢰	전 구간 mTLS 를 앱 수정 없이
트래픽 분할(카나리)	파드 개수 비율 · Nginx canary 가중치	정확한 % · 헤더 기반 라우팅
재시도·타임아웃	앱 라이브러리(Resilience4j 등)	언어와 무관하게 일괄 적용
서비스 간 인가	NetworkPolicy (L3/L4)	L7 인가 (경로·메서드 단위)
분산 추적	OpenTelemetry 계측	계측 없이 자동 수집
관측	Prometheus + 앱 지표	모든 호출의 지표가 자동으로

[체크] 도입 판단 기준: 서비스가 20개를 넘고, 언어가 여러 개이고, mTLS 요구가 있는가? 셋 다 아니면 미룬다.

네트워킹 진단 지도

확인 순서	명령	정상 조건	아니면
① 파드가 Ready 인가	<code>kubectl get pod</code>	READY 1/1	Probe 로
② 엔드포인트에 있는가	<code>get endpointslice</code>	파드 IP 가 목록에	셀렉터 라벨 확인
③ 이름이 풀리는가	<code>nslookup <svc></code>	ClusterIP 반환	CoreDNS · NetworkPolicy
④ 파드에 직접 붙는가	<code>curl <파드IP>:<포트></code>	응답	앱·컨테이너 포트 확인
⑤ Service 로 붙는가	<code>curl <svc>:<포트></code>	응답	targetPort 불일치 · kube-proxy
⑥ 정책이 막는가	<code>get networkpolicy</code>	허용 규칙 존재	정책에 규칙 추가
⑦ 외부에서 붙는가	<code>describe ingress</code>	ADDRESS 존재·타킷 Healthy	서브넷 태그 · 헬스체크 경로

[체크] ④가 되고 ⑤가 안 되면 **targetPort** 와 컨테이너 포트 불일치가 1순위 원인이다.

[핵심 요약] 4장 네트워킹

트래픽이 안 가면 EndpointSlice 부터 본다. 나머지는 그다음이다.

항목	한 줄 정리	실무 포인트
네트워크 모델	파드마다 IP, NAT 없음	고유하지만 고정은 아니다. IP 하드코딩 금지
Service	가상 IP + 갱신되는 파드 목록	ping 이 안 되는 것이 정상. EndpointSlice 를 본다
Service 타입	ClusterIP → NodePort → LoadBalancer 계단식	LB 는 서비스마다 과금. Ingress 로 묶는다
kube-proxy	커널 규칙만 심고 빠진다	프록시가 아니다. 재시작해도 연결은 유지
VPC CNI	파드가 VPC IP 를 직접 받는다	노드당 파드 수 상한·서브넷 크기를 먼저 계산
Ingress	L7 규칙. Nginx Controller 가 처리한다	모든 Ingress 가 NLB 하나를 공유한다
CoreDNS	<code>ndots:5</code> 로 짧은 이름을 확장	외부 도메인 호출이 많으면 질의가 4배
NetworkPolicy	기본은 전부 허용	egress 막을 때 DNS 를 먼저 열어 둔다

[체크] '우리 클러스터에서 아무 파드나 DB 에 붙을 수 있는가?' 답이 '예'라면 오늘 배운 것을 바로 적용할 곳이 있다.

5. 스토리지와 상태 관리

- 컨테이너는 사라진다. 무엇을 밖으로 빼야 하는가
- PV · PVC · StorageClass 와 동적 프로비저닝
- EBS CSI 와 EFS CSI 의 AZ 제약 · 접근 모드 · 성능
- 볼륨 확장 · 스냅샷 · 백업. 데이터가 사라지는 지점

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[기초] 컨테이너 파일의 휘발성

컨테이너는 읽기 전용 이미지 위에 얇은 쓰기 층을 하나 얹어 만든다. 그 층이 컨테이너와 함께 사라진다.

- 이미지 레이어는 **읽기 전용**이라 여러 컨테이너가 같은 레이어를 공유한다
- 컨테이너가 만든 파일은 **맨 위 쓰기 층**에만 쌓이고, 컨테이너가 지워지면 함께 지워진다

컨테이너 A	컨테이너 B	
-----	-----	
쓰기 층 (얇다)	쓰기 층 (얇다)	<- 각자 따로. 지우면 사라진다
=====	=====	
app.jar	(읽기 전용)	<- 공유
JRE	(읽기 전용)	<- 공유
알파인 베이스	(읽기 전용)	<- 공유
=====	=====	

그래서 이런 일이 벌어진다

- 파드를 재시작하면 업로드해 둔 파일이 없어진다
- 같은 이미지로 파드 10개를 띄워도 디스크를 10배 쓰지 않는다
- 로그를 파일로 출력하면 파드와 함께 사라진다

남겨야 하는 것은 볼륨으로 밖에 빼야 한다 -> 이번 장의 주제

[정리] '재시작하면 초기화된다는 컨테이너의 설계이다. 남길 것을 고르는 일이 전부이다.

남는 것과 사라지는 것

컨테이너 파일시스템은 파드와 함께 사라진다. 남겨야 할 것을 먼저 구분한다.

- 재시작으로 사라지는 것과 파드 삭제로 사라지는 것이 다르다
- 비유: 컨테이너 디스크는 호텔 방 안, emptyDir 는 층 공용 창고, PVC 는 집

저장 위치	컨테이너 재시작	파드 삭제	노드 장애	사용하는 곳
컨테이너 파일시스템	사라짐	사라짐	사라짐	임시 파일 · 로그(수집 전)
emptyDir	유지	사라짐	사라짐	사이드카 간 공유 · 캐시 · 스크래치
emptyDir (Memory)	유지	사라짐	사라짐	tmpfs. 빠르지만 메모리 limits 를 소모한다
hostPath	유지	유지	사라짐	노드 로그 수집(DaemonSet)만
PVC (EBS)	유지	유지	유지(같은 AZ)	DB · 업로드 파일 · 상태
PVC (EFS)	유지	유지	유지	여러 파드 공유가 필요할 때
외부 서비스 (S3·RDS)	유지	유지	유지	가능하면 이쪽

[정리] 가장 안전한 선택은 클러스터 밖에 두는 것이다. S3·RDS 로 뺄 수 있으면 그게 최선이다.

PV · PVC · StorageClass

신청서(PVC) · 실물(PV) · 규격서(StorageClass) 로 역할이 구분된다.

- 파드는 PV 를 모른다
- **PVC 만 참조한다.** 환경이 바뀌어도 매니페스트가 같은 이유이다
- 요즘은 StorageClass 가 PV 를 자동 생성한다 (동적 프로비저닝)



개발자가 적는 것은 위 레인의 PVC 하나뿐이고, 아래 레인은 자동으로 만들어진다.

[체크] PVC 가 **Pending** 이면 순서대로 본다. StorageClass 존재 → CSI 드라이버 동작 → 용량·AZ 제약.

EKS StorageClass 설정

EKS 기본 `gp2` 는 Old 하다. `gp3` StorageClass 를 직접 만들어 기본값으로 삼는다.

- `gp3` 는 `gp2` 보다 약 **20% 저렴**하고 IOPS·처리량을 용량과 독립적으로 지정한다
- `volumeBindingMode: WaitForFirstConsumer` 가 **AZ 불일치를 막는 핵심 설정**이다

```
kind: StorageClass
metadata:
  name: gp3
  annotations: {storageclass.kubernetes.io/is-default-class: "true"}
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"          # gp3 는 용량과 무관하게 3000 IOPS 가 기본 포함
  throughput: "125"     # MiB/s
  encrypted: "true"     # 기본으로 켜다. 나중에 켜려면 재생성해야 한다
allowVolumeExpansion: true      # 없으면 나중에 용량을 못 늘린다
reclaimPolicy: Retain           # PVC 를 지워도 볼륨은 남긴다
volumeBindingMode: WaitForFirstConsumer  # 파드가 배치될 AZ 를 보고 볼륨 생성
```

[주의] 기본값 `Immediate` 는 PVC 생성 즉시 **임의 AZ** 에 볼륨을 만든다. 파드가 다른 AZ 로 가면 영원히 Pending 이다.

접근 모드와 EBS vs EFS

EBS 는 **한 노드에만** 붙는다. 여러 파드가 같은 볼륨을 쓰려면 EFS 가 필요하다.

- ReadWriteOnce 의 단위는 파드가 아니라 **노드**이다
- 같은 노드의 파드끼리는 공유된다
- ReadWriteMany 를 지원하는 것은 EKS 에서 EFS 뿐이다

	EBS CSI	EFS CSI
접근 모드	ReadWriteOnce (한 노드)	ReadWriteMany (여러 노드)
AZ 제약	있음 . 볼륨과 노드가 같은 AZ 여야	없음. 리전 단위
성능	낮은 지연. DB 에 적합	NFS 기반. 지연이 큼
용량	미리 지정 (확장 가능)	무제한 · 쓴 만큼 과금
비용	GB 당 저렴	GB 당 약 3배 . 하지만 미사용분은 안 냄
사용하는 곳	DB · 큐 · 인덱스	공유 업로드 파일 · 여러 파드가 읽는 설정
Deployment 와 궁합	replicas 2 이상이면 위험	여러 파드 동시 마운트 OK

[주의] EBS + Deployment + replicas 2 는 흔한 사고이다. 롤링 업데이트 때 새 파드가 볼륨을 확보하지 못해 **배포가 멈춘다**.

StatefulSet 과 볼륨

`volumeClaimTemplates` 는 파드마다 PVC 를 자동 생성한다. 이름 규칙이 정해져 있다.

- PVC 이름 = <템플릿>-<StatefulSet>-<인덱스>
- StatefulSet 을 지워도 PVC 는 남는다
- 의도된 안전장치이자 비용 누수 지점이다

```
kubectl get pvc -n $NS
# NAME          STATUS  VOLUME          CAPACITY  STORAGECLASS
# data-shop-db-0 Bound    pvc-a1b2...      20Gi      gp3
# data-shop-db-1 Bound    pvc-c3d4...      20Gi      gp3
# data-shop-db-2 Bound    pvc-e5f6...      20Gi      gp3

# 스케일 다운해도 PVC 는 남는다 - 다시 올리면 원래 데이터를 그대로 쓴다
kubectl scale statefulset shop-db -n $NS --replicas=1
kubectl get pvc -n $NS          # 여전히 3개

# 1.27+ 자동 정리 설정 (기본은 Retain = 남김)
# spec:
#   persistentVolumeClaimRetentionPolicy:
#     whenScaled: Delete      # 스케일 다운 시 PVC 삭제
#     whenDeleted: Retain     # StatefulSet 삭제 시에는 유지
```

reclaimPolicy 와 데이터의 운명

PVC 를 지웠을 때 실제 볼륨을 지울 것인가 남길 것인가. 한 번 지우면 되돌릴 수 없다.

- **Delete**(대부분의 기본값): PVC 삭제 → PV 삭제 → EBS 볼륨 삭제 → 데이터 소멸
- **Retain**: PVC 삭제 → PV 는 Released 로 남음
- 볼륨과 데이터는 보존된다

상황	Delete	Retain
PVC 삭제	볼륨·데이터 즉시 삭제	PV 가 Released 로 남음. 데이터 보존
네임스페이스 삭제	그 안의 모든 볼륨 삭제	전부 보존
실수 복구	불가능 (스냅샷이 없으면)	PV 를 새 PVC 에 다시 연결 가능
재사용	자동	수동. <code>claimRef</code> 를 비워야 다시 바인딩된다
비용	사용하지 않으면 안 냄	방치하면 계속 과금
권장	개발·테스트 · 캐시	운영 데이터는 전부 여기

[주의] 운영 클러스터에서 `kubectl delete namespace` 는 그 안의 모든 PVC 를 지운다. Delete 정책이면 데이터도 함께 사라진다.

볼륨 스냅샷

- VolumeSnapshotClass · VolumeSnapshot · VolumeSnapshotContent 세 리소스
- 같은 리전에 있고 정합성도 보장하지 않으므로 **스냅샷은 백업이 아니다**

```
kind: VolumeSnapshot
metadata: {name: shop-db-0-20260805, namespace: <내 NS>}
spec:
  volumeSnapshotClassName: ebs-snapshot
  source: {persistentVolumeClaimName: data-shop-db-0}
---
# 복원 - 스냅샷을 dataSource 로 지정해 '새' PVC 를 만든다
kind: PersistentVolumeClaim
metadata: {name: data-shop-db-0-restored, namespace: <내 NS>}
spec:
  storageClassName: gp3
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 20Gi}} # 원본 이상이어야 한다
  dataSource: {apiGroup: snapshot.storage.k8s.io,
               kind: VolumeSnapshot, name: shop-db-0-20260805}
```

[주의] DB 볼륨은 쓰기를 멈추거나 DB의 백업 기능을 함께 사용해야 정합성이 맞는다. 스냅샷만으로는 깨진 파일을 복사할 수 있다.

AZ 제약과 배치

EBS 볼륨은 **AZ** 를 넘지 못한다. 그래서 스토리지가 파드의 배치를 제약한다.

- PV 에 박히는 nodeAffinity 때문에 그 AZ 의 노드에만 스케줄된다
- AZ 장애 시 그 AZ 의 EBS 도 함께 사용하지 못한다
- 복제는 애플리케이션 몫이다

```
# PV 에 박힌 AZ 제약을 확인한다
kubectl get pv pvc-a1b2c3 -o jsonpath='{.spec.nodeAffinity}' | jq
# {"required":{"nodeSelectorTerms":[{"matchExpressions":[{"key":"topology.ebs.csi.aws.com/zone","operator":"In",
#   "values":["ap-northeast-2a"]}]}]}}
#
# → 이 볼륨을 쓰는 파드는 2a 존 노드에만 갈 수 있다

# 앞에서 본 그 예러가 여기서 나온다
# 0/6 nodes are available: 4 node(s) had volume node affinity conflict
# → 2a 존 노드에 자리가 없다는 뜻
```

[정리] AZ 다중화는 EBS 로 할 수 없다. DB 복제(StatefulSet 3대) 또는 RDS Multi-AZ 로 해결한다.

EFS 로 여러 파드가 공유하기

ReadWriteMany 가 필요하면 EFS 다. 동적 프로비저닝은 접근점(**Access Point**) 으로 한다.

- EFS 는 미리 만들어 두고 StorageClass 에 **파일시스템 ID** 를 적는다
- 파드마다 격리된 디렉터리를 주려면 `provisioningMode: efs-ap`

```
kind: StorageClass
metadata: {name: efs-shared}
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap          # PVC 마다 접근점을 만들어 디렉터리를 격리
  filesystemId: fs-0a1b2c3d4e      # 미리 만들어 둔 EFS
  directoryPerms: "700"
  uid: "1000"
  gid: "1000"
reclaimPolicy: Retain
---
```

PVC 쪽은 EBS 와 문법이 같다. accessModes 만 다르다.

```
#   accessModes: [ReadWriteMany]
#   storageClassName: efs-shared
```

[주의] EFS 마운트 대상(mount target)의 **보안그룹**이 노드에서 **2049** 포트를 허용해야 한다. 안 되면 마운트에서 멈춘다.

hostPath 와 로컬 볼륨

편해 보이지만 **노드에 묶이고 보안 구멍이 되기 때문에 쓸 수 있는 경우가 매우 좁다.**

- hostPath 는 노드의 파일시스템을 그대로 연다
- 파드가 노드를 장악할 수 있다
- 정당한 용도는 사실상 DaemonSet 의 로그·메트릭 수집뿐이다

용도	위험	대안
<code>/var/run/docker.sock</code> 마운트	컨테이너 탈출 = 노드 장악	CI 는 Kaniko·BuildKit 사용
<code>/</code> 또는 <code>/etc</code> 마운트	노드 설정 변조 가능	필요한 하위 경로만 readOnly
앱 데이터 저장	노드가 사라지면 데이터 소실	PVC (EBS/EFS)
노드 간 공유 목적	공유되지 않는다(노드마다 별개)	EFS
로그 수집 (<code>/var/log</code>)	낮음 (readOnly)	정당한 용도
로컬 NVMe 활용	노드 종속. 재배치 시 소실	local PV + 명시적 nodeAffinity

[주의] Pod Security Standards `baseline` 은 대부분의 `hostPath` 를 차단한다. 기본으로 막는 것이 맞다.

사용하지 않는 볼륨 찾기

Retain 정책을 사용하면 반드시 정리가 필요하다. 정기 점검을 절차로 만든다.

- Released PV 와 파드가 참조하지 않는 PVC 가 정리 대상이다
- EBS 20Gi 하나가 월 약 2,000원이다. 개수가 쌓이면 무시할 수 없다

① 아무 파드도 쓰지 않는 PVC 찾기

```
used=$(kubectl get pods -A -o json |  
    jq -r '.items[].spec.volumes[]?.persistentVolumeClaim.claimName' | sort -u)  
kubectl get pvc -A --no-headers | while read ns name rest; do  
    echo "$used" | grep -qx "$name" || echo "미사용: $ns/$name"  
done
```

② Released 상태로 남은 PV (Retain 정책의 흔적)

```
kubectl get pv --field-selector=status.phase=Released
```

③ 실제 EBS 볼륨 중 어디에도 안 붙은 것

```
aws ec2 describe-volumes --filters Name=status,Values=available \  
    --query 'Volumes[].{ID:VolumeId,Size:Size}' --output table
```

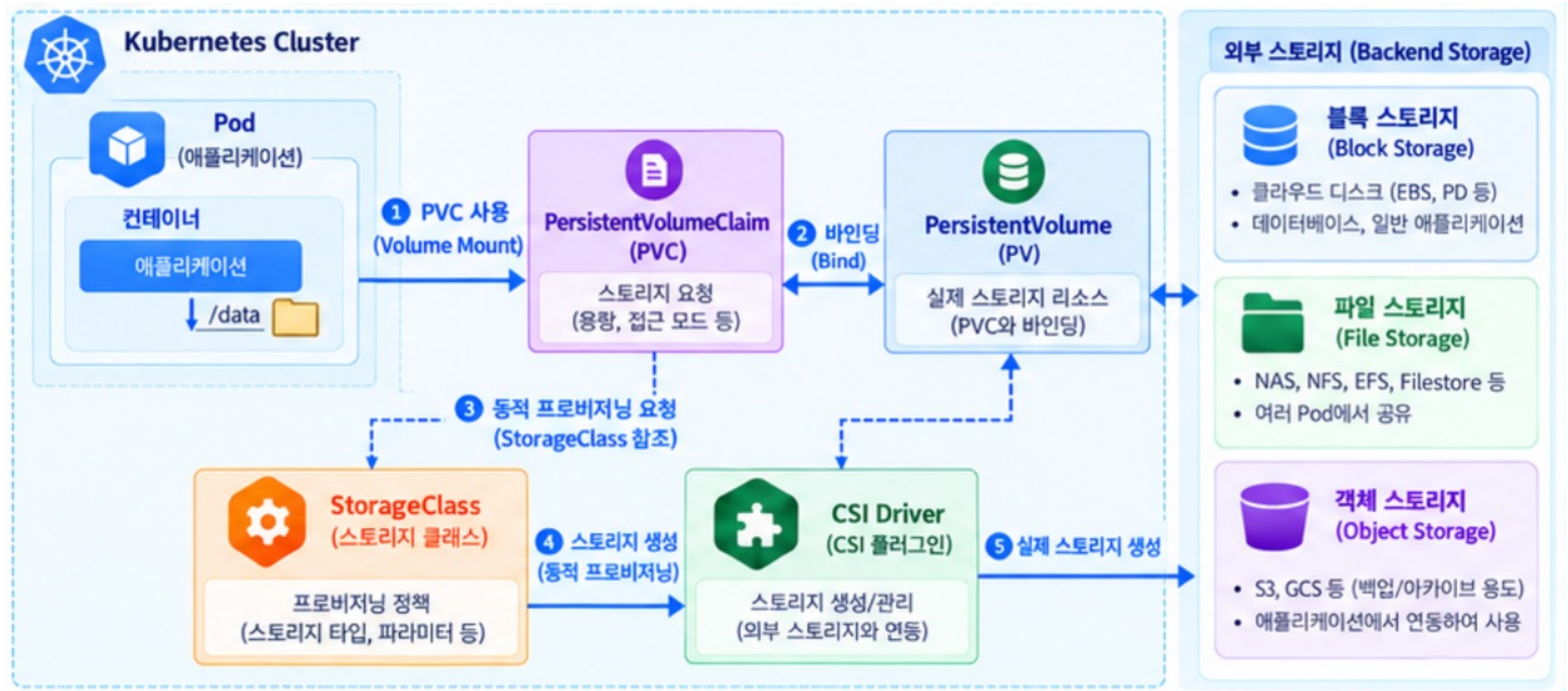
[체크] ③의 available 볼륨은 어느 인스턴스에도 붙어 있지 않다는 뜻이다. 대부분 정리 대상이다.

스토리지 진단 지도

증상	첫 확인	흔한 원인
PVC 가 Pending	<code>describe pvc</code> 이벤트	SC 없음 · WaitForFirstConsumer(정상) · 쿼터 초과
파드가 ContainerCreating 지속	<code>describe pod</code> 이벤트	attach 실패 · AZ 불일치 · Multi-Attach
Multi-Attach error	PVC 를 사용하는 파드 목록	RWO 볼륨을 여러 노드에서 사용
volume node affinity conflict	PV 의 nodeAffinity	볼륨 AZ 에 배치 가능한 노드 없음
볼륨 확장이 안 됨	SC 의 <code>allowVolumeExpansion</code>	false · EBS 6시간 제한 · 축소 시도
PVC 지웠는데 PV 가 Released	<code>get pv</code>	정상(Retain). <code>claimRef</code> 를 비우면 재사용
디스크 가득 참	<code>kubectl exec -- df -h</code>	확장 필요 · 로그 파일 누적
파드는 뜨는데 데이터가 없음	마운트 경로 확인	mountPath 오타 · subPath 착오

[체크] `describe pvc` 와 `describe pod` 의 **이벤트 섹션**만 정확히 읽으면 위 대부분이 즉시 판별된다.

스토리지 아키텍처



[핵심 요약] 5장 스토리지

운영 StorageClass 는 Retain + WaitForFirstConsumer.

항목	한 줄 정리	실무 포인트
무엇이 남는가	컨테이너 FS < emptyDir < PVC < 외부	가능하면 S3·RDS 로 클러스터 밖에
PVC / PV / SC	신청서 · 실물 · 규격서	파드는 PVC 만 참조. 환경 이식성의 근거
gp3 StorageClass	EKS 기본 gp2 는 직접 교체한다	<code>expansion</code> · <code>encrypted</code> 는 처음에 켜다
WaitForFirstConsumer	파드 배치 후 볼륨을 만든다	AZ 불일치를 원천 차단하는 한 줄
접근 모드	RWO 는 '한 노드'다	EBS + Deployment replicas 2 = 배포 멈춤
reclaimPolicy	PVC 삭제 시 볼륨을 어떻게 할지	운영은 Retain. Delete 면 네임스페이스 삭제로 데이터 소멸
스냅샷	표준 API 로 EBS 스냅샷	스냅샷은 백업이 아니다. 리전·정합성
AZ 제약	EBS 는 존을 넘지 못한다	AZ 다중화는 앱 복제나 RDS Multi-AZ 로

[체크] '지금 운영 클러스터의 StorageClass 정책은 Delete 인가 Retain 인가?' 모른다면 지금 확인한다.

6. 설정과 Secret 관리

- 설정을 이미지에서 분리하는 이유와 방법
- ConfigMap · Secret 의 주입 방식과 각각의 대가
- Secret 은 왜 암호화가 아닌가. etcd 암호화와 IAM
- External Secrets 로 AWS Secrets Manager 와 연동

[기초] 설정을 주입하는 두 가지 방법

환경변수와 파일은 갱신 방식이 다르다. 이 차이가 이 장 내내 문제를 만든다.

- 환경변수는 시작할 때 한 번 읽는다
- 이후에는 바뀌지 않는다
- 파일은 다시 읽을 수 있다. 대신 앱이 다시 읽도록 만들어야 한다

	환경변수	파일 마운트
읽는 시점	프로세스 시작 시 한 번	앱이 읽을 때마다
운영 중 변경	반영되지 않는다	약 60초 뒤 파일이 바뀐다
앱이 할 일	없음	다시 읽는 코드가 필요하다
적합한 것	거의 안 바뀌는 값 · 접속 정보	자주 바뀌는 값 · 큰 설정
흔한 오해	'재배포 없이 바뀔 것'	'파일만 바뀌면 앱도 알 것'

[정리] 둘 다 파드를 다시 뜨게 하지 않으면 확실하지 않다. 다음 장들이 그 방법을 다룬다.

설정을 이미지에서 분리하는 이유

같은 이미지가 개발·스테이징·운영에서 그대로 돌아야 한다. 그것이 분리의 목적이다.

- 이미지에 설정이 박히면 환경마다 다시 빌드해야 한다
- 테스트한 것과 배포한 것이 달라진다
- 12요소 앱의 원칙: 설정은 환경에 두고 코드에 두지 않는다

설정 종류	어디에 두나	이유
DB 호스트 · 큐 주소	ConfigMap	환경마다 다르고 민감하지 않다
기능 플래그 · 타임아웃	ConfigMap	운영 중 조정이 필요하다
DB 비밀번호 · API 키	Secret (+ 외부 저장소)	노출 시 피해가 크다
TLS 인증서	Secret (kubernetes.io/tls)	전용 타입이 있다
레지스트리 자격증명	Secret (dockerconfigjson)	imagePullSecrets 로 참조
빌드 시점 상수	이미지 안	환경과 무관하고 바뀌지 않는다
대용량 파일(모델·사전)	볼륨 또는 S3	etcd 1.5MiB 한도에 걸린다

[정리] 판단 기준은 두 가지이다. 환경마다 달라지는가, 노출되면 피해가 큰가.

ConfigMap 주입 방식 세 가지

세 방식은 반영 시점이 다르다. 이 차이가 운영 중 설정 변경의 성패를 가른다.

- 환경변수는 시작 시 한 번
- 이후 변경은 절대 반영되지 않는다
- 볼륨 마운트는 약 60초 뒤 파일이 갱신된다. 앱이 다시 읽어야 의미가 있다

방식	문법	변경 반영	사용하는 곳
개별 환경변수	<code>valueFrom.configMapKeyRef</code>	안 됨 (재시작 필요)	값 몇 개만 사용할 때
전체 환경변수	<code>envFrom.configMapRef</code>	안 됨	키가 많고 전부 필요할 때
볼륨 마운트	<code>volumes.configMap</code>	약 60초 뒤 파일 갱신	설정 파일(yaml-properties) 통째로
subPath 마운트	<code>volumeMounts.subPath</code>	안 됨 (심볼릭 링크 미갱신)	기존 파일 옆에 하나만 추가할 때

```
envFrom:
- configMapRef: {name: shop-config}      # 모든 키를 환경변수로
volumeMounts:
- {name: cfg, mountPath: /app/config}    # 파일로 - 갱신이 반영된다
```

[주의] subPath 를 사용하면 갱신이 반영되지 않는다. 편리해 보이지만 운영 중 변경이 필요하면 사용하지 않는다.

설정이 바뀌면 다시 뜨게 하기

ConfigMap 을 수정해도 **Deployment** 는 아무 일도 하지 않는다. 파드 템플릿이 안 바뀌기 때문이다.

- 파드 템플릿 해시가 같으면 새 ReplicaSet 이 만들어지지 않는다
- 관용구: 설정 내용의 체크섬을 어노테이션에 넣는다
- 내용이 바뀌면 템플릿도 바뀐다

① 체크섬 어노테이션 - 설정 내용의 해시를 파드 템플릿에 넣는다

```
spec:
  template:
    metadata:
      annotations:
        checksum/config: "a1b2c3d4e5f6"    # 파이프라인에서 계산해 넣는다
```

② kubectl 만으로 - 가장 단순하고 확실하다

```
# kubectl create cm shop-config --from-file=app.yaml -o yaml --dry-run=client \
# | kubectl apply -f -
# kubectl rollout restart deploy/shop-api -n $NS
```

[체크] 설정 변경 절차에 **rollout restart** 가 포함돼 있는가? 없으면 반영 누락이 반드시 발생한다.

Secret 과 base64 인코딩

Secret 의 기본은 **base64 인코딩**이다. 인코딩은 누구나 되돌릴 수 있다.

- `kubectl get secret -o yaml + base64 -d` 면 그대로 평문이 나온다
- 실질적인 보호는 셋이 한다
- etcd 암호화 · RBAC · 외부 저장소

```
# 누구나 되돌릴 수 있다 - 이것이 base64 의 성질이다
kubectl get secret shop-db-cred -n $NS -o jsonpath='{.data.password}' | base64 -d
#  S3cretP@ssw0rd!           ← 평문

# 그래서 실제 보호는 이 셋으로 한다
# ① etcd 저장 시 암호화 - EKS: KMS 키로 Secret 암호화(클러스터 생성 시 설정)
# ② RBAC 으로 접근 제한 - secrets 리소스에 get/list 를 아무에게나 주지 않는다
# ③ 외부 저장소로 이관 - AWS Secrets Manager + External Secrets Operator

# 누가 Secret 을 읽을 수 있는지 확인
kubectl auth can-i get secrets -n $NS --token=$(kubectl create token default -n $NS)
```

[주의] Secret 을 환경변수로 주입하면 `kubectl describe pod`·크래시 덤프·`/proc/<pid>/environ` 에 **평문으로 노출**될 수 있다.

Secret 타입과 사용

Secret 에는 **타입**이 있고, 타입마다 쿠버네티스가 기대하는 키 이름이 정해져 있다.

- 타입을 맞춰야 **Ingress·imagePullSecrets** 가 자동으로 인식한다
- stringData** 를 사용하면 base64 인코딩을 직접 하지 않아도 된다. 저장 시 자동 변환된다

타입	필요한 키	사용하는 곳
Opaque (기본)	자유	일반 시크릿번호 · API 키
kubernetes.io/tls	tls.crt, tls.key	Ingress TLS · 서비스 간 mTLS
kubernetes.io/dockerconfigjson	.dockerconfigjson	프라이빗 레지스트리 pull
kubernetes.io/basic-auth	username, password	기본 인증
kubernetes.io/service-account-token	(자동 생성)	1.24+ 는 자동 발급 안 됨

```
kubectl create secret generic shop-db-cred -n $NS \
  --from-literal=username=shop --from-literal=password='S3cret!'
kubectl create secret tls shop-tls -n $NS --cert=tls.crt --key=tls.key
kubectl create secret docker-registry ecr-cred -n $NS \
  --docker-server=<계정>.dkr.ecr.ap-northeast-2.amazonaws.com --docker-password=...
```

[참고] ECR 은 **imagePullSecrets** 없이도 된다. **노드 IAM 역할에 AmazonEC2ContainerRegistryReadOnly** 가 있으면 자동 인증된다.

클러스터 밖의 시크릿 관리

가장 확실한 보호는 시크릿을 클러스터에 두지 않는 것이다. AWS Secrets Manager 를 원본으로 삼는다.

- External Secrets Operator 가 외부 저장소를 읽어 쿠버네티스 **Secret** 을 자동 생성·갱신한다
- Git 에는 참조만 올라가므로 시크릿 값이 매니페스트에 남지 않는다

```
kind: SecretStore
metadata: {name: aws-sm, namespace: <내 NS>}
spec:
  provider:
    aws: {service: SecretsManager, region: ap-northeast-2,
          auth: {jwt: {serviceAccountRef: {name: eso-sa}}}} # IRSA
---
kind: ExternalSecret # 시크릿 '값'이 아니라 '위치'만 적는다
metadata: {name: shop-db-cred, namespace: <내 NS>}
spec:
  refreshInterval: 1h # 주기적으로 다시 읽어 회전에 대응
  secretStoreRef: {name: aws-sm, kind: SecretStore}
  target: {name: shop-db-cred} # 이 이름으로 K8s Secret 이 생긴다
  data:
    - {secretKey: password, remoteRef: {key: prod/shop/db, property: password}}
```

[정리] 이 매니페스트는 **Git** 에 그대로 올려도 안전하다. 시크릿 값이 아니라 '어디서 가져올지'만 적혀 있다.

설정 관리의 실무 패턴

환경별 차이를 어떻게 관리할 것인가. 두 가지 방식이 있고 각각 적합한 규모가 다르다.

- 복사-붙여넣기는 3개 환경까지 견딘다
- 그 이상은 반드시 어긋난다
- 값만 분리하면 매니페스트는 하나로 유지된다. 구조가 다르면 그때 도구를 고른다

방식	구조	장점	한계
환경별 디렉터리 복사	dev/, stg/, prod/ 전체 복사	이해가 쉽다. 도구 불필요	중복. 반드시 어긋난다
값만 분리	환경별 ConfigMap 만 교체	이미지와 매니페스트가 하나로 유지된다	구조 차이는 담지 못한다

[체크] 환경 차이가 값 몇 개뿐이면 ConfigMap 분리로 충분하다. 전용 도구는 별도 DevOps 과정에서 다룬다.

설정 파일과 애플리케이션 리로드

파일이 갱신돼도 **앱이 다시 읽지 않으면 의미가 없다**. 리로드 방식을 미리 정한다.

- 쿠버네티스는 파일을 갱신할 뿐이다. **앱에 알려 주지 않는다**
- **앱이 감시 · 시그널로 알림 · 파드 재시작** 세 가지 방법이 있다

방식	구현	장점	한계
앱이 파일 감시	inotify · 프레임워크 기능	무중단. 즉시 반영	앱 수정 필요
시그널(SIGHUP)	사이드카가 <code>kill -HUP</code>	nginx 등 기존 앱에 적용 가능	사이드카 추가
파드 재시작	체크섬 어노테이션 · <code>rollout restart</code>	가장 단순하고 확실	롤링 시간 소요
오퍼레이터	Reloader 등이 자동 재시작	자동화됨	의존성 추가
폴링	앱이 주기적으로 다시 읽음	단순	반영 지연

[정리] 특별한 이유가 없으면 **파드 재시작**이 답이다. 무중단 배포가 되어 있으면 무중단이다.

Secret 회전

시크릿번호는 주기적으로 바뀌어야 한다. 바뀔 때 서비스가 안 끊기게 설계한다.

- 회전 중에는 옛 값과 새 값이 둘 다 유효한 구간이 필요하다
- DB 시크릿번호를 한순간에 바꾸면 그 순간의 커넥션이 전부 끊긴다

안전한 회전 순서 (두 개의 자격증명을 번갈아 사용한다)

- | | |
|----------------------|--|
| ① 새 자격증명 생성 | DB 에 shop_user_b 계정 추가 (기존 shop_user_a 유지) |
| ② Secrets Manager 갱신 | 시크릿 값을 b 로 교체 |
| ③ 전파 대기 | External Secrets refreshInterval 만큼 |
| ④ 애플리케이션 재시작 | rollout restart → 새 커넥션은 b 로 |
| ⑤ 검증 | 로그·지표에서 인증 실패가 없는지 |
| ⑥ 옛 자격증명 폐기 | shop_user_a 삭제 |

※ RDS 는 Secrets Manager 회전 람다가 ①~② 를 자동으로 한다
(single-user 는 잠깐 끊김 · alternating-user 는 무중단)

[주의] 한 계정의 시크릿번호만 바꾸는 방식(single-user) 은 회전 순간에 인증 실패가 발생한다. 무중단이 필요하다면 계정 두 개를 번갈아 사용한다.

설정 검증과 안전장치

잘못된 설정은 **배포 전에** 걸러야 한다. 배포 후에 알면 이미 장애이다.

- 설정 오류는 **조용히 실패**한다. 앱은 기동되는데 동작이 이상해진다
- 필수 값 누락은 **시작 시 즉시 실패**하게 만드는 편이 낫다 (fail fast)

단계	무엇을 검증	도구·방법
작성 시	YAML 문법·스키마	IDE 스키마 · kubecconform
커밋 시	필수 키 존재·형식	pre-commit 혹 · CI 린트
렌더링 시	필수 값 누락	CI 에서 렌더링 검사
배포 시	정책 위반	Admission 정책
시작 시	앱이 필수 설정을 확인	없으면 즉시 종료. 가장 확실한 방법
운영 중	설정 변경 감사	감사 로그 · Git 이력

[체크] 앱이 `DB_PASSWORD` 없이도 시작되는가? 그렇다면 **설정 누락**을 배포 후에나 알게 된다.

네임스페이스 설계

설정과 권한의 경계는 네임스페이스이다. 무엇을 기준으로 나눌지 먼저 정한다.

- 네임스페이스는 **RBAC · 쿼터 · 정책 · NetworkPolicy**의 단위이다. **나누는 기준이 곧 통제 단위이다**
- 환경(dev/prod)을 같은 클러스터의 네임스페이스로 나누는 것은 신중히 판단한다

나누는 기준	장점	단점	적합한 경우
팀별	권한·쿼터가 팀에 맞는다	팀이 여러 서비스를 가지면 뒤섞임	가장 흔한 기준
서비스별	격리가 명확	네임스페이스가 많아진다	서비스가 크고 독립적
환경별	클러스터 하나로 절약	격리가 약하다	비운영 환경만
환경별 클러스터	완전한 격리	비용·운영 부담 2배	운영과 비운영은 분리

네임스페이스 shop-prod

함께 달려 오는 것

RBAC Role

ResourceQuota

LimitRange

NetworkPolicy

[주의] 운영과 개발을 같은 클러스터에 두면 개발 실수가 운영에 영향을 준다. 노드 자원 · CRD · 애드온이 공유된다.

설정 · Secret 진단 지도

증상	첫 확인	흔한 원인
ConfigMap 을 고쳤는데 반영 안 됨	주입 방식 확인	환경변수 방식 · subPath 사용
파드가 <code>CreateContainerConfigError</code>	<code>describe pod</code> 이벤트	ConfigMap/Secret 이름 오타 · 키 없음
Secret 이 마운트 안 됨	네임스페이스 확인	Secret 은 네임스페이스를 넘지 못한다
<code>ImagePullBackOff</code> (프라이빗)	<code>imagePullSecrets</code> 확인	ECR 이면 노드 IAM 정책 확인
External Secret 이 안 만들어짐	<code>describe externalsecret</code>	IRSA 권한 · 경로 오타 · SecretStore 오류
환경변수에 값이 없음	<code>kubectl exec -- env</code>	<code>optional: true</code> 로 조용히 무시됨
Ingress TLS 가 안 붙음	Secret 타입 확인	타입이 <code>Opaque</code> · 키 이름 불일치
앱이 설정 파일을 못 찾음	<code>exec -- ls <경로></code>	mountPath 가 기존 디렉터리를 덮어씀

[주의] 볼륨을 마운트하면 그 경로의 **기존 내용이 가려진다**. `/app` 에 마운트하면 앱 바이너리가 사라진 것처럼 보인다.

[핵심 요약] 6장 설정과 Secret

Secret 은 암호화가 아니다. 보호는 RBAC 과 외부 저장소가 한다.

항목	한 줄 정리	실무 포인트
설정 분리 목적	같은 이미지가 모든 환경에서 돌게	환경마다 빌드하면 테스트한 것과 배포한 것이 달라진다
주입 방식	환경변수는 시작 시 1회, 볼륨은 갱신됨	<code>subPath</code> 는 갱신되지 않는다
변경 반영	ConfigMap 만 수정하면 아무 일도 일어나지 않는다	체크섬 어노테이션 또는 <code>rollout restart</code>
Secret	base64 는 인코딩이지 암호화가 아니다	etcd 암호화 · RBAC · 외부 저장소 세 계층
Secret 타입	타입마다 기대하는 키 이름이 있다	TLS·레지스트리는 전용 타입을 사용한다
ECR 인증	노드 IAM 역할로 자동 인증	<code>imagePullSecrets</code> 가 필요 없다
External Secrets	원본은 AWS, 클러스터에는 복사본	매니페스트를 Git 에 올려도 안전하다
설정 관리 방식	환경별 복사는 3개까지	값만 분리하면 매니페스트가 하나로 유지된다

[체크] '우리 팀에서 Secret 을 읽을 수 있는 사람은 몇 명인가?' `kubectl auth can-i` 로 지금 확인할 수 있다.

7. 보안 심화

- 누가 무엇을 할 수 있는가. 인증 · 인가 · RBAC
- ServiceAccount 와 IRSA. 파드에 액세스 키를 넣지 않는 법
- Pod Security Standards 와 SecurityContext 로 컨테이너 가두기
- Admission 정책으로 위반 배포를 애초에 막기

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[기초] 인증과 인가의 구분

'너는 누구인가' 와 '그것을 해도 되는가' 는 별개의 질문이다. 순서도 정해져 있다.

- **인증(Authentication)** 은 신원을 확인한다. 실패하면 **401**
- **인가(Authorization)** 는 권한을 확인한다. 실패하면 **403**

	인증 (AuthN)	인가 (AuthZ)
묻는 것	너는 누구인가	그것을 할 수 있는가
실패 코드	401 Unauthorized	403 Forbidden
쿠버네티스 담당	IAM · OIDC · 인증서 · SA 토큰	RBAC (Role · Binding)
흔한 원인	kubeconfig 만료 · IAM 매핑 누락	Role 에 verb 나 리소스가 빠짐
먼저 일어나는 것	이쪽이 먼저	인증을 통과해야 여기까지 온다

[체크] 에러 코드만 봐도 어디를 수정해야 하는지 갈린다. **401** 은 신원, **403** 은 권한이다.

클러스터 접근 주체

쿠버네티스는 사용자 계정을 만들지 않는다. 외부 신원을 신뢰할 뿐이다.

- 사람은 클러스터 밖의 신원(IAM·OIDC·인증서)으로 인증한다
- 파드는 클러스터 안의 ServiceAccount 로 인증한다. 두 경로는 완전히 다르다

주체	인증 방법	EKS 에서	확인 명령
사람 (개발자)	IAM → 토큰 교환	<code>aws eks get-token</code> · Access Entry	<code>aws sts get-caller-identity</code>
CI 파이프라인	IAM 역할 가정	OIDC 페더레이션 (GitHub Actions 등)	동일
파드	ServiceAccount 토큰	프로젝티드 토큰 자동 마운트	<code>kubectl auth can-i --as=system:serviceaccount:...</code>
파드 → AWS	IRSA / Pod Identity	SA 에 IAM 역할 연결	<code>aws sts get-caller-identity</code> (파드 안에서)
노드 (kubelet)	노드 인증서	자동	확인할 것 없음

[정리] `kubectl get users` 같은 명령이 없는 이유이다. 쿠버네티스에 사용자 객체는 존재하지 않는다.

RBAC 의 네 가지 리소스

권한은 동사(verb) x 리소스로 정하고, 바인딩으로 주체에 연결한다. 이 두 단계가 전부이다.

- Role/ClusterRole = 권한 묶음, RoleBinding/ClusterRoleBinding = 주체 연결
- 거부 규칙이 없다
- 기본이 전부 거부이고 허용만 더한다 (이 장 내내 이 원칙이 작동한다)

	네임스페이스 범위	클러스터 범위
권한 정의	Role	ClusterRole
주체 연결	RoleBinding	ClusterRoleBinding
조합 ①	Role + RoleBinding	그 네임스페이스에서만
조합 ②	ClusterRole + RoleBinding	그 네임스페이스에서만 (재사용에 유용)
조합 ③	ClusterRole + ClusterRoleBinding	모든 네임스페이스
대상 리소스	pods · deployments · secrets ...	nodes · pv · CRD · namespaces

[체크] 조합 ②를 기억한다. 표준 ClusterRole(**view·edit**)을 만들어 두고 네임스페이스마다 **RoleBinding** 으로 연결하는 것이 실무 표준이다.

최소 권한 Role 작성법

* 로 시작하지 않는다. 필요한 동사만 적고, 부족하면 그때 더한다.

- `kubectl auth can-i` 로 의도한 것과 실체가 같은지 반드시 확인한다
- 서브리소스(`pods/log`, `pods/exec`)를 빠뜨리지 않는다. 실질적인 권한이 여기 있다

```
kind: Role
metadata: {name: shop-developer, namespace: <내 NS>}
rules:
- apiGroups: [""]
  resources: [pods, services, configmaps]
  verbs: [get, list, watch]
- apiGroups: [""]
  resources: [pods/log]                # 로그 조회는 별도 서브리소스다
  verbs: [get]
- apiGroups: ["apps"]
  resources: [deployments]
  verbs: [get, list, watch, update, patch] # create/delete 는 CI 만
# secrets 는 의도적으로 넣지 않는다. pods/exec 도 마찬가지.
```

[주의] `pods/exec` 를 주면 컨테이너 안에서 무엇이든 할 수 있다. 마운트된 Secret 도 전부 읽힌다.

ServiceAccount 와 파드 권한

파드는 반드시 ServiceAccount 를 갖는다. 명시하지 않으면 `default` 가 붙는다.

- 1.24+ 는 SA 토큰을 자동 생성하지 않는다
- 프로젝티드 토큰으로 파드에 주입한다
- API 를 사용하지 않는 파드는 `automountServiceAccountToken: false` 로 막는다

```
kind: ServiceAccount
metadata: {name: shop-api, namespace: <내 NS>}
automountServiceAccountToken: false      # 이 SA 를 쓰는 파드에 기본으로 토큰을 안 준다
---
spec:
  serviceAccountName: shop-api           # 명시하지 않으면 default SA 가 붙는다
  automountServiceAccountToken: false    # 파드 수준에서도 끌 수 있다
---
# 파드가 어떤 권한을 갖는지 확인
#   kubectl auth can-i --list \
#     --token=$(kubectl create token shop-api -n $NS)
```

[주의] `default` SA 를 그대로 사용하면 네임스페이스마다 권한이 뒤섞인다. 워크로드마다 전용 SA 를 만든다.

IRSA 와 IAM 역할 연결

파드에 액세스 키를 넣지 않는다. **ServiceAccount** 에 **IAM 역할**을 연결한다.

- SA 토큰(OIDC JWT)을 AWS STS 가 검증하고 **임시 자격증명**을 발급한다
- 액세스 키가 없으므로 **유출도 회전도 필요 없다**. 15분~1시간짜리 임시 키만 존재한다

```
# ① 클러스터의 OIDC Provider 등록 (클러스터당 한 번)
eksctl utils associate-iam-oidc-provider --cluster skala-k8s-adv --approve

# ② SA + IAM 역할을 한 번에 만든다 (신뢰 정책까지 자동 구성)
eksctl create iamserviceaccount \
  --cluster skala-k8s-adv --namespace $NS --name shop-api \
  --attach-policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess \
  --approve

# ③ SA 에 어노테이션이 붙는다 - 이것이 IRSA 의 실체이다
kubectl get sa shop-api -n $NS -o jsonpath='{.metadata.annotations}'
# {"eks.amazonaws.com/role-arn":"arn:aws:iam::123:role/shop-api-role"}

# ④ 파드 안에서 확인 - IAM 역할로 나온다
kubectl exec deploy/shop-api -n $NS -- aws sts get-caller-identity
```

[체크] SA 어노테이션을 붙인 뒤에는 **파드를 재시작**해야 한다. 토큰 볼륨은 파드 생성 시에 주입되기 때문이다.

SecurityContext 의 컨테이너 제약

컨테이너는 기본적으로 너무 많은 것을 할 수 있다. 필요 없는 것을 하나씩 끈다.

- 가장 중요한 두 가지: **root** 로 돌지 않기, 파일시스템을 읽기 전용으로
- `privileged: true` 는 사실상 **노드 루트 권한**이다. 정당한 이유가 거의 없다

```
spec:
  securityContext:                                # 파드 수준 - 모든 컨테이너에 적용
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001                                # 마운트된 볼륨의 그룹 소유자
    seccompProfile: {type: RuntimeDefault}        # 위험한 시스템콜 차단
  containers:
    - name: shop-api
      securityContext:                             # 컨테이너 수준 - 파드 설정을 덮어쓴다
        allowPrivilegeEscalation: false           # setuid 로 권한 상승 차단
        readOnlyRootFilesystem: true             # 쓰기가 필요하면 emptyDir 를 마운트
        capabilities: {drop: ["ALL"]}            # 리눅스 capability 전부 제거
      volumeMounts:
        - {name: tmp, mountPath: /tmp}          # readOnly 라 /tmp 는 따로 준다
```

[체크] `readOnlyRootFilesystem: true` 로 바꾸면 대개 `/tmp` 와 캐시 경로에서 실패한다. **emptyDir** 를 마운트해 해결한다.

Pod Security Standards

컨테이너 보안 설정을 세 단계 표준으로 묶고, 네임스페이스 라벨 한 줄로 강제한다.

- PSP(PodSecurityPolicy)는 **1.25 에서 제거**됐다. 후속이 PSS 이다
- **enforce**(차단) · **audit**(감사 로그) · **warn**(경고) 세 모드를 **함께 적용**한다

수준	무엇을 막나	사용하는 곳
privileged	아무것도 안 막음	시스템 애드온 전용 네임스페이스
baseline	privileged · hostPath · hostNetwork · hostPID · 위험한 capability	최소 기준 . 모든 네임스페이스에
restricted	baseline + runAsNonRoot 강제 + capability drop ALL + seccomp	애플리케이션 네임스페이스 목표치

라벨 세 줄이면 적용된다. 단계적 도입: 먼저 warn/audit → 관찰 → enforce

```
kubectl label ns shop \
  pod-security.kubernetes.io/enforce=baseline \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted
```

[체크] 곧바로 **enforce=restricted** 를 적용하면 **기존 워크로드가 배포되지 않는다**. **warn** 으로 먼저 얼마나 걸리는지 본다.

Admission 과 조직 고유 규칙

PSS 로 부족한 조직 고유 규칙은 Admission 정책으로 만든다. 앞 표의 4번 단계이다.

- 예: 'latest 태그 금지', 'resources 필수', '허용된 레지스트리만'
- ValidatingAdmissionPolicy** 는 1.30+ 내장이다. 웹훅 서버 없이 CEL 로 작성한다

방식	설치	표현력	언제
ValidatingAdmissionPolicy	내장(1.30+)	CEL 식. 검증만	단순 규칙. 의존성을 늘리기 싫을 때
Kyverno	별도 설치	YAML 정책. 검증·변형·생성	쿠버네티스 팀에게 가장 쉬움
OPA Gatekeeper	별도 설치	Rego 언어. 매우 강력	복잡한 정책 · 이미 OPA 를 사용할 때
직접 만든 웹훅	직접 개발·운영	무엇이든	위 셋으로 안 될 때만

```
kind: ClusterPolicy                # Kyverno. Enforce 전에 Audit 으로 먼저 관찰한다
spec:
  rules:
  - name: no-latest-tag
    match: {any: [{resources: {kinds: [Pod]}]}}
    validate:
      message: "이미지 태그에 latest 를 쓸 수 없습니다"
      pattern: {spec: {containers: [{image: "!:latest"}]}}
```

[주의] 웹훅 기반 정책 엔진이 중단되면 **failurePolicy** 설정에 따라 모든 배포가 막힐 수 있다. 가용성을 함께 설계한다.

이미지 보안

클러스터 보안은 **이미지에서 시작한다**. 취약한 이미지는 어떤 정책으로도 막을 수 없다.

- 베이스 이미지를 줄이면 취약점이 줄어든다
- distroless · alpine 은 CVE 수가 한 자릿수
- 태그가 아니라 다이제스트로 고정하면 같은 이미지가 보장된다

대책	무엇을	도구 / 방법
베이스 최소화	셸·패키지 매니저 제거	distroless · alpine · scratch
빌드 시 스캔	CI 에서 CVE 검사·차단	Trivy · Gype · ECR 스캔
레지스트리 제한	허용된 레지스트리만	Admission 정책 (앞 슬라이드)
다이제스트 고정	<code>image@sha256:...</code>	태그는 바뀔 수 있다
서명 검증	우리가 빌드한 것만 실행	Cosign + Kyverno 검증 정책
non-root 빌드	Dockerfile 에 <code>USER</code>	PSS restricted 의 전제조건
SBOM 생성	무엇이 들어 있는지 기록	Syft · 취약점 공시 시 영향 파악

[체크] '우리 이미지에 셸이 들어 있는가?' 있으면 침해 시 공격자가 사용할 도구가 하나 더 있는 것이다.

감사 로그와 침해 탐지

사고가 났을 때 누가 무엇을 했는지 알 수 있어야 한다. 그러려면 미리 켜 뒀어야 한다.

- EKS 감사 로그는 **기본 꺼짐** 상태이다. 클러스터 생성 시 또는 지금 바로 켜다
- 런타임 탐지(Falco 등)는 **감사 로그가 못 보는 컨테이너 내부 행동**을 본다

① EKS 컨트롤 플레인 로그 활성화 (audit 이 핵심)

```
aws eks update-cluster-config --name skala-k8s-adv \
  --logging '{"clusterLogging":[{"types":["audit","authenticator"],
    "enabled":true}]}'
```

② CloudWatch Logs Insights 로 조회 - 누가 Secret 을 읽었는가

```
# fields @timestamp, user.username, verb, objectRef.resource, objectRef.name
# | filter objectRef.resource = "secrets" and verb in ["get","list"]
# | sort @timestamp desc
```

③ 클러스터 안에서 이상 행동 탐지 (런타임)

Falco: 컨테이너에서 쉘 실행 · 민감 파일 접근 · 아웃바운드 연결 감지

[주의] 감사 로그는 **컨 시점부터** 기록된다. 사고 후에 켜면 그 이전은 영원히 알 수 없다.

보안 점검 체크리스트

순위	항목	확인 방법	노력
1	감사 로그 활성화	<code>aws eks describe-cluster logging</code>	낮음
2	<code>cluster-admin</code> 바인딩 최소화	<code>kubectl get clusterrolebinding</code>	낮음
3	네임스페이스 PSS <code>baseline</code> 적용	네임스페이스 라벨 확인	낮음
4	<code>automountServiceAccountToken: false</code>	SA·파드 스펙 확인	낮음
5	Secret 접근 RBAC 점검	<code>auth can-i --list</code>	중간
6	IRSA 로 액세스 키 제거	Secret 안의 AWS 키 검색	중간
7	PSS <code>restricted</code> 로 상향	warn → 수정 → enforce	높음
8	이미지 스캔 · 서명 검증	CI 파이프라인 개조	높음

[체크] 지금 2번을 확인한다. `kubectl get clusterrolebinding -o wide | grep cluster-admin` 에 몇 명이 나오는가?

컨테이너 탈출 경로

공격자가 파드에서 노드로 넘어가는 길은 정해져 있다. 그 길을 하나씩 막는다.

- 탈출 경로 대부분은 **매니페스트 설정 하나로 열린다**. 정책으로 막을 수 있는 이유이다
- PSS `restricted` 를 적용하면 아래 대부분이 **자동으로 차단된다**

열리는 설정	무엇이 가능해지나	차단 방법
<code>privileged: true</code>	사실상 노드 root	PSS baseline 이 차단
<code>hostPID: true</code>	노드 프로세스 조회·시그널	PSS baseline 이 차단
<code>hostNetwork: true</code>	노드 네트워크 스택 직접 사용	PSS baseline 이 차단
<code>hostPath: /</code>	노드 파일시스템 읽기·쓰기	PSS baseline 이 제한
docker/containerd 소켓	임의 컨테이너 실행 = 노드 장악	마운트 금지. Kaniko 사용
<code>CAP_SYS_ADMIN</code>	마운트·네임스페이스 조작	<code>capabilities.drop: [ALL]</code>
root 로 실행	취약점 악용 성공률 상승	<code>runAsNonRoot: true</code>

[체크] `kubectl get pods -A -o json | grep -c privileged` 를 실행한다. 운영 클러스터에서 이 값이 0 이 아니면 각각의 이유를 확인한다.

공급망 보안

우리 코드보다 의존성과 베이스 이미지에 취약점이 훨씬 많다.

- 이미지 하나에 들어가는 것. 우리 코드 · 라이브러리 수백 개 · OS 패키지 수백 개
- 서명 검증까지 가야 '우리가 빌드한 것만 실행된다' 가 보장된다

단계	무엇을	도구	차단 지점
의존성 스캔	라이브러리 CVE	Dependabot · Snyk	PR
이미지 스캔	OS 패키지 CVE	Trivy · ECR 스캔	CI · 레지스트리
SBOM 생성	구성 목록 기록	Syft	빌드
서명	빌드 출처 증명	Cosign	빌드
서명 검증	서명된 것만 실행	Kyverno 검증 정책	Admission
출처 증명	어떤 파이프라인이 빌드했나	SLSA provenance	빌드·검증

[정리] 스캔만 하고 차단하지 않으면 목록만 쌓인다. CI 에서 심각도 High 이상은 빌드를 실패시킨다.

멀티테넌시 격리

한 클러스터를 여러 팀이 사용한다면, 네임스페이스만으로는 부족하다.

- 네임스페이스는 논리적 경계일 뿐이라 커널과 노드는 그대로 공유된다
- 격리 수준을 높이려면 노드 분리 → 클러스터 분리 순으로 간다

격리 수준	방법	막을 수 있는 것	비용
① 논리	네임스페이스 + RBAC + 쿼터	권한 실수 · 자원 독점	없음
② 네트워크	NetworkPolicy	옆 팀 서비스 접근	낮음
③ 정책	PSS + Admission	위험한 파드 설정	낮음
④ 노드	테인트·어피니티로 노드 분리	노드 자원 경쟁 · 노드 탈출	노드 추가
⑤ 런타임	gVisor · Firecracker(Fargate)	커널 취약점 악용	성능·호환성
⑥ 클러스터	팀별 클러스터	거의 전부	운영 부담 N배

[체크] 신뢰 경계가 어디인가? 같은 회사 팀 간이면 ①~③ 으로 충분하다. 외부 사용자 코드를 실행한다면 ⑥ 이상이 필요하다.

보안 사고 대응 절차

침해가 의심되면 **순서가 정해져 있다**. 급하다고 파드부터 지우면 증거가 사라진다.

- **격리 먼저, 삭제는 나중이다**. 지운 파드에서는 아무것도 파악할 수 없다
- 감사 로그와 컨테이너 상태가 **유일한 증거**이다

순서	무엇을	명령.방법
① 격리	네트워크를 끊는다 (파드는 지우지 않는다)	deny-all NetworkPolicy 를 그 파드에만
② 스케줄 제외	그 노드에 새 파드가 안 가게	<code>kubect1 cordon <node></code>
③ 증거 보존	파드 스펙·로그·프로세스 목록	<code>get -o yaml ·logs ·exec ps</code>
④ 범위 확인	같은 이미지·같은 SA 를 사용하는 것	<code>get pods -A -o wide</code> ·라벨 조회
⑤ 감사 로그	그 SA 가 무엇을 호출했나	CloudWatch Insights
⑥ 자격증명 회전	SA 토큰·IAM·DB 시크릿번호	회전 절차
⑦ 제거·복구	파드 삭제 · 이미지 교체	GitOps 로 알려진 상태 복원
⑧ 회고	어떻게 들어왔나 · 무엇을 막을까	정책 추가

[주의] ①에서 파드를 **삭제하지 않는다**. 컨트롤러가 즉시 새 파드를 만들어 같은 취약점으로 다시 감염될 뿐이다.

보안 진단 지도

'권한이 없다'와 '정책에 막혔다'를 구분하는 표. 에러 메시지가 답을 준다.

- 인가(403)와 Admission(403)은 에러 메시지가 다르다 (6단계 표와 함께)

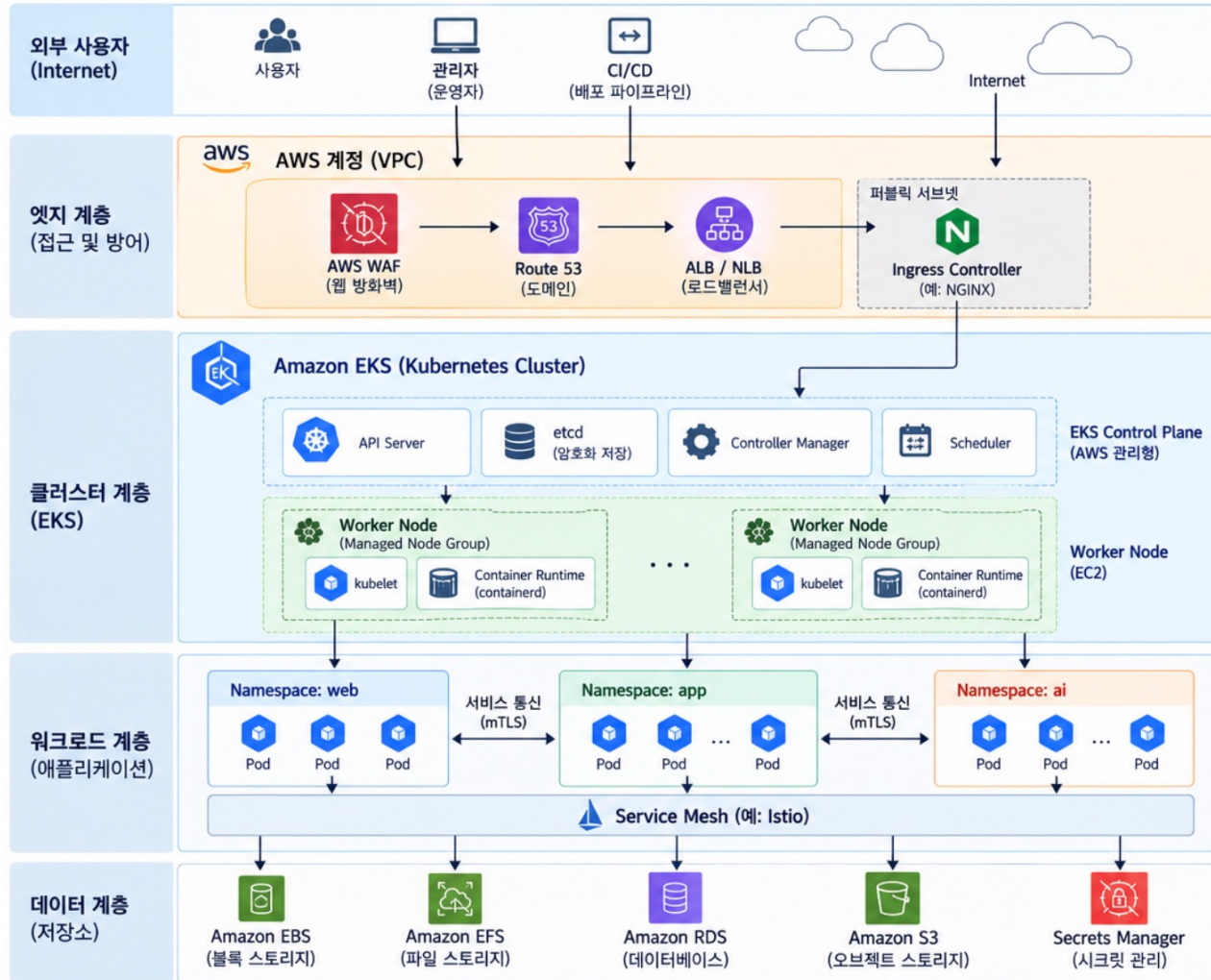
증상 / 메시지	어느 단계	확인
Unauthorized (401)	인증	kubeconfig 만료 · IAM 매핑
forbidden: User ... cannot ...	인가(RBAC)	auth can-i · Role/Binding 확인
violates PodSecurity	Admission (PSS)	네임스페이스 라벨 · SecurityContext
admission webhook denied	Admission (정책 엔진)	Kyverno/Gatekeeper 정책 확인
failed calling webhook	Admission (엔진 장애)	정책 엔진 파드 상태 · failurePolicy
파드 안 AWS 호출 실패	IRSA	SA 어노테이션 · 파드 재시작 · 신뢰 정책
CreateContainerError (root)	런타임	runAsNonRoot 인데 이미지가 root
볼륨 쓰기 실패	런타임	readOnlyRootFilesystem · fsGroup 불일치

[체크] 5번 줄이 나오면 클러스터 전체 배포가 멈춘 상태일 수 있다. 정책 엔진 가용성을 최우선으로 확인한다.

보안 아키텍처

EKS 보안 아키텍처

네트워크, 접근제어, 워크로드, 데이터, 모니터링까지 고려한 전체 보안 구성



주요 보안 구성 요소

1 네트워크 보안

- VPC, Public/Private Subnet 분리
- 보안그룹 (Security Group)
- 네트워크 ACL (NACL)
- Private Endpoint (ECR, S3 등)
- VPC Flow Logs

2 접근제어 (IAM)

- IAM 사용자/역할 최소 권한 원칙
- IRSA (Pod → AWS 서비스 접근)
- OIDC Provider 연동
- MFA, 권한 경계, 정책 조건

3 클러스터 보안

- EKS Control Plane (AWS 관리형, 암호화)
- API Server 인증/인가 (RBAC)
- 감사 로그 (CloudTrail)
- EKS 버전 및 패치 관리

4 워크로드 보안

- Pod Security (PSA) 적용
- NetworkPolicy (Pod 간 트래픽 제어)
- 컨테이너 이미지 스캔 (ECR)
- 일기 전용 루트 파일 시스템, 권한 최소화
- Taint/Toleration, Namespace 격리

5 통신 보안

- TLS (Ingress, Ingress Controller)
- mTLS (Service Mesh)
- 내부 서비스 간 암호화 통신 (Istio)
- Cert-manager (자동 인증서 발급)

6 데이터 보안

- EBS/EFS/S3 암호화 (KMS)
- RDS 암호화, 백업
- 시크릿 관리 (Secrets Manager)
- 기밀 데이터 암호화 및 접근 제한

7 모니터링 및 탐지

- CloudWatch, Container Insights
- Prometheus / Grafana
- 보안 이벤트 분석 (GuardDuty)
- 로그 수집/분석 (CloudTrail, VPC Flow Logs)

8 이미지 및 공급망 보안

- ECR 이미지 스캔 (취약점)
- 서명된 이미지 사용 (Sigstore 등)
- CI/CD 파이프라인 보안 (권한, 시크릿 관리)

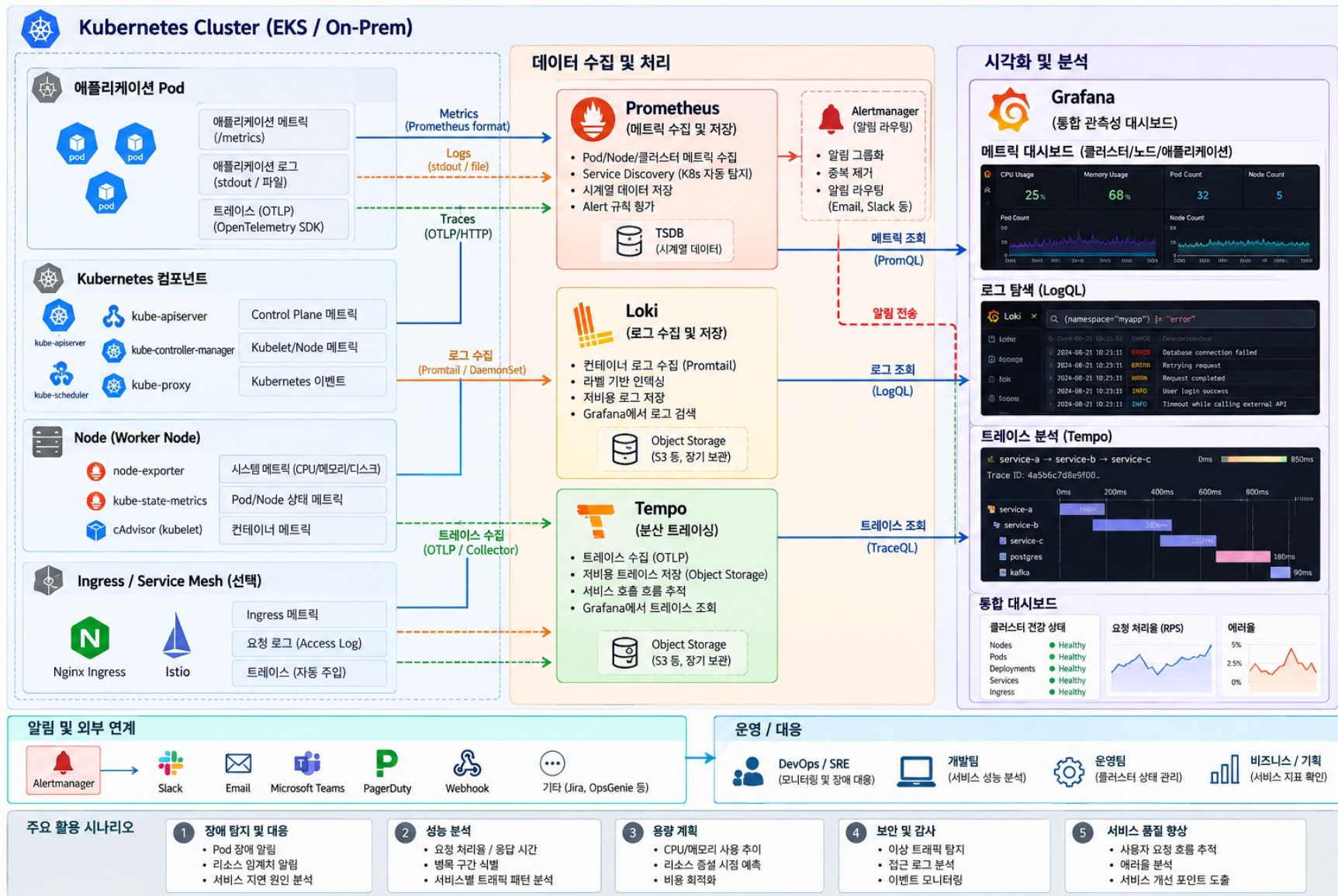
[핵심 요약] 7장 보안

항목	한 줄 정리	실무 포인트
인증	쿠버네티스는 사용자를 만들지 않는다	EKS 는 IAM 을 신뢰. 파드는 SA 로
RBAC	권한 정의 + 바인딩 두 단계	ClusterRole + RoleBinding 조합을 재사용
서브리소스	<code>pods/log</code> · <code>pods/exec</code> 는 별도 권한	exec 를 주면 Secret 도 읽힌다
ServiceAccount	안 적으면 default 가 붙는다	워크로드마다 전용 SA · 토큰 자동마운트 끄기
IRSA	SA 에 IAM 역할을 연결	액세스 키 불필요. 적용 후 파드 재시작
SecurityContext	컨테이너가 할 수 있는 일을 좁힌다	non-root + readOnlyRootFS + drop ALL
PSS	라벨 한 줄로 세 단계 표준 강제	warn 으로 먼저 관찰 → enforce
Admission 정책	조직 규칙을 배포 전에 차단	엔진 가용성과 failurePolicy 를 함께 설계

[체크] '우리 파드가 침해되면 공격자가 클러스터에서 무엇을 할 수 있는가?' 이 질문에 구체적으로 답할 수 있으면 통과.

8. 관측성: 로그 · 메트릭 · 추적

- Probe 세 가지. 잘못 설정하면 오히려 장애를 만든다
- 메트릭. Prometheus 로 무엇을 언제 보는가
- 로그 파이프라인. 컨테이너 로그가 어디로 가는가
- 분산 추적과 알림. 증상에서 원인까지



[기초] 메트릭 · 로그 · 추적의 차이

셋은 답하는 질문이 다르다. 무엇을 물을지 정해야 무엇을 볼지 정해진다.

- **메트릭**은 숫자의 시계열이고 '지금 정상인가'에 답한다
- 로그는 **사건의 기록**이고, 추적은 **요청 하나가 지나간 경로**이다

	메트릭	로그	추적
형태	시간에 따른 숫자	시각 + 문장	요청 하나의 구간들
답하는 질문	지금 정상인가	그때 무슨 일이 있었나	어디서 느려졌나
보관 비용	낮다 (숫자만)	가장 높다	중간 (표본만 남긴다)
대표 도구	Prometheus	CloudWatch · Loki	OpenTelemetry · Tempo
먼저 보는 순서	① 이상 감지	③ 원인 문장 확인	② 구간 좁히기

[정리] 장애 때는 **메트릭으로 좁히고 추적으로 구간을 찾고 로그로 확인한다**. 로그부터 뒤지면 오래 걸린다.

Probe 세 가지의 역할

세 Probe 는 실패했을 때의 조치가 다르다. 그것이 구分的 전부이다.

- **liveness** 실패 → 컨테이너를 재시작한다 / **readiness** 실패 → 트래픽만 끊는다
- startup 은 기동이 느린 앱을 위해 **liveness** 를 늦추는 용도이다

Probe	질문	실패 시 조치	없으면
liveness	살아 있는가?	컨테이너 재시작	행(hang) 상태로 영원히 남는다
readiness	요청을 받을 준비가 됐는가?	엔드포인트에서 제거	준비 안 된 파드로 트래픽 → 5xx
startup	기동이 끝났는가?	성공까지 다른 Probe 를 보류	느린 앱이 기동 중 재시작 루프

```

startupProbe:                                # 최대 5분(30×10초) 기다린다
  httpGet: {path: /health/live, port: 8080}
  failureThreshold: 30
livenessProbe:                                # startup 성공 후부터 동작
  httpGet: {path: /health/live, port: 8080}
  failureThreshold: 3
readinessProbe:                               # 준비되면 엔드포인트에 들어간다
  httpGet: {path: /health/ready, port: 8080}
  
```

[주의] **liveness** 와 **readiness** 의 엔드포인트를 같게 사용하지 않는다. DB 장애 시 전 파드가 동시에 재시작되어 상황이 악화된다.

Probe 설정의 함정

Probe 는 너무 민감해도, 너무 둔해도 문제가 된다. 숫자에 근거가 있어야 한다.

- 감지 시간 = `periodSeconds` × `failureThreshold`
- liveness 에서는 재시작까지의 시간이 된다
- 타임아웃 기본값은 1초이다. 느린 앱에서 오탐의 주범이 된다

증상	원인	조정할 값
부하가 몰릴 때 파드가 재시작됨	liveness 타임아웃(1초) 초과	<code>timeoutSeconds</code> : 3~5 · liveness 를 가볍게
기동 중 CrashLoopBackOff	liveness 가 기동 전에 시작	<code>startupProbe</code> 추가
DB 장애 시 전 파드 재시작	liveness 가 의존성을 검사	liveness 는 self-check 만
배포 중 5xx	readiness 없음	readinessProbe 추가
장애 감지가 너무 느림	<code>period·threshold</code> 가 큼	<code>10s × 3 = 30초</code> 를 기준으로 조정
Probe 가 앱 성능을 갉아먹음	무거운 헬스체크(쿼리 실행)	캐시된 상태만 반환하게
exec Probe 가 좀비 프로세스 생성	셸 스크립트 Probe	httpGet 으로 바꾼다

[체크] 헬스 엔드포인트가 **DB 쿼리**를 실행하고 있지 않은가? Probe 주기 × 파드 수만큼 DB 에 부하가 간다.

네 가지 황금 신호

지표는 많을수록 좋은 것이 아니다.

- 지연 · 트래픽 · 오류 · 포화도
- 이 넷이면 대부분의 장애가 드러난다
- 사용자가 느끼는 것부터 본다. 노드 CPU 는 그다음이다

신호	무엇	쿠버네티스에서 보는 것	알림 기준 예
지연 (Latency)	요청 처리 시간	앱 메트릭 p50/p95/p99	p99 > 1s 5분 지속
트래픽 (Traffic)	초당 요청 수	앱 메트릭 · Ingress 지표	평소 대비 ±50%
오류 (Errors)	실패 비율	5xx 비율 · 파드 재시작 수	5xx > 1% 5분 지속
포화도 (Saturation)	자원 여유	CPU 스로틀 · 메모리 사용률 · 큐 길이	throttle > 5%
(+) 쿠버네티스 상태	선언과 현실의 차이	<code>kube_deployment_status_replicas</code> 차이	5분 이상 불일치

[정리] 알림은 사용자 영향이 있는 것에만 설정한다. '노드 CPU 80%' 는 대시보드로만 보고 알림으로는 만들지 않는다.

Prometheus 구성

Prometheus 는 **Pull** 방식이다. 파드가 지표를 노출해 두면 주기적으로 수집해 간다.

- kube-prometheus-stack 을 설치하면 **Prometheus·Grafana·Alertmanager·exporter** 가 한 번에
- 수집 대상은 **ServiceMonitor / PodMonitor CRD** 로 선언한다

```
kind: ServiceMonitor
metadata:
  name: shop-api
  namespace: <내 NS>
  labels: {release: kube-prometheus-stack} # Prometheus 가 이 라벨로 찾는다
spec:
  selector: {matchLabels: {app: shop-api}} # 이 Service 를
  endpoints:
    - port: metrics # 이 이름의 포트에서
      path: /actuator/prometheus # 이 경로를
      interval: 30s # 30초마다 수집한다
  ---
# 앱 Service 에 포트 이름이 있어야 한다
# ports: [{name: metrics, port: 8081, targetPort: 8081}]
```

[체크] 수집이 안 되면 **Prometheus UI** 의 **Targets** 화면을 본다. 대상에 없으면 라벨 문제, 있는데 DOWN 이면 네트워크·경로 문제이다.

꼭 알아 둘 PromQL

쿼리 몇 개면 앞에서 미룬 근거 있는 requests 산정이 가능해진다.

- 카운터에는 항상 `rate()` 를 씌운다. 누적값 그대로는 의미가 없다
- `quantile_over_time` 으로 **7일 p95** 를 추출하는 것이 리소스 산정의 근거이다

```
# 미뤄 둔 것 - requests 산정용 7일 p95 CPU
quantile_over_time(0.95,
  rate(container_cpu_usage_seconds_total{namespace="shop",
    container="shop-api"}[5m])[7d:5m])

# 메모리는 최대값 기준 (CPU 와 달리 초과하면 죽는다)
max_over_time(container_memory_working_set_bytes{namespace="shop"}[7d])

# CPU 스로틀링 비율 - 5% 넘으면 limits 를 손본다
rate(container_cpu_cfs_throttled_periods_total[5m])
/ rate(container_cpu_cfs_periods_total[5m])

# 5xx 비율
sum(rate(http_requests_total{status=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m]))
```

[체크] 첫 쿼리 결과가 현재 requests 보다 훨씬 낮으면 과다 예약이다. 앞의 판단 기준($\times 1.2$)을 적용한다.

커스텀 메트릭 기반 HPA

CPU 로는 부족한 워크로드가 있다. **큐 길이·동시 요청 수**로 확장하는 것이 맞을 때가 있다.

- CPU 는 **후행 지표**이다. 큐가 쌓인 뒤에야 CPU 가 오른다
- Prometheus Adapter 가 PromQL 결과를 **쿠버네티스 메트릭 API** 로 노출한다

워크로드	CPU 기반이 부적절한 이유	적절한 지표
큐 소비자(worker)	대기 중일 때 CPU 가 낮다	큐 길이 · 메시지 지연
I/O 대기가 많은 API	CPU 는 낮는데 응답은 느리다	동시 요청 수 · p95 지연
WebSocket 서버	연결 유지 중 CPU 가 거의 0	활성 연결 수
배치 처리	처리량이 CPU 와 비례하지 않음	미처리 항목 수
일반 웹 API	(적절함)	CPU 로 충분

```
metrics:
- type: Pods
  pods:
    metric: {name: http_inflight_requests}    # Prometheus Adapter 가 노출한 이름
    target: {type: AverageValue, averageValue: "30"}    # 파드당 동시 30건 유지
```

[주의] 외부 지표로 확장할 때는 **상한(maxReplicas)**이 더 중요하다. 큐가 폭증하면 파드가 무한히 늘어난다.

컨테이너 로그와 노드 파일

컨테이너의 stdout/stderr 는 **노드 디스크의 파일**이 된다. 그걸 수집해야 남는다.

- 파드가 지워지면 그 노드의 로그 파일도 정리된다. **kubectl logs** 만 믿으면 안 된다
- 로그를 파일로 출력하면 수집이 어려워지므로 **stdout** 으로 내보내는 것이 원칙이다



Fluent Bit 은 DaemonSet 이라 노드마다 하나씩 실행된다. 파드를 지우면 containerd 레인의 파일이 함께 사라지므로, 그 전에 가져갔는지가 사후 분석의 갈림길이다.

[주의] 노드 디스크가 로그로 차면 **DiskPressure** → **파드 축출**이 일어난다. 로테이션 설정을 확인한다.

구조화 로그와 상관관계

로그를 **JSON** 으로 출력하고, **요청 ID** 를 넣는다. 이 둘이면 검색이 전혀 달라진다.

- 텍스트 로그는 파싱이 실패한다. **앱에서 JSON 으로 출력하는 것이 가장 확실하다**
- trace ID 를 로그에 넣으면 **로그와 추적이 이어진다**

이런 로그는 - 사람은 읽지만 기계는 못 찾는다

```
2026-08-05 14:23:01 ERROR 주문 실패 order=1234 user=99
java.lang.NullPointerException: ...
```

이렇게 바꾼다 - 필드로 검색된다

```
{"ts":"2026-08-05T14:23:01Z", "level":"ERROR",
  "msg":"주문 실패", "order_id":1234, "user_id":99,
  "trace_id":"a1b2c3d4", "error":"NullPointerException"}
```

그러면 이런 조회가 가능해진다

```
order_id = 1234
trace_id = "a1b2c3d4"
level=ERROR AND user_id=99
```

```
파싱 없이 필드 검색
여러 서비스의 로그가 한 줄에
조건 조합
```

[체크] 로그에 **trace ID** 가 있는가? 없으면 마이크로서비스에서 한 요청의 흐름을 재구성할 수 없다.

분산 추적

추적은 한 요청이 지나간 경로와 각 구간의 시간을 보여 준다. '어디서 느린가'에 답한다.

- 메트릭은 '느리다' 를 알려 준다
- 추적은 '어느 구간이 느린지' 를 알려 준다
- OpenTelemetry 가 표준이다. 계측은 한 번, 백엔드는 교체할 수 있다

	메트릭	로그	추적
답하는 질문	무엇이 잘못됐나	왜 그런가	어디서 그런가
데이터 양	적음 (집계)	많음	중간 (샘플링)
보관 비용	낮음	높음	중간
장애 대응 순서	① 감지	③ 원인 확정	② 범위 좁히기
EKS 조합 예	Prometheus/AMP	CloudWatch Logs	AWS X-Ray / Jaeger

[정리] 전량 수집은 비싸다. 오류 요청과 느린 요청은 100%, **정상**은 1~10% 를 샘플링하는 것이 무난하다.

알림 설계

알림은 사람을 깨우는 것이다. 깨울 만한 것만 알린다.

- 알림 기준은 **증상**으로 작성한다. '노드 CPU 90%' 가 아니라 '주문 API p99 > 2초'
- 모든 알림에는 **무엇을 해야 하는지**가 함께 있어야 한다 (runbook 링크)

등급	기준	채널	예시
Page (즉시 호출)	사용자 영향이 지금 있음	전화 · 온콜	결제 5xx > 5% · 서비스 전면 중단
Ticket (업무시간)	영향이 곧 생김	슬랙 · 지라	디스크 80% · 인증서 30일 남음
Info (기록만)	알아 두면 좋음	대시보드	배포 완료 · HPA 스케일 이벤트

```
- alert: ShopApiHighErrorRate
  expr: |
    sum(rate(http_requests_total{namespace="shop",status=~"5.."}[5m]))
    / sum(rate(http_requests_total{namespace="shop"}[5m])) > 0.05
  for: 5m                                # 5분 지속돼야 발화 - 스파이크 무시
  labels: {severity: page}
  annotations:
    summary: "shop-api 5xx 비율 5% 초과"
    runbook: "https://wiki/runbook/shop-api-5xx"    # 반드시 넣는다
```

[체크] 지난 한 달 알림 중 실제로 조치한 비율은? 30% 미만이면 알림이 너무 많은 것이다.

대시보드 설계

대시보드는 질문에 답하려고 만든다. 지표를 모아 놓는 곳이 아니다.

- 화면 하나에 질문 하나
- '이 서비스는 지금 건강한가?'
- 계층을 나눈다. 서비스 개요 → 워크로드 상세 → 인프라

계층	보는 사람	답하는 질문	핵심 패널
① 서비스 개요	누구나 · 온콜	지금 사용자가 불편한가?	5xx 비율 · p95 지연 · 요청량
② 워크로드	담당 개발자	어느 파드·버전이 문제인가?	파드별 지표 · 재시작 수 · 버전별 비교
③ 의존성	담당 개발자	DB·큐가 병목인가?	커넥션 수 · 큐 길이 · 외부 호출 지연
④ 인프라	플랫폼팀	자원이 부족한가?	노드 자원 · 스로틀 · 축출
⑤ 비용	관리자	어디에 돈이 쓰이나?	네임스페이스별 요청량

[체크] 대시보드를 열고 10초 안에 '정상인가?' 를 판단할 수 있는가? 못 하면 패널이 너무 많은 것이다.

SLO 와 에러 버짓

'항상 100%' 는 불가능하다. **얼마나 실패해도 되는지**를 정하고 그 안에서 움직인다.

- SLO 는 **기술 목표가 아니라 합의**이다. 제품·운영이 함께 정한다
- 에러 버짓이 남아 있으면 **배포 속도를 올리고**, 소진되면 안정화에 집중한다

SLO 예: 주문 API 가용성 99.9% (30일 기준)

허용 실패 시간 $30\text{일} \times 0.1\% = 43\text{분 } 12\text{초}$

허용 실패 요청 전체 요청의 0.1%

에러 버짓 소진율(burn rate)

1배 → 30일 동안 딱 맞게 소진 (정상)

6배 → 5일이면 소진 . 경고(ticket)

14.4배 → 2일이면 소진 . 즉시 호출(page)

알림 규칙 (다중 윈도우 방식)

5분 창에서 14.4배 AND 1시간 창에서 14.4배 → page

6시간 창에서 6배 AND 3일 창에서 6배 → ticket

[정리] 소진율로 알림하면 오탐이 준다. 짧은 스파이크는 긴 창에서 걸러지고, 지속적 열화는 확실히 감지된다.

로그 비용 관리

로그는 가장 빨리 비싸지는 관측 데이터이다. 수집 단계에서 줄인다.

- 비용 = 수집량 × 보관 기간
- 둘 다 조정할 수 있다
- DEBUG 로그를 운영에서 켜 두는 것이 비용 폭증의 1순위 원인이다

방법	구체적으로	절감 효과	위험
로그 레벨 조정	운영은 INFO 이상	50~90%	디버깅 정보 감소
헬스체크 로그 제외	Probe 요청은 남기지 않는다	20~40%	거의 없음
샘플링	동일 반복 로그는 N개 중 1개	가변	드문 사례 누락
필드 정리	불필요한 메타데이터 제거	10~30%	거의 없음
보관 기간 차등	최근 7일 hot · 이후 S3	저장 비용 80%	조회 지연
네임스페이스별 정책	비운영은 짧게	환경 비중만큼	거의 없음

[체크] Probe 요청이 로그의 몇 %인가? 10초 주기 × 파드 수 × 두 종류면 생각보다 크다.

장애 대응 흐름 정리

알림에서 복구까지의 표준 흐름. 이 장의 도구를 순서대로 사용한다.

- 각 단계의 목표 시간을 정해 두면 병목이 보인다

단계	목표	사용하는 것	시간 목표
감지	사람보다 먼저 안다	SLO 기반 알림	1~5분
확인	실제 문제인가	서비스 개요 대시보드	1분
범위	어느 서비스·파드인가	지표를 파드·버전별로 분해	3분
가설	무엇이 원인인가	최근 변경 확인 (Git 이력)	3분
검증	가설이 맞는가	로그 · 추적 (trace ID)	5분
완화	사용자 영향을 먼저 끊는다	롤백 · 스케일 아웃 · 트래픽 차단	5분
복구	원인을 제거	수정 배포	가변
회고	재발을 막는다	알림·정책·문서 갱신	1주 내

[정리] 완화가 복구보다 앞이다. 원인을 몰라도 롤백하면 사용자 영향은 멈춘다. 원인 분석은 그다음이다.

관측성 진단 지도

'안 보인다' 를 좁히는 표. 수집 · 노출 · 조회 세 단계로 나뉜다.

- 대부분 라벨과 경로 문제로 좁혀진다

증상	첫 확인	흔한 원인
Prometheus 에 대상이 안 나옴	Targets 화면	ServiceMonitor 라벨 · 네임스페이스 셀렉터
대상은 있는데 DOWN	Targets 의 Error 열	포트 이름 불일치 · 경로 오타 · NetworkPolicy
지표는 있는데 값이 0	앱의 <code>/metrics</code> 직접 확인	앱이 지표를 안 내보냄
HPA 가 <code>unknown</code>	metrics-server / Adapter	requests 미설정 · Adapter 미설치
<code>kubectl logs</code> 가 비어 있음	앱이 stdout 으로 출력하는가	파일 로깅 · 버퍼링 미플러시
로그가 수집기에 안 들어옴	Fluent Bit 파드 로그	파싱 실패 · 전송 권한(IRSA)
로그가 잘려서 나옴	멀티라인 설정	스택트레이스 줄바꿈 처리
디스크가 로그로 참	<code>df -h</code> · 로테이션 설정	<code>containerLogMaxSize</code> 미설정

[체크] `kubectl exec <pod> -- curl localhost:8081/metrics` 로 앱이 실제로 내보내는지 먼저 확인하면 절반이 걸러진다.

[핵심 요약] 8장 관측성

liveness 와 readiness 는 엔드포인트를 분리한다. 같이 사용하면 장애를 만든다.

항목	한 줄 정리	실무 포인트
Probe 구분	liveness 는 재시작하고 readiness 는 트래픽만 끊는다	같은 엔드포인트를 사용하면 연쇄 재시작
Probe 숫자	$\text{period} \times \text{threshold} = \text{감지 시간}$	타임아웃 기본 1초. 부하 시 오탐의 주범
황금 신호	지연·트래픽·오류·포화도	사용자 영향이 있는 것에만 알림
Prometheus	pull 방식. ServiceMonitor 로 선언	라벨과 포트 이름이 맞지 않으면 조용히 실패한다
PromQL	카운터에는 <code>rate()</code>	7일 p95 가 requests 산정의 근거
커스텀 메트릭	CPU 는 후행 지표이다	큐·동시요청 기반 확장. 상한을 반드시
로그	stdout → 노드 파일 → 수집기	파일 로깅은 파드와 함께 사라진다
추적	어느 구간이 느린지	trace ID 를 로그에 넣어 로그와 잇는다

[체크] '지금 장애가 나면 몇 분 안에 원인 구간을 특정할 수 있는가?' 30분을 넘으면 관측성에 투자할 때이다.

9. GitOps 와 배포 전략

- GitOps 의 네 원칙. push 파이프라인과 무엇이 다른가
- Argo CD 구조와 Sync·Health. 드리프트를 읽는 법
- Argo Rollouts 로 카나리·블루그린을 선언형으로
- 블루그린과 카나리. 무엇을 언제 고를 것인가
- 개요 과정이다. 도구 운영은 별도 DevOps 과정에서 다룬다

[기초] Push 배포와 Pull 배포

누가 클러스터를 바꾸는가. 방향이 바뀌면 권한 구조가 통째로 바뀐다.

- **Push:** CI 가 클러스터에 접속해 적용한다
- CI 가 클러스터 권한을 들고 있어야 한다
- **Pull:** 클러스터 안 에이전트가 Git 을 보고 스스로 맞춘다. 밖에서 들어오지 않는다

	Push (kubectl apply · CI)	Pull (GitOps)
바꾸는 주체	클러스터 밖의 CI	클러스터 안의 에이전트
필요한 권한	CI 가 클러스터 자격증명을 보관	CI 는 클러스터 권한이 없다
침해 시 영향	CI 가 뚫리면 클러스터가 뚫린다	Git 저장소 권한만 지키면 된다
손으로 바꾼 것	아무도 모른다	되돌려 놓는다 (드리프트 교정)
현재 상태의 근거	마지막에 누가 무엇을 했는지 추적	Git 이력이 곧 현재 상태

[정리] GitOps 의 보안 이점은 방향에서 나온다. **밖에서 안으로 들어오는 경로 하나가 사라진다.**

GitOps 네 원칙

핵심은 **방향**이다. CI 가 클러스터로 보내는 **Push** 가 아니라, 클러스터가 Git 에서 가져오는 **Pull** 이다.

- Git 이 유일한 진실의 원천이다
- 클러스터 상태는 Git 의 결과일 뿐이다
- 사람이 `kubectl apply` 를 하지 않게 되는 것이 목표이다 (RBAC 과 이어진다)

원칙	의미	실무에서의 효과
① 선언형	원하는 상태를 코드로 기술	명령 이력이 아니라 상태를 본다
② 버전 관리·불변	Git 에 저장, 이력이 남는다	'언제 누가 왜' 가 커밋에 남는다
③ 자동 적용(pull)	에이전트가 Git 을 보고 맞춘다	클러스터 자격증명을 CI 에 주지 않는다
④ 지속적 조정	차이가 발생하면 계속 되돌린다	손으로 고친 것이 자동 복구된다

[정리] ③이 보안상 가장 큰 이득이다. **CI 시스템이 클러스터 접근 권한을 갖지 않는다.** 침해 경로 하나가 사라진다.

Argo CD 구조

Argo CD 는 **Application** 리소스 하나가 '어느 Git 을 어느 클러스터에 맞출지' 를 정의한다.

- 구성요소는 셋이다. **repo-server**(렌더링) · **application-controller**(비교·적용) · **api-server**(UI/API)
- 평문 YAML 과 템플릿 도구를 모두 지원해서 **저장소 구조를 바꾸지 않아도 된다**

```
kind: Application
metadata: {name: shop-api-prod, namespace: argocd}
spec:
  project: shop
  source:
    repoURL: https://github.com/skala/shop-manifests
    targetRevision: main
    path: manifests/shop-api          # 매니페스트 경로
  destination: {server: https://kubernetes.default.svc, namespace: <내 NS>}
  syncPolicy:
    automated: {prune: true, selfHeal: true}  # 자동 적용 + 자동 복구
    syncOptions: [CreateNamespace=true]
    retry: {limit: 3, backoff: {duration: 30s, factor: 2}}
```

[주의] `prune: true` 는 **Git** 에서 지운 리소스를 클러스터에서도 지운다. 커지 않으면 삭제가 반영되지 않는다.

동기화 상태와 드리프트

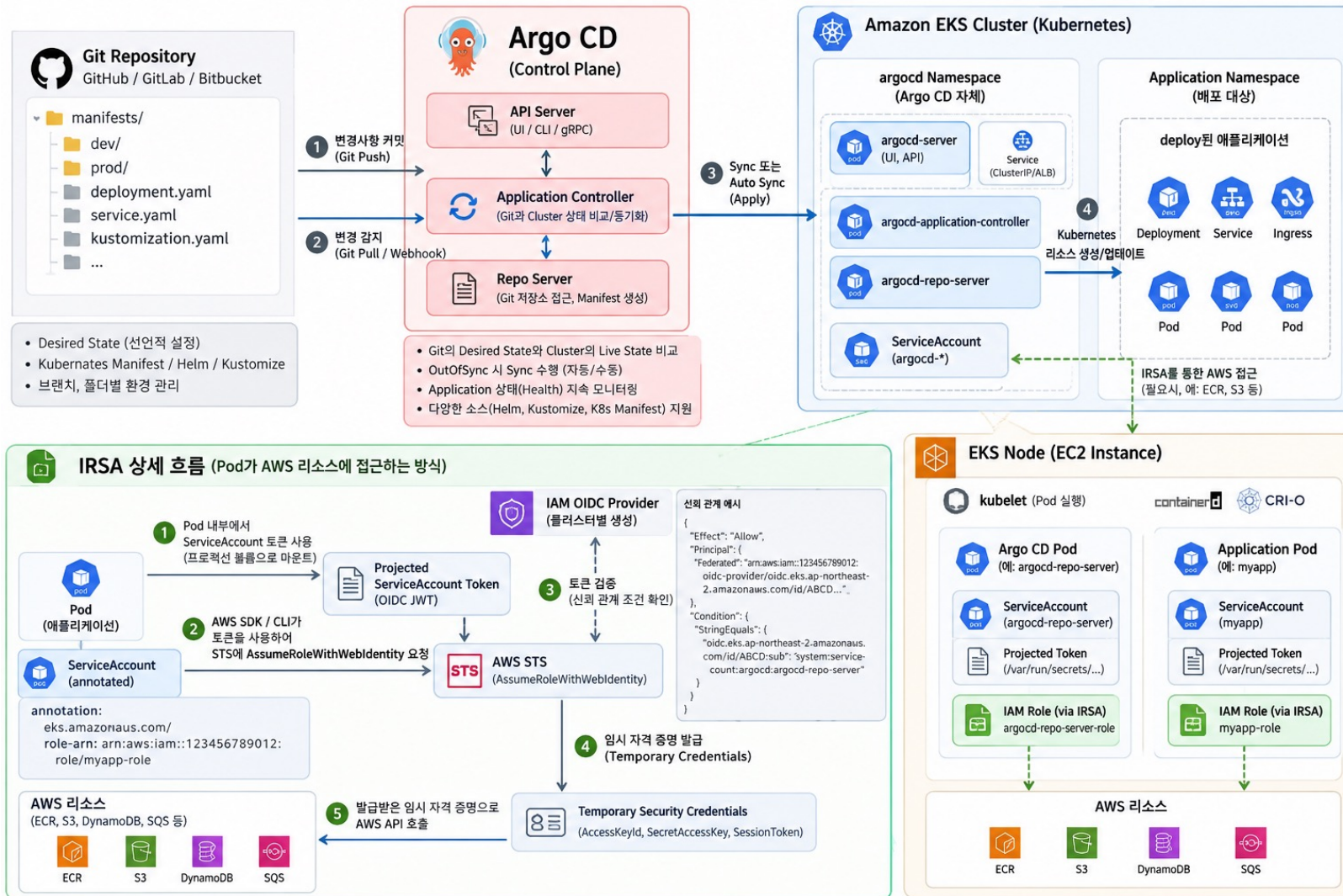
Argo CD 는 두 축으로 상태를 본다. **Sync**(Git 과 같은가)와 **Health**(정상 동작하는가).

- Synced라고 해서 반드시 Healthy한 것은 아니다.
- Drift 역시 장애가 아니라 Git의 Desired State와 Cluster의 Live State 사이에 차이가 발생했음을 알려주는 신호이다.

Sync	Health	무슨 상황인가	대응
Synced	Healthy	정상	할 일 없음
OutOfSync	Healthy	Git 이 앞서거나 누가 손으로 고침	Sync 하거나 원인 확인
Synced	Progressing	배포 진행 중	기다린다
Synced	Degraded	Git 대로 적용됐는데 동작하지 않는다	앱·설정 문제
OutOfSync	Degraded	실패한 배포가 방치됨	Git 을 되돌린다(revert)
Unknown	판정 불가	Git 접근 실패 · 렌더링 실패	repo-server 로그 확인

[체크] Synced + Degraded 가 가장 중요한 조합이다. '배포는 됐는데 동작하지 않는다.' 여기서 앞의 진단 지도를 사용한다.

ArgoCD 중심 아키텍처



Argo Rollouts 의 단계 배포

앞에서 미룬 카나리와 블루그린을 선언형으로 만든다. Deployment 를 Rollout 으로 바꾼다.

- 단계(step)와 대기 시간을 **매니페스트에 적는다**. 사람이 지켜보지 않아도 된다
- 지표를 보고 **자동 중단·롤백**하는 것이 핵심 가치이다 (analysis)

```
kind: Rollout                                # Deployment 대신
spec:
  replicas: 10
  strategy:
    canary:
      canaryService: shop-api-canary
      stableService: shop-api-stable
      steps:
        - setWeight: 10                      # 10% 만 새 버전으로
        - pause: {duration: 5m}              # 5분 관찰
        - analysis:                          # 지표 검사 - 실패하면 자동 중단·롤백
            templates: [{templateName: error-rate-check}]
        - setWeight: 50
        - pause: {duration: 10m}
        - setWeight: 100
```

[정리] analysis 가 없는 카나리는 **사람이 지켜봐야 하는 카나리**이다. 지표가 여기서 값을 한다.

블루그린과 카나리 선택

둘 다 '점진적'이지만 위험을 다루는 방식이 다르다.

- 블루그린: **전부 준비한 뒤 한순간에 전환**한다. 롤백이 즉시, 리소스는 2배
- 카나리: **일부만 노출하며 관찰**한다. 추가 리소스가 적고, 두 버전 공존 시간이 길다

	블루그린	카나리
전환 방식	0% → 100% 한 번에	10% → 50% → 100% 단계적
리소스	배포 중 2배	한 단계 분량만 추가
롤백 속도	즉시 (서비스 셀렉터 전환)	단계 롤백 (상대적으로 느림)
버전 공존	전환 순간만	전체 배포 기간 내내
실사용자 검증	전환 후에야 가능	소수 사용자로 먼저
DB 스키마 요구	앞뒤 호환 (전환 순간)	앞뒤 호환 (긴 시간)
적합한 경우	스키마 변경 · 큰 변경	일상적 배포 · 성능 영향 확인

[주의] 두 방식 모두 **DB 앞뒤 호환이 전제**이다. 그것 없이는 어떤 전략도 무중단이 아니다.

[핵심 요약] 9장 GitOps

항목	한 줄 정리	실무 포인트
방향의 전환	push 가 아니라 pull	CI 에 클러스터 자격증명이 필요 없어진다
앞 내용과의 연결	조정 루프가 클러스터 밖까지 확장된 것	Git=원하는 상태, 클러스터=현재 상태
Application	Git 경로 + 대상 + 정책	<code>prune</code> 은 나중에, <code>selfHeal</code> 은 그다음
Sync / Health	독립된 두 축	Synced + Degraded 는 앱 문제 (앞의 지도로)
Argo Rollouts	카나리·블루그린을 선언형으로	<code>analysis</code> 가 없으면 반쪽이다 (지표)
전략 선택	블루그린은 짧은 공존, 카나리는 실사용 검증	DB 스키마 호환 기간으로 판단
여기서 다루지 않은 것	저장소 구조 · 권한 모델 · ApplicationSet	별도 DevOps 과정 · 실습 파일은 lab/ch10/ 에

10. 운영 · 트러블슈팅 · 비용

- 클러스터·노드 업그레이드의 순서와 사전 점검
- 백업과 재해 복구. 무엇을 어디에 남기는가
- 비용 최적화. 어디서 사고 어떻게 막는가

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[기초] 버전 번호를 읽는 법

1.31.4 의 세 자리는 성격이 다르다. 업그레이드 규칙이 여기서 나온다.

- 마이너는 한 단계씩만 올린다
- 두 단계를 건너뛰지 못한다

자리	예	의미	올릴 때
메이저	1.31.4	호환을 깨는 큰 변경	사실상 고정되어 있다
마이너	1.31.4	기능 추가 · API 제거	한 번에 한 단계. 사전 점검 필수
패치	1.31.4	버그·보안 수정	바로 올려도 된다
지원 기간	마이너 하나당 약 14개월	그 뒤에는 확장 지원 요금	끝나면 AWS 가 자동으로 올린다
내 버전 확인	<code>kubectl version</code>	클라이언트와 서버가 따로 찍힌다	둘의 마이너 차이는 ±1 이내
노드 버전 확인	<code>kubectl get nodes</code>	VERSION 열이 kubelet 버전	API 서버보다 높을 수 없다

[주의] 마이너를 두 단계 건너뛰려면 중간 단계를 반드시 거쳐야 한다. 1.29 → 1.31 은 두 번에 나눈다.

업그레이드의 순서

컨트롤 플레인 **먼저**, 노드가 **나중**이다. 반대로 하면 안 된다.

- kubelet 은 API 서버보다 **최대 3 마이너 낮을 수 있지만 높을 수는 없다**
- EKS 는 **마이너 한 단계씩만** 올릴 수 있다. 1.29 → 1.31 은 두 번에 나눠서 한다

순서	대상	무엇을	주의
①	사전 점검	제거된 API 사용 여부 확인	kubent.pluto 로 매니페스트 검사
②	애드온 호환 확인	VPC CNI · CoreDNS · CSI · LB Controller	애드온이 새 버전을 지원하는가
③	컨트롤 플레인	EKS 콘솔/CLI 로 업그레이드	약 20~40분. 되돌릴 수 없다
④	애드온	EKS Add-on 버전 상황	컨트롤 플레인 다음
⑤	노드그룹	관리형 노드그룹 롤링 교체	PDB 가 없으면 동시 종료
⑥	검증	워크로드 상태·지표 확인	지표로 회귀 확인

[주의] EKS 컨트롤 플레인 업그레이드는 롤백이 불가능하다. ①②를 건너뛰면 되돌릴 방법이 없다.

노드 교체와 드레인

노드를 내리는 절차는 차단 → 비우기 → 삭제이다. 각 단계에서 확인할 것이 있다.

- `cordons` 은 새 파드만 막고, `drain` 이 실제로 비운다
- **PDB** 를 어기는 드레인은 대기한다. 영원히 멈출 수도 있다

① 새 파드 배치 차단 (기존 파드는 그대로)

```
kubectl cordon ip-10-0-1-23
```

② 비우기 - `DaemonSet` 은 대상이 아니므로 무시하도록 지정

```
kubectl drain ip-10-0-1-23 \
  --ignore-daemonsets --delete-emptydir-data --timeout=600s
```

③ 진행이 안 되면 무엇이 막는지 본다

```
kubectl get pdb -A # minAvailable 을 못 맞추는 PDB 가 있는가
kubectl get pods -o wide --field-selector spec.nodeName=ip-10-0-1-23
```

④ 노드 제거 후 복귀시키려면

```
kubectl uncordon ip-10-0-1-23
```

[체크] `drain` 이 멈추면 대개 **PDB** 를 만족할 수 없는 상태이다. replicas 를 늘리거나 PDB 를 조정한다.

노드 그룹 전략

한 종류의 노드로 모든 워크로드를 감당하지 않는다. **성격별로 나눈다.**

- 노드 분리는 스케줄링 제어와 한 세트
- 앞의 테인트 · 어피니티가 여기서 쓰인다
- 스팟은 종단을 건디는 워크로드에만. 2분 전 통지가 전부이다

노드 그룹	용도	구성	주의
general (온디맨드)	상시 서비스	m 계열 · 여러 AZ	기본값. 테인트 없음
spot	스테이트리스 · 배치	여러 타입 혼합	중단 처리 필수 (graceful)
system	애드온·컨트롤러	작게. 테인트로 격리	일반 워크로드가 못 들어오게
memory (r 계열)	캐시 · 인메모리 처리	테인트 + 어피니티	비싸다. 꼭 필요한 것만
gpu	ML 추론·학습	테인트 필수	가장 비싸다. 유희 시간 최소화
arm64 (Graviton)	비용 최적화	멀티아키 이미지 필요	이미지 빌드 확인

[주의] 스팟 중단 통지는 **2분**이다. `terminationGracePeriodSeconds` 가 그보다 길면 **강제 종료**된다.

백업의 세 가지 구분

세 가지를 구분한다. 매니페스트 · 애플리케이션 데이터 · 클러스터 상태.

- GitOps 를 하면 매니페스트 백업은 이미 되어 있다
- EKS 는 etcd 백업이 필요 없다
- AWS 가 한다. 우리 역할은 데이터와 볼륨 대상이다

대상	무엇	방법	RPO 목표
매니페스트	리소스 정의	Git	0 (커밋 단위)
Secret 원본	시크릿 값	AWS Secrets Manager	0
PV 데이터	볼륨 내용	VolumeSnapshot · Velero	1~24시간
DB 데이터	논리 백업	pg_dump · RDS 자동 백업	5분~1시간
클러스터 상태	네임스페이스 · CRD · 리소스	Velero (스냅샷 + 리소스)	1일
etcd	전체 상태	EKS 는 AWS 가 자동	우리 몫 아님

[주의] 백업은 복원 연습을 해 봐야 백업이다. 한 번도 복원해 보지 않은 백업은 백업이 아니다.

재해 복구 시나리오

복구 목표를 **RTO(얼마나 빨리)**와 **RPO(얼마나 최근까지)** 로 정하고, 거기에 맞춰 설계한다.

- '모든 영역을 빠르게 ' 관점은 비용을 감당하기 어렵다
- 서비스별로 등급을 나눈다
- GitOps + 외부 데이터 저장소면 클러스터 재생성이 가장 빠른 복구가 된다

시나리오	영향	복구 방법	예상 RTO
파드 장애	일부 요청 실패	자동(컨트롤러)	초 단위
노드 장애	그 노드 파드 재배포	자동(5분 40초)	~6분
AZ 장애	그 존 파드·EBS 사용 불가	topologySpread 로 나머지 존	즉시~수분
네임스페이스 삭제 사고	서비스 전체 소실	Argo CD 재동기화 + PV 복원	10~30분
클러스터 손상	전면 중단	새 클러스터 + GitOps + 스냅샷 복원	1~3시간
리전 장애	전면 중단	타 리전 DR 클러스터 (사전 구성 필요)	설계에 따라

[체크] 네 번째 줄은 **연습해 두지 않으면** 실제 상황에서 사용하지 못한다.

requests 와 비용의 관계

EKS 비용의 대부분은 EC2이다. 그리고 그 크기를 정하는 것은 requests 이다.

- 가장 큰 낭비는 **과다 예약**이다. 실사용의 3~10배로 설정된 requests
- 두 번째는 **사용하지 않는 자원**이다. 고아 PVC · 유휴 LB · 미사용 노드

항목	흔한 낭비	확인 방법	절감폭
노드(EC2)	requests 과다 예약	예약률 vs 실사용률	30~60%
노드 유형	온디맨드만 사용	스팟 비율 확인	최대 70%
로드밸런서	서비스마다 LB	<code>kubectl get svc -A</code> LoadBalancer 수	LB 개수만큼
EBS	고아 PVC·볼륨	<code>available</code> 볼륨 목록	볼륨 크기만큼
데이터 전송	AZ 간 트래픽	VPC 플로우 로그	10~20%
로그	과다 수집·긴 보관	CloudWatch 수집량	보관 정책에 따라
미사용 네임스페이스	테스트 후 방치	최근 배포 시각	그만큼

[체크] 첫 줄부터 본다. 노드 비용이 전체의 70~80% 이므로 여기서 30% 를 줄이면 다른 모든 것보다 크다.

비용 점검 절차

한 번 줄이고 끝나는 일이 아니다. 분기 점검 절차로 만든다.

- 아래 순서로 확인하면 30분 안에 주요 낭비가 드러난다

① 예약 대비 실사용 - 가장 큰 항목부터

```
kubectl describe nodes | grep -A5 'Allocated resources'
```

→ PromQL 의 7일 p95 와 비교한다

② 요청량 상위 워크로드 (실사용과 대조할 대상)

```
kubectl get pods -A -o custom-columns=\
'NS:.metadata.namespace,NAME:.metadata.name,\
CPU:.spec.containers[*].resources.requests.cpu'
```

③ 유헤 로드밸런서 . ④ 고아 볼륨

```
kubectl get svc -A --field-selector spec.type=LoadBalancer
kubectl get pv --field-selector=status.phase=Released
aws ec2 describe-volumes --filters Name=status,Values=available --output table
```

⑤ 방치된 네임스페이스 - 최근 배포가 언제인가

```
kubectl get deploy -A --sort-by=.metadata.creationTimestamp | head -20
```

[체크] 이 다섯 줄을 분기 1회 정기 작업으로 등록한다. 사람이 기억해서 하는 일은 결국 안 하게 된다.

용량 계획

지금 어느 정도의 트래픽을 감당하고, **과다한 트래픽 유입에 따라 어느 영역에서 한계에 이르게 되는지** 인지한다.

- 병목은 대개 **애플리케이션이 아니라 의존성**에 있다. DB 커넥션·외부 API 한도가 그것이다
- 쿠버네티스 자체의 한계도 있다. **노드당 파드 수·서브넷 IP·etcd 크기**

한계	언제 부딪히나	확인·대응
노드당 파드 수	작은 파드를 많이 기동할 때	VPC CNI 계산 · 프리픽스 위임
서브넷 IP	파드가 수천 개	클러스터 생성 시 결정 . /19 이상
DB 커넥션	HPA 로 파드가 늘어날 때	파드 수 × 풀 크기 · 프록시 도입
Service 수 (iptables)	1000개 이상	IPVS 전환 검토
etcd 크기	리소스·이벤트 누적	TTL 정리 · 이벤트 보존 기간
로드밸런서 대상 수	파드 수천 개	NLB 타깃 그룹 한도 · Nginx 워커 수
이미지 pull 대역	동시 대규모 스케일 아웃	이미지 크기 줄이기 · 노드 캐시

[체크] 부하 테스트로 확인한다. 계산만으로는 어디가 먼저 한계에 닿는지 알 수 없다. 대개 예상과 다르다.

부록. 트러블슈팅 Case

- 증상에서 시작하는 15개 Case
- 각 **Case** 는 확인 순서와, 그 출력이면 원인이 무엇인지를 짚지어 둔다
- 현장에서는 '몇 장 문제' 라는 표시가 붙어 오지 않는다
- 마지막 **Case** 는 아무 단서도 없을 때의 시작 순서이다

증상에서 시작하는 색인

- 왼쪽 열의 이슈 요청으로 연락이 오면 **오른쪽 Case** 부터 참조할 수 있다.

이렇게 연락이 온다	Case	먼저 볼 것
'배포했는데 안 떠요'	1 · 2 · 3 · 4	<code>describe pod</code> 의 마지막 이벤트
'접속이 안 돼요'	6 · 7 · 8	<code>get endpointslice</code>
'배포가 안 끝나요' · '바뀐 게 없어요'	9 · 10	<code>get rs</code> 와 <code>rollout status</code>
'데이터가 사라졌어요'	11	<code>get pvc</code> 와 <code>describe pvc</code>
'권한이 없대요' · '배포가 거부돼요'	12	<code>auth can-i</code> 와 거부 메시지
'느려요' · '가끔 죽어요'	5 · 13 · 14	지표를 먼저, 로그는 나중에

[정리] 단서가 하나도 없으면 **Case 15** 부터 본다. 어디서 끊겼는지를 먼저 정하는 순서이다.

[Case 1] Pending 정체

증상. 파드가 만들어졌는데 노드에 배치되지 않는다. 컨테이너는 아직 없다.

- 스케줄러가 노드를 고르지 못한 것이다. 이유는 이벤트에 그대로 적혀 있다

이벤트에 이렇게 나오면	원인	조치
<code>Insufficient cpu/memory</code>	요청량이 남은 용량보다 크다	requests 재산정
<code>untolerated taint</code>	테인트가 있는 노드뿐이다	toler레이션 추가
<code>didn't match node affinity</code>	어피니티·셀렉터가 너무 좁다	라벨과 조건 확인
<code>volume node affinity conflict</code>	볼륨이 다른 AZ 에 있다	같은 AZ 또는 EFS
<code>exceeded quota</code>	네임스페이스 몫을 넘었다	요청량 축소

[함정] `Insufficient` 를 보고 노드를 늘리는 조치가 가장 흔하다. 실사용의 3~10배로 예약된 경우가 더 많다.

[Case 2] 이미지 수신 실패

증상. 파드는 노드에 배치됐는데 컨테이너가 시작되지 않는다. 로그가 비어 있다.

- 이미지를 가져오는 단계이고, **메시지 뒷부분**이 원인을 구분해 준다

상태 · 메시지	원인	조치
<code>manifest ... not found</code>	태그 오타 또는 없는 태그	저장소에서 태그를 확인한다
<code>unauthorized · denied</code>	비공개 저장소 자격증명 없음	<code>imagePullSecrets</code> 또는 IRSA
<code>no such host · timeout</code>	레지스트리 주소 · 네트워크	주소와 egress 경로를 확인한다
<code>ErrImageNeverPull</code>	<code>imagePullPolicy: Never</code> 인데 없다	정책 또는 사전 적재 확인
<code>ImagePullBackOff</code>	위 원인의 재시도 대기 상태	원인은 위 네 줄 중 하나이다

[정리] `ErrImagePull` 은 방금 실패, `ImagePullBackOff` 는 재시도를 미루는 중이다. 같은 원인의 다른 시점이다.

[Case 3] 컨테이너 생성 실패

증상. 이미지는 받았는데 컨테이너가 만들어지지 않는다. 상태가 `Create...Error` 이다.

- kubelet 이 컨테이너를 만들기 직전에 막힌 것이다. **앱은 아직 실행되지 않았다**

상태	이벤트에 나오는 것	원인
CreateContainerConfigError	couldn't find key ...	ConfigMap·Secret 키 이름 불일치
CreateContainerConfigError	secret ... not found	참조한 리소스가 없다
CreateContainerError	no such file or directory	command 경로가 이미지에 없다
RunContainerError	permission denied	runAsUser 와 파일 소유권 충돌
Init:...	초기화 컨테이너가 끝나지 않았다	logs -c <init컨테이너> 를 본다

[함정] optional: true 로 넘기면 파드는 뜬다. **설정 없이 동작하는 앱이 더 위험한 경우가 많다.**

[Case 4] 반복 재시작

증상. `CrashLoopBackOff` 이다. 재시작 횟수가 계속 올라간다.

- `Last State` 의 종료 코드와 사유가 가치를 나눈다. 지금 로그가 아니라 **직전 로그**를 본다

Last State	원인	먼저 볼 것
Error · 종료 코드 1	앱이 스스로 종료했다	<code>logs --previous</code> 의 마지막 줄
OOMKilled · 137	메모리 한도 초과	Case 5 로 간다
Error · 종료 코드 137	Probe 실패로 강제 종료	liveness 설정과 기동 시간 비교
Completed · 0	실행할 일이 끝났다	Deployment 가 아니라 Job 이어야 한다
재시작 0 인데 <code>Running</code> 아님	아직 시작 전이다	Case 2 · 3 으로 간다

[정리] `--previous` 없이 `logs` 만 보면 **방금 시작한 컨테이너**의 빈 로그가 나온다. 원인은 직전 로그에 있다.

[Case 5] OOMKilled 와 종료 코드

증상. 로그가 중간에 끊기고 오류 메시지가 없다. 파드가 조용히 사라진다.

- **limits** 를 올리기 전에 런타임이 한도를 알고 있는지 먼저 확인한다

확인	무엇을 보는가	그러면
종료 사유	Last State 의 OOMKilled · 137	커널이 죽인 것이다. 앱 오류가 아니다
사용량 추이	지표의 최대값	평균으로 산정했으면 원인이 그것이다
런타임 인식	JVM 힙 상한이 노드 기준인지	컨테이너 limits 를 모를 수 있다
조치 순서	런타임에 상한을 알린다	-XX:MaxRAMPercentage 등
그래도 부족하면	근거 숫자와 함께 limits 상향	최대값 × 1.3 을 기준으로

[함정] 메모리 requests 를 평균으로 산정하면 반드시 OOMKilled 가 발생한다. CPU 는 분위수, 메모리는 최대값이다.

[Case 6] 이름 해석 실패

증상. 파드 안에서 서비스 이름으로 부르면 실패한다. IP 로는 된다.

- 이름 해석 단계라, **IP 로 되는지 먼저 확인**하면 원인이 절반으로 줄어든다

확인	무엇을 보는가	그러면
IP 로 접속	<code>curl <파드IP>:<포트></code>	되면 DNS 문제로 확정된다
CoreDNS 상태	<code>get pod -n kube-system -l k8s-app=kube-dns</code>	죽어 있으면 클러스터 전체 문제
egress 정책	53/UDP 허용 규칙이 있는가	egress 를 막을 때 가장 흔한 사고
이름 형태	짧은 이름 · FQDN	다른 네임스페이스면 <code><svc>.<ns></code>
검색 지연	<code>ndots</code> 로 인한 불필요한 조회	FQDN 끝에 점을 붙여 확인한다

[함정] NetworkPolicy 를 적용하면 **DNS 도 함께 막힌다**. 증상은 DB 접속 오류처럼 보이지만 실제로는 이름 해석 실패이다.

[Case 7] 해석은 되나 연결 실패

증상. `nslookup` 은 ClusterIP 를 돌려준다. 그런데 연결은 되지 않는다.

- 첫 명령은 `kubectl get endpointslice` 이다. 비어 있는지 아닌지로 가지가 갈린다

엔드포인트	다음 확인	원인
비어 있다	Service 셀렉터와 파드 라벨	값이 다르면 대상이 0개가 된다
비어 있다	<code>get pod</code> 의 <code>READY</code> 열	<code>0/1</code> 이면 readiness 미통과
차 있다	<code>port-forward</code> 로 파드 직접 호출	응답하면 앱은 정상이다
차 있다	<code>port · targetPort · containerPort</code>	세 숫자 중 하나가 어긋났다
차 있다	NetworkPolicy ingress	대상 파드 쪽에서 막고 있다

[정리] 라벨 셀렉터 오타는 오류를 내지 않는다. 대상이 0개가 될 뿐이라 조용히 실패한다.

[Case 8] 외부 502 · 504

증상. 클러스터 안에서는 되는데 인터넷에서 부르면 오류가 온다.

- NLB 는 L4 라서 5xx 를 만들지 않는다. 클라이언트가 본 502·504 는 전부 Nginx 나 앱이 낸 것이다

확인	무엇을 보는가	그러면
안쪽 확인	클러스터 안에서 ClusterIP 호출	되면 Case 7 은 아니다
타깃 등록	NLB 대상 그룹의 상태	<code>unhealthy</code> 면 헬스체크 경로·포트
타깃 종류	Nginx 컨트롤러 파드가 살아 있는가	<code>kubectl logs</code> 로 502 가 Nginx 에서 났는지 본다
보안그룹	노드·ENI 인바운드	LB 에서 오는 트래픽이 막혔는가
간헐적 504	NLB idle(350s) 과 앱 keep-alive	앱 쪽이 더 짧으면 간헐 오류

[함정] 간헐적 502 의 대표 원인은 앱 keep-alive 가 NLB idle timeout(350초) 보다 짧은 것이다. 재현이 어려워 오래 끈다.

[Case 9] 롤아웃 정체

증상. `rollout status` 가 오래 걸린다. 옛 파드와 새 파드가 섞여 있다.

- 새 파드가 안 뜨는 것과 옛 파드를 못 내리는 것은 원인이 다르다. 먼저 가른다

확인	무엇을 보는가	원인
새 ReplicaSet	<code>DESIRED</code> · <code>CURRENT</code> · <code>READY</code>	숫자가 어디서 멈췄는지 보인다
새 파드 상태	<code>READY</code> 가 오르지 않는다	readiness 또는 Case 1~4
옛 파드 이벤트	<code>Cannot evict pod ... budget</code>	PDB 가 축출을 막고 있다
PDB 와 전략	<code>minAvailable</code> 과 <code>maxUnavailable</code>	둘이 충돌하면 교체가 멈춘다
쿼터	<code>exceeded quota</code>	새 파드를 만들 몫이 없다

[함정] `minAvailable` 을 replicas 와 같게 두면 한 개도 내릴 수 없어 롤아웃이 영원히 멈춘다. 가장 흔한 구성 실수이다.

[Case 10] 변경 미반영

증상. 배포도 했고 오류도 없다. 그런데 옛 동작 그대로이다.

- 무엇을 바꿨는가에 따라 원인이 갈린다. 이미지인지 설정인지 먼저 정한다

무엇을 바꿨나	확인	원인
이미지 (같은 태그)	<code>get rs</code> 에 새 것이 없다	템플릿이 같아 롤아웃이 없었다
이미지 (같은 태그)	이벤트가 <code>already present</code>	<code>IfNotPresent</code> 로 옛 이미지 재사용
ConfigMap 환경변수	파드 <code>AGE</code> 가 그대로	환경변수는 기동 시 한 번만 읽는다
ConfigMap 볼륨	약 60초 뒤 파일은 바뀐다	앱이 다시 읽지 않으면 그대로이다
<code>subPath</code> 마운트	파일이 영영 안 바뀐다	<code>subPath</code> 는 갱신되지 않는다

[정리] 재발을 막는 방법은 두 가지이다. **태그에 커밋 해시**를 넣고, 설정 해시를 애노테이션에 넣어 롤아웃을 유발한다.

[Case 11] 볼륨 연결 실패

증상. PVC 가 `Pending` 이거나, 파드가 볼륨을 기다리며 시작하지 않는다.

- PVC `Pending` 이 항상 오류는 아니다. 기다리는 것이 정상인 설정이 있다

증상	확인	원인
PVC <code>Pending</code>	StorageClass 바인딩 모드	<code>WaitForFirstConsumer</code> 면 정상
PVC <code>Pending</code>	<code>describe pvc</code> 이벤트	StorageClass 없음 또는 프로비저너 오류
Multi-Attach error	접근 모드와 replicas	RWO 는 한 노드에만 붙는다
파드가 다른 AZ	<code>volume node affinity conflict</code>	EBS 는 AZ 를 넘지 못한다
확장이 안 됨	<code>allowVolumeExpansion</code>	false 면 용량을 늘릴 수 없다

[정리] `Retain` 이면 PVC 를 지워도 실제 볼륨은 남는다. 복원은 원본을 덮어쓰지 않고 새 PVC 로 한다.

[Case 12] 권한 거부와 배포 거부

증상. 403 Forbidden 이 나거나, 배포가 Admission 단계에서 거부된다.

- 누가 무엇을 하려다 막혔는지를 먼저 정한다. 사람인지 파드인지가 다르다

막힌 주체	확인	원인
내 kubectl	<code>auth can-i <동사> <리소스></code>	Role·ClusterRole 범위 밖
파드 안 API 호출	<code>can-i --as=</code> 로 그 SA 를 흉내 낸다	SA 에 RoleBinding 이 없다
파드의 AWS 호출	파드 안 <code>AWS_ROLE_ARN</code> 환경변수	IRSA 미적용 또는 재시작 전
배포 자체가 거부	거부 메시지의 규칙 이름	PSS · Admission 정책 위반
일부만 거부	네임스페이스 라벨	<code>enforce</code> 수준이 다르다

[함정] IRSA 는 파드를 다시 만들어야 적용된다. 역할을 연결하고 바로 확인하면 옛 자격증명이 그대로 보인다.

[Case 13] 노드 이상

증상. 노드가 **NotReady** 이거나, 파드가 이유 없이 다른 노드로 옮겨진다.

- 노드 문제는 서비스가 멀쩡해 보일 때 더 위험하다. 상태 보고가 함께 멈추기 때문이다

증상	확인	의미
노드 NotReady	그 노드 파드가 Running 으로 보인다	상태 보고가 멈춘 것이다
파드가 Evicted	<code>describe node</code> 의 Conditions	DiskPressure · MemoryPressure
디스크 압박	이미지와 로그가 노드를 채웠다	이미지 정리와 로그 회전
파드가 옮겨짐	NoExecute 테인트 자동 부여	kubelet 이 응답하지 않는다
일부만 영향	그 노드의 파드 목록	분산이 안 돼 있었다면 전면 장애

[함정] kubelet 이 멈추면 컨테이너는 계속 실행되지만 **Probe** 도 함께 멈춘다. 중단돼도 교체되지 않는다.

[Case 14] 간헐적 지연

증상. '가끔 느려요' 라는 문의가 반복된다. 재현이 안 되고 언제인지도 모른다.

- 지표로 범위를 좁히고, 추적으로 구간을 찾고, 로그로 확인한다. 로그부터 뒤지면 오래 걸린다

단계	무엇을 보는가	그러면
범위	p95 · p99 를 파드별로 분해	특정 파드인지 전체인지 갈린다
한 파드만 느림	그 파드의 CPU 스로틀링	limits 가 낮아 제한받고 있다
전체가 느림	의존 대상 응답 시간	DB · 외부 API 구간을 본다
주기적으로 느림	배치 · CronJob 실행 시각	같은 노드에서 자원을 다툰다
확인	해당 시간대 로그만	여기서 처음 로그를 본다

[정리] '가끔 느려요' 를 알림 규칙 한 줄로 바꿔 적을 수 있어야 끝난다. 예: `p99 > 1s` 가 5분 지속.

[Case 15] 단서가 없을 때의 순서

증상. '뭔가 안 돼요' 만 들어왔다. 어디서부터 볼지 정해야 한다.

- 끊긴 지점을 특정하기 전에 아무것도 수정하지 않는다. 수정해서 나아지면 원인을 모른 채 넘어간다

순서	명령	여기서 갈리면
1	<code>kubectl get pod</code> 의 STATUS 와 READY	Case 1~5 로 간다
2	<code>kubectl get endpointslice</code>	Case 6~8 로 간다
3	<code>kubectl get rs</code> 와 <code>rollout status</code>	Case 9 · 10 으로 간다
4	<code>kubectl get events --sort-by=.lastTimestamp</code>	최근 10분에 무슨 일이 있었는지
5	그래도 모르면 지표부터	Case 14 의 순서를 따른다

[정리] 산출물은 증상 · 원인 · 근거 세 줄이다. '어쩌다 고쳐졌다' 가 아니라 다음에 재현할 수 있는 진단이어야 한다.



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.